第3章: React の基礎

この章では、React（リアクト：Meta社が開発したUI構築用JavaScriptライブラリ）の基本的な概念をゼロから学びます。React は**ユーザーインターフェース（UI：User Interface。ユーザーが見て操作する画面部分の総称）** を効率的に作るためのライブラリ（Library：特定の機能を提供するプログラム部品の集まり。自分のコードから呼び出して使う）です。

この章を読む前に頭に入れておきたいこと:

- **JSX**（ジェイエスエックス：JavaScript XMLの略）は、JavaScript の中に HTML 風の構文を書ける「拡張構文」です。ブラウザは JSX を直接理解できないため、ビルドツール（Vite, Babel など）が裏で `React.createElement(...)` という関数呼び出しに変換しています。つまり JSX は「見た目が HTML だけど、実体は JavaScript の関数呼び出し」と覚えてください。
- **コンポーネント**（Component：UIの部品）は、画面の一部を表す関数です。「JSX を返す関数」を作って、それを別の関数から `<MyComponent />` のように呼び出すだけで、UI を組み立てられます。
- **state**（ステート：状態）の値は **直接書き換えてはいけません。** React は state の更新を「参照が変わったかどうか」で判定するため、`obj.name = "..."` のような直接代入では再レンダリング（再描画）が起こりません。必ず `setXxx(...)` という更新関数を呼びます。
- **Strict Mode**（ストリクトモード：開発時の不具合検出モード）が有効だと、開発中はコンポーネントの関数が**わざと2回呼ばれます。**「あれ、console.log が2回出る？」と驚くかもしれませんが、これは仕様です。本番ビルドでは1回しか呼ばれません。

この章で学ぶこと

概念	一言で言うと	身近な例え
コンポーネント	画面の部品	レゴブロック。小さな部品を組み合わせてページを作る
JSX / TSX	HTMLのようにUIを書く構文	HTMLとJavaScriptを混ぜて書ける魔法の書き方
Props	親から子へのデータ受け渡し	上司から部下への指示書。「このデータを使って表示して」
State	コンポーネントの内部データ	変化する黒板。書き換えると画面が自動で更新される
イベント	ユーザー操作への反応	ボタンを押す→何かが起きる、という仕組み
useEffect	画面描画以外の処理	画面が表示された後に行う「裏方の仕事」（API通信など）
カスタムフック	共通ロジックの切り出し	よく使う手順をマニュアルにまとめておくこと

> **Reactを学ぶ前に：** React は「宣言的UI」（Declarative UI：宣言的ユーアイ。「最終的にこう見えてほしい」を書くスタイル）という考え方を採用しています。これは「画面がどうあるべきか」を記述するスタイルで、jQuery（ジェイクエリー：古くからあるDOM操作用JavaScriptライブラリ）のように「画面のどこをどう操作するか」を1ステップずつ記述する「命令的UI」（Imperative UI：めいれいてきユーアイ）とは大きく異なります。最初は戸惑うかもしれませんが、慣れると非常に直感的に感じるようになります。
>
> 例えるなら、命令的UIは「友達に道を教えるとき、最初の交差点を右、次の信号を左、3軒目の家…」と1手ずつ指示する方法。宣言的UIは「○○駅前の郵便局」と最終目的地だけ伝え、行き方はカーナビ（=React）に任せる方法です。

この章のゴールと地図

この章はとても長いので、最初に「どこへ向かうのか」を確認しておきましょう。山登りの前に登山マップを見ておくのと同じで、「今どこにいて、頂上まであとどれくらいか」が分かっていると、途中で迷子になりません。

この章を読み終えると、次のことができるようになります。

1. JSX で画面の見た目を書き、`{ }` で変数や計算結果を埋め込める
2. 画面を部品（コンポーネント）に分け、props でデータを親から子へ渡せる
3. `useState` で「変わる値」を持ち、ボタンや入力欄の操作で画面を更新できる
4. フォームの入力を state と同期させ（制御コンポーネント）、送信時に入力チェックができる
5. `useEffect` が「いつ」実行されるかを説明しながら使える
6. よくある間違い（state の直接変更・`onClick={f()}`・key の付け忘れ・依存配列の書き忘れ）を見つけて直せる

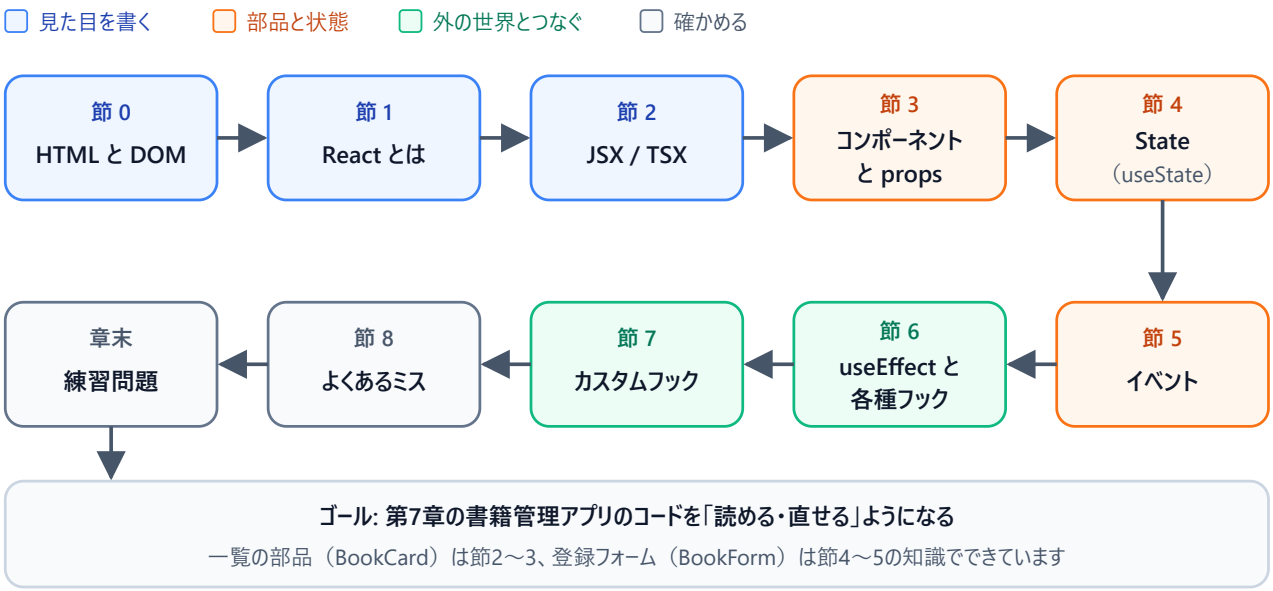


図3-1: この章の学習地図。上の段を左から右へ、下の段を右から左へ進みます

この章の知識が、このあとどこで使われるか:

この章の節	学ぶこと	このあと登場する場所
2〜3（JSX・コンポーネント・props）	見た目の書き方、部品への分割、データの受け渡し	第7章の一覧画面の部品（`BookCard`・`RatingStars`・`StatusBadge` など）
4〜5（state・イベント）	変わる値の保持、入力フォーム、送信	第7章の登録フォーム `BookForm`、第8章の編集・削除ボタン
6〜7（useEffect・各種フック）	表示したあとに動く処理、後片付け、ロジックの再利用	タイマーやイベント監視など「ブラウザでしかできない処理」（データ取得の主役は第4章で学ぶ Server Component）
8（よくあるミス）	バグの見つけ方と直し方	エラーが出たときの原因探し全般

この章の読み進め方: 各サンプルは「**▼ このコードがやること（日本語の説明）** → **コード** → **実行結果** → **1つずつ分解した解説**」の順に並んでいます。最初はコードを細かく読まず、まず日本語の説明と実行結果だけを読んで「何が起きるのか」をつかみ、そのあとで解説を読むと理解しやすくなります。途中の **つまずきポイント** は、初心者が実際によくハマる失敗を「NG（ダメな書き方）→ OK（正しい書き方）」で並べたものです。

> **ヒント（この章のコードの動かし方）：** `▼ 実行結果` が付いている `.ts` ファイルのサンプルは、Node.js（v22.18 以降。本書では v25 で確認）で `node ファイル名.ts` と実行すると、同じ結果が出ます（Node.js が型注釈を取り除いてから実行してくれます）。`react` を `import` しているサンプルは、空のフォルダで `npm init -y` → `npm pkg set type=module` → `npm install react react-dom` を実行してから試してください。画面を持つ `.tsx` のコンポーネントは、第6章で作る Next.js プロジェクトに `app/practice/page.tsx` のようなファイルを作り、1行目に `"use client";` と書き、試したいコンポーネントに `export default` を付けて貼り付けると、`http://localhost:3000/practice` で動きを確認できます（`"use client"` の意味は第4章で学びます）。

目次

- この章のゴールと地図

0. 前提知識: HTMLとDOMの超基礎
1. React とは
2. JSX / TSX
3. コンポーネント
4. State (useState)
5. イベントハンドリング

6. useEffect
    - 6.6 useRef（値の保持・要素のフォーカス）
    - 6.7 useMemo（重い計算結果のメモ化）
    - 6.8 useCallback（関数のメモ化）
    - 6.9 useContext（Propsのバケツリレー回避）
    - 6.10 useReducer（複数のstateをまとめる）
7. カスタムフック
8. よくあるミスと対処法
9. 練習問題（解答・解説つき）

---

0. 前提知識: HTMLとDOMの超基礎

React は「画面を作る」ための仕組みです。「画面」とは内部的には **HTML** で記述された文書です。React のコードを読む前に、HTML と DOM について最低限おさえておきましょう。

0.1 HTMLって何？

HTML（HyperText Markup Language）は、**Webページの構造を記述する言語**です。`<タグ名>...</タグ名>` という形で、文章のどの部分が「見出し」で、どの部分が「段落」で、どこに「リンク」があるか、をコンピュータに教えます。

```
<!-- ブラウザに「これはHTML5の文書です」と宣言する一行。HTMLファイルの先頭に必ず書く -->
<!DOCTYPE html>
<!-- ページ全体の最も外側のタグ。<html>...</html> の中にすべてが入る -->
<html>
  <!-- 文書の「メタ情報（タイトル・文字コード・読み込むCSS等）」を入れる場所。画面には表示されない -->
  <head>
    <!-- ブラウザのタブ部分に表示される文字 -->
    <title>はじめてのページ</title>
  </head>
  <!-- 画面に実際に表示される「本文」を入れる場所 -->
  <body>
    <!-- 一番大きな見出し（heading 1）。1ページに1つだけ書くのが原則 -->
    <h1>こんにちは</h1>
    <!-- <p> は paragraph（段落）の略。通常の文章ブロック -->
    <p>これは段落（paragraph）です。</p>
    <!-- <a> はアンカー（anchor）。href 属性で飛び先URLを指定するクリック可能なリンク -->
    <a href="https://example.com">リンク</a>
  </body>
</html>
```

▼ ブラウザで表示すると：

```
┌──────────────────────────────┐
│ こんにちは                   │ ← <h1> の部分が大きな見出し
│                              │
│ これは段落（paragraph）です。 │ ← <p> の部分は普通の段落
│                              │
│ リンク                       │ ← <a> の部分は青い下線付きリンク
└──────────────────────────────┘
```

0.2 よく出てくるHTMLタグ

タグ	意味	サンプル
`<div>`	汎用ブロック（箱）	`<div>...</div>`
`<span>`	汎用インライン（行内）	`<span>太字</span>`
`<h1>` 〜 `<h6>`	見出し（h1が一番大きい）	`<h1>タイトル</h1>`
`<p>`	段落	`<p>本文</p>`
`<a>`	リンク	`<a href="...">テキスト</a>`
`<ul>` `<li>`	順序なしリスト	`<ul><li>項目</li></ul>`
`<button>`	ボタン	`<button>OK</button>`
`<input>`	入力欄	`<input type="text" />`
`<form>`	フォーム	`<form>...</form>`
`<img>`	画像	`<img src="..." alt="..." />`

タグには **属性**（attribute） を付けられます。`<a href="https://example.com">` の `href="..."` の部分が属性です。

0.3 DOM（Document Object Model）

ブラウザは HTML を読み込むと、それを**ツリー構造のオブジェクト**として記憶します。これが **DOM** です。

```
html
 └─ body
      ├─ h1 ("こんにちは")
      ├─ p  ("これは段落です。")
      └─ a  ("リンク") [href="https://example.com"]
```

「DOM を操作する」とは、JavaScript からこのツリーを読み書きして画面を変更することです（例: `element.textContent = "新しい文字"` で文字を書き換える、`element.style.color = "red"` で色を変える、など）。React は「**DOM 操作を直接書かなくて済むようにする**」ためのライブラリ、と言い換えてもOKです。React に「画面はこういう状態であってほしい」と JSX で宣言すれば、内部で必要な DOM 操作を裏で勝手にやってくれます。

0.4 ブラウザの開発者ツールで触ってみる

Chrome や Edge で右クリック →「検証（Inspect）」を選ぶと、開発者ツール（DevTools）が開きます。「Elements」タブが今そのページの DOM ツリーです。**Console** タブで JavaScript を実行できます。

```
// Consoleタブで実行
// document はブラウザが用意している「ページ全体を表すオブジェクト」。常にグローバルに存在する。
// ▶ ページタイトル（<title>タグの中身）を取得。文字列が返る
document.title
// querySelector はCSSセレクタで要素を1つ取得するメソッド。"h1" は「h1タグを探す」という意味。
// .textContent は「その要素のテキスト部分だけ」を取り出すプロパティ。
// ▶ h1の文字を取得
document.querySelector("h1").textContent
```

> **これだけ覚えれば次に進める:** HTMLは「タグで文書の構造を書くもの」、DOMは「ブラウザがそれを読み込んだ後のツリー状のデータ」、React は「DOMを直接いじらず、JSX という書き方で画面を宣言するライブラリ」。これだけ頭に入れて先へ進みましょう。

---

1. React とは

1.1 コンポーネントベースの考え方

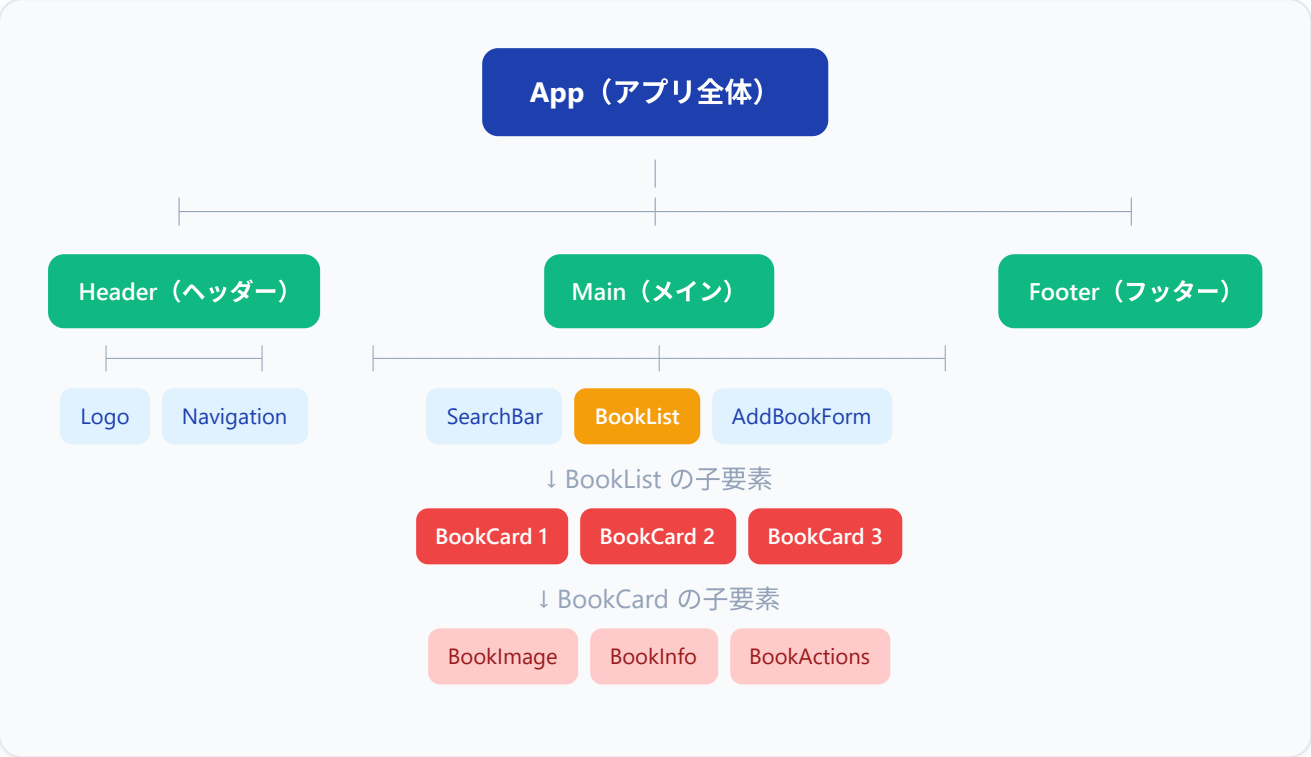
React は **Facebook（現 Meta）** が開発した、ユーザーインターフェース（UI：画面）を構築するための JavaScript ライブラリです。2013年にオープンソース（誰でも無料で使える形）として公開され、現在では世界で最も使われているUIライブラリです。

React の最大の特徴は「**コンポーネント**」（Component：コンポーネント。UIの部品。ボタン、カード、ヘッダーなど、画面を構成する一つひとつの要素。React では「JSX を返す関数」として書く）という考え方です。

> **なぜReactが人気なの？** 従来のWeb開発では、HTMLファイル全体を書き換える必要がありました。React では、変化した部分だけを効率的に更新できるため、**高速で滑らかな画面更新**が可能です。また、一度作った部品（コンポーネント）を使い回せるため、開発効率も大幅に向上します。

コンポーネントとは、UI の一部分を独立した再利用可能なパーツとして定義したものです。レゴブロックのように、小さなパーツを組み合わせて画面全体を構築します。

例えば、書籍管理アプリを考えてみましょう。



このように、画面を「コンポーネントの木構造（ツリー）」として捉えます。それぞれのコンポーネントは：

- **独立している**: 自分の状態（state）とロジックを持つ
- **再利用できる**: `BookCard` は何度でも使い回せる
- **組み合わせできる**: 小さなコンポーネントを組み合わせて大きなコンポーネントを作る

上の木構造を、実際のブラウザ画面に重ねてみると次のようになります。色付きの枠1つ1つが「コンポーネント」です。

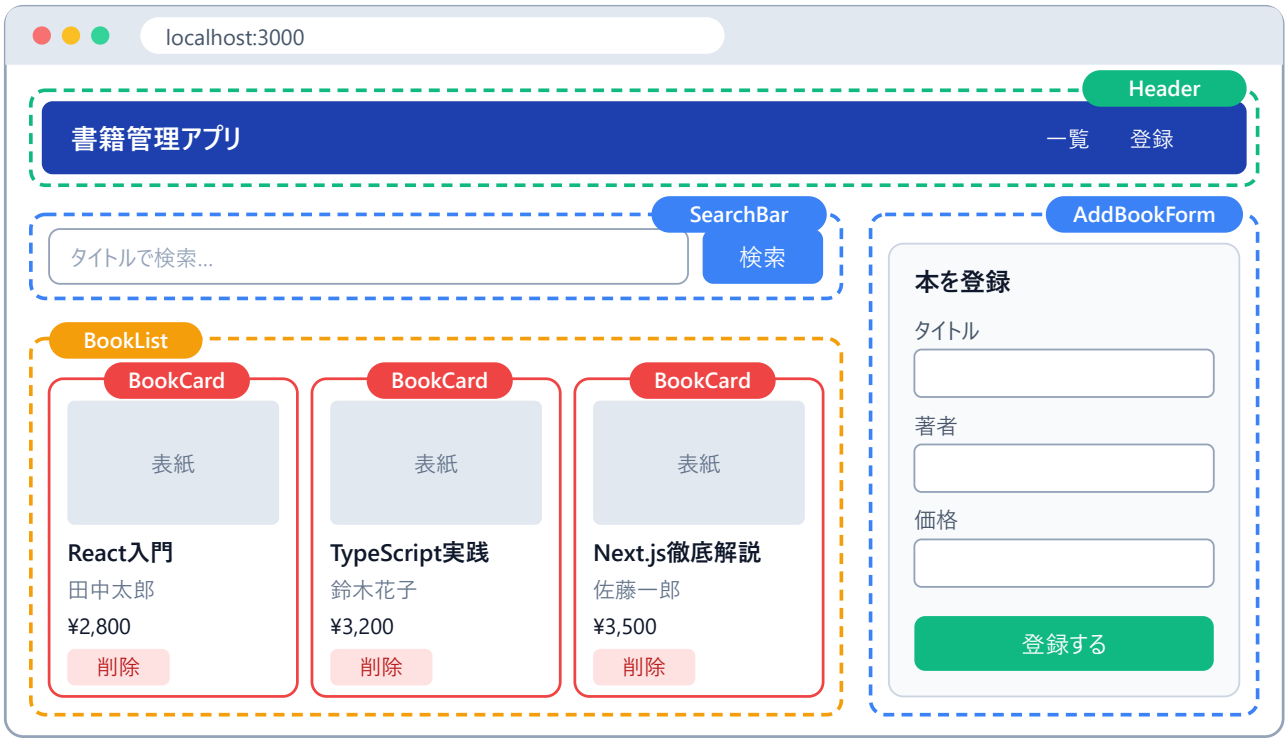


図3-2: 書籍管理アプリの画面イメージ。点線や色枠で囲んだ1つ1つがコンポーネント（部品）です

画面をよく見ると、`BookCard` が3つ並んでいます。これは「本のカード」という**1つの設計図（コンポーネント）** に、3冊それぞれ違うデータ（タイトル・著者・価格）を渡して、3回使っているからです。データの渡し方（props）は 3.2 で学びます。

> **ヒント（画面をコンポーネントに分けるコツ）**：完成した画面のスクリーンショットに、ペンで枠を描き込んでみましょう。枠を描く目安は2つです。①**同じ形がくり返されている所**（本のカードが何枚も並ぶ、など）、②**役割に名前を付けられる所**（「検索欄」「登録フォーム」など）。枠に付けた名前が、そのままコンポーネント名の候補になります。

1.2 仮想DOM の仕組み

ブラウザが持つ **DOM（Document Object Model）** を直接操作すると、非常に遅くなります。React は「仮想DOM（Virtual DOM）」という仕組みを使って、この問題を解決します。



もう少し詳しく見てみましょう。



ポイント: 仮想DOM は JavaScript のオブジェクトに過ぎません。JavaScript オブジェクトの比較は、実際の DOM 操作よりもはるかに高速です。React がこの差分を計算し、必要最小限の DOM 操作だけを行うことで、パフォーマンスが向上します。

1.3 なぜ React を選んだのか

フロントエンドのフレームワーク/ライブラリには多くの選択肢があります。

特徴	React	Vue	Angular	Svelte
学習コスト	中	低	高	低
エコシステム	非常に大きい	大きい	大きい	成長中
求人数	非常に多い	多い	多い	少なめ
TypeScript対応	良好	良好	標準搭載	良好
コミュニティ	最大級	大きい	大きい	成長中

React を選ぶ理由:

1. **巨大なエコシステム**: npm パッケージ、ツール、ライブラリが圧倒的に多い
2. **求人需要**: 日本でもグローバルでも、React エンジニアの需要は非常に高い

3. **学んだ知識の応用範囲が広い**: React Native（モバイルアプリ）、Next.js（フルスタック）など
4. **TypeScript との親和性**: 型安全な開発が自然にできる
5. **豊富な学習リソース**: 公式ドキュメント、書籍、動画、コミュニティが充実

---

2. JSX / TSX

2.1 JSX とは何か

JSX（JavaScript XML：ジェイエスエックス） は、JavaScript の中に HTML のようなコードを書ける構文拡張（言語の文法を拡張したもの）です。TypeScript で使う場合は **TSX**（TypeScript XML）と呼びます（ファイル拡張子が `.tsx`）。

JSX は実際にはブラウザが直接理解できるものではなく、ビルドツール（Vite, Webpack, Babel など、ソースコードを実行可能な形式に変換するツール）によって通常の JavaScript に変換されます。変換結果は「`React.createElement()` という関数を呼ぶだけのコード」になります。つまり JSX の `<h1>...</h1>` は内部的には「React 用のオブジェクトを作る関数呼び出し」に過ぎません。

```
// JSX で書いたコード
// const = 再代入できない変数を宣言するキーワード（JavaScript ES2015以降）
// element = 変数名（自分でつけた名前）
// <h1>こんにちは、React！</h1> = JSX。HTML そっくりだが、これは JS 式（オブジェクト）。
const element = <h1>こんにちは、React！</h1>;

// ↓ ビルドツールによって変換される ↓

// 実際に実行される JavaScript コード
// React.createElement(タグ名, 属性オブジェクト, 子要素...) の形で React 要素を作る。
//   第1引数 "h1": どのタグを作るか
//   第2引数 null: 属性（className や onClick など）。今回はないので null
//   第3引数 "こんにちは、React！": 子要素（タグの中身）
const element = React.createElement("h1", null, "こんにちは、React！");
```

JSX のおかげで、UI の構造を直感的に記述できます。なお、ブラウザがコードを動かす前に「JSX → JavaScript」への変換が必ず走っている、というのは重要なポイントです。エラーメッセージや devtools の表示で `createElement` が出てきても驚かないようにしましょう。

JSX が画面に表示されるまでの流れを、絵で確認しておきましょう。たとえるなら、JSX は「料理の注文票」です。注文票（JSX）は厨房向けの伝票（`createElement` の呼び出し）に書き直され、それをもとに「料理の設計メモ」（React 要素）が作られ、最後にコック（React DOM）が実際の料理（画面の DOM）を作ります。

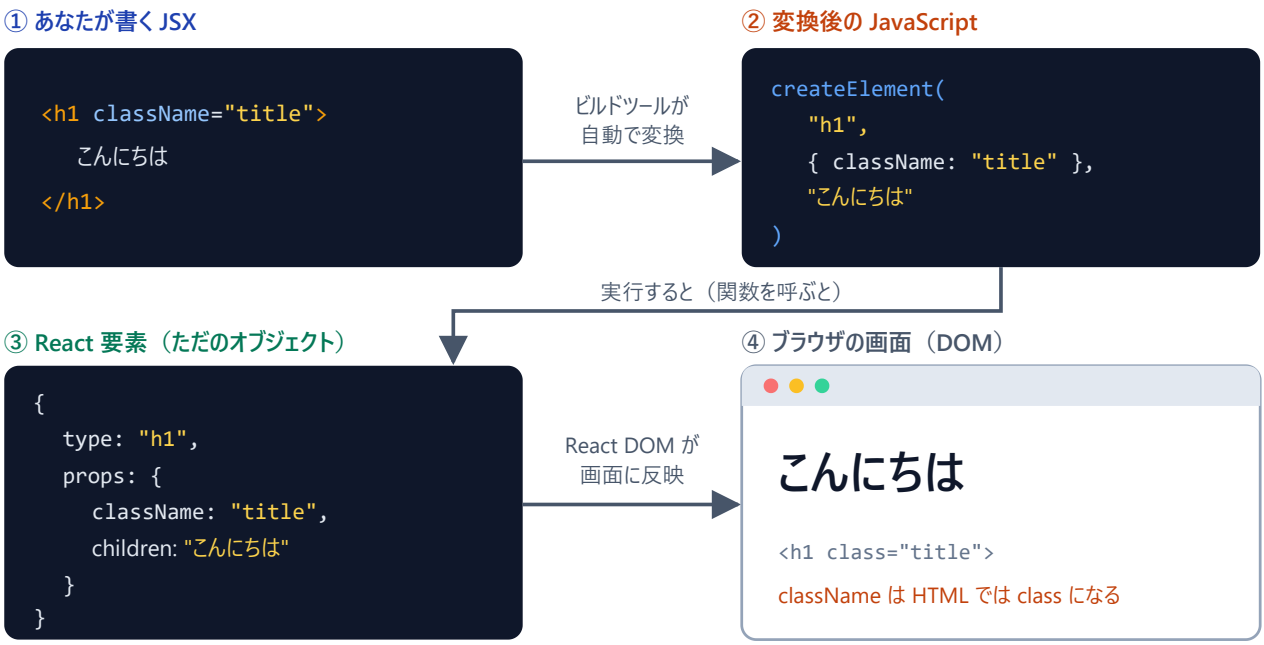


図3-3: JSX が画面に表示されるまで。JSX は関数呼び出しに変換され、その結果の「オブジェクト」を React が DOM に反映します

JSX の正体を Node.js で確かめる

> **▼ このコードがやること（先に日本語で）**：JSX を使わずに、変換後の書き方（ `createElement` ）を直接呼び出して「React 要素」を作り、その中身を表示します。さらに、その React 要素から HTML の文字列を作ってみます。「JSX は見た目こそ HTML だが、正体は**ただの JavaScript のオブジェクト**」ということを、自分の目で確かめるのが目的です。

```ts
// ファイル名: jsx-object.ts
// React 本体から createElement（JSX の正体）と Children を取り込む
import { createElement, Children } from "react";
// React 要素を HTML の文字列に変換する関数を取り込む
import { renderToStaticMarkup } from "react-dom/server";

// (1) <h1 className="title">こんにちは</h1> と同じ意味の呼び出し
const heading = createElement("h1", { className: "title" }, "こんにちは");

// React 要素の中身をのぞいてみる
console.log("type :", heading.type);
console.log("props:", heading.props);

// (2) 入れ子の JSX は、createElement の入れ子になる
// <ul><li>りんご</li><li>バナナ</li></ul> と同じ意味
const list = createElement(
  "ul",
  null,
  createElement("li", { key: "a" }, "りんご"),
  createElement("li", { key: "b" }, "バナナ"),
);
console.log("ul の子要素の数:", Children.count(list.props.children));

// (3) React 要素（ただのオブジェクト）から HTML を作る
console.log(renderToStaticMarkup(heading));
console.log(renderToStaticMarkup(list));
```

▼ **実行方法**（空のフォルダで実行します。最初の3行は1回だけで OK）

```
npm init -y
npm pkg set type=module
npm install react react-dom
node jsx-object.ts
```

▼ **実行結果**

```
type : h1
props: { className: 'title', children: 'こんにちは' }
ul の子要素の数: 2
<h1 class="title">こんにちは</h1>
<ul><li>りんご</li><li>バナナ</li></ul>
```

▼ コードを1つずつ分解して解説

解説1: createElement で React 要素を作る

```
const heading = createElement("h1", { className: "title" }, "こんにちは");
```

- `createElement(タグ名, 属性のオブジェクト, 中身)` は、JSX の `<h1 className="title">こんにちは</h1>` をビルドツールが変換した**あとの姿**です。
- 戻り値の `heading` は画面そのものではなく、「h1 を作ってほしい」という**設計メモ（React 要素）** です。この時点では、まだどこにも表示されていません。

> **用語: React 要素（React element）** ...「どのタグを、どんな属性と中身で作るか」を書いただけのオブジェクト。JSX を書くと、この React 要素が作られる。

解説2: React 要素の中身は type と props

```
console.log("type :", heading.type);
console.log("props:", heading.props);
```

- `type` には `"h1"` というタグ名、`props` には属性（`className`）と中身（`children`）がまとめて入っています。
- タグで囲んだ中身が `children` という名前で `props` に入る点に注目してください。これが 3.2 で学ぶ **children props** の正体です。
- 実際の React 要素には `key` などほかの項目もありますが、まずは「`type` と `props` を持つオブジェクト」と覚えれば十分です。

解説3: React 要素から HTML ができる

```
console.log(renderToStaticMarkup(heading));
```

- `renderToStaticMarkup` は、React 要素（設計メモ）を HTML の文字列に変換する関数です（Next.js がサーバーで HTML を作るときも、似た仕組みが動いています）。
- 結果の `<h1 class="title">` を見ると、JSX で書いた `className` が HTML の `class` に変わっています。JSX で `class` ではなく `className` と書く理由（2.2 で説明）が、ここで確かめられます。
- `list` では `key` を渡しましたが、HTML には出てきません。`key` は React が内部で使うだけの目印だからです（8.3 で詳しく学びます）。

> **用語: レンダリング（rendering）** … React 要素から実際の画面（DOM や HTML）を作ること。

2.2 基本的な構文

最小の React コンポーネント

> **▼ このコードがやること（先に日本語で）**：「画面に "Hello, World!" という見出しを1つ表示するだけ」の、**最小のコンポーネント**を作ります。ポイントは2つだけ——①コンポーネントは「**JSX（見た目）を `return` で返す関数**」であること、②その関数名は **大文字で始める**こと（Reactが「これは部品だ」と判断する目印）。この2つさえ守れば、どんなに小さくてもコンポーネントになります。

```
// ============================================================================
// 一番小さい React コンポーネントの例
// ============================================================================
// function = JS の関数宣言キーワード
// App      = 関数（コンポーネント）の名前。React では「大文字で始まる関数」が
//            自動的にコンポーネントとして扱われる。小文字始まりだと普通の関数。
// ()       = 引数の括弧（このコンポーネントは引数を受け取らないので空）
// { ... } = 関数の本体ブロック
function App() {
  // return = この関数が返す値を指定するキーワード
  // <h1>...</h1> は JSX。HTMLそっくりだが、内部では React.createElement に変換される。
  return <h1>Hello, World!</h1>;
}
```

> **▼ ブラウザでの表示:**
>
> 画面には「**Hello, World!**」という大きな見出しが1行表示されます。
>
> # はHTMLで一番大きな見出しを意味するため、文字サイズが大きく、太字になります。

▼ コードを1つずつ分解して解説

解説1: 関数を定義する

```
function App() {
```

- `function` （ファンクション）は「**関数を作る**」というJavaScriptのキーワードです。

- `App` がこの関数（コンポーネント）の名前です。Reactでは「**大文字で始まる関数**」を自動的にコンポーネントとして扱います。小文字始まりだと普通の関数になってしまいます。
- `()` は引数を受け取る括弧ですが、このコンポーネントは何も受け取らないので空です。

> **用語: コンポーネント（Component）** … 画面の部品。Reactでは「JSXを返す関数」のこと。

解説2: JSXを return で返す

```
  return <h1>Hello, World!</h1>;
```

- `return`（リターン）は「**この関数が返す値**」を指定するキーワードです。コンポーネントは必ず「見た目（JSX）」を return で返します。
- `<h1>Hello, World!</h1>` は見た目を表すJSXです。HTMLそっくりですが、内部では `React.createElement(...)` という関数呼び出しに変換されます。
- これだけで「画面にHello, World!と表示する部品」が完成します。コンポーネントの最小形です。

> **用語: JSX** … JavaScriptの中にHTML風の見た目を書ける構文。実体はJavaScriptの式（値になるもの）。

複数の要素を返す場合

JSX では、**必ず1つのルート要素** で囲む必要があります。

```
// ============================================================================
// ❌ NG例 – 複数の要素を「並べただけ」では返せない
// ============================================================================
// JSX は内部的には1個のオブジェクトを返す JavaScript 式に変換される。
// 並列に複数の要素を書くと「どれが戻り値？」と JS が判断できずエラーになる。
function App() {
  return (
    // ← 1つ目のルート要素
    <h1>タイトル</h1>
    // ← 2つ目のルート要素 → ❌ エラー
    <p>本文</p>
  );
}
// ▼ エラー
// JSX expressions must have one parent element.
```

```
// ============================================================================
// ✅ 解決策1: <div> で1つに包む
// ============================================================================
function App() {
  return (
    // (1) ルート要素は1つ。<div> でまとめる。
    //     JSX のインデントは見やすさのためで、必須ではない。
    <div>
      {/* (2) <div> の中の子要素1つ目 */}
      <h1>タイトル</h1>
      {/* (3) <div> の中の子要素2つ目 */}
      <p>本文</p>
    </div>
  );
}
```

▼ ブラウザでの表示:

```
タイトル              ← <h1> 大きな見出し
本文                  ← <p>  通常の段落
```

ただし HTML 出力に余計な `<div>` タグが1つ増える点に注意。

```
// ============================================================================
// ✅ 解決策2: Fragment (<>...</>) で包む【こちらが推奨】
// ============================================================================
// Fragment は「論理的に1つにまとめるが、DOMには何も出さない」特殊なラッパー。
function App() {
  return (
    // 開始タグ: 名前のないタグ（return の直後は JSX の外なので // でコメントを書く）
    <>
      <h1>タイトル</h1>
      <p>本文</p>
      {/* 終了タグの直前は JSX の中なので、この形でコメントを書く */}
    </>
  );
}
```

▼ ブラウザでの表示:

上の `<div>` 版と見た目は同じだが、実際の HTML には `<div>` が出力されない。レイアウト崩れを起こしたくないときに便利。

▼ コードを1つずつ分解して解説

解説1: Fragment（空タグ）で全体を1つに包む

```
return (
  // 開始タグ: 名前のないタグ（return の直後は JSX の外なので // でコメントを書く）
  <>
```

- JSXには「**returnで返せるのは1つの要素だけ**」というルールがあります。複数の要素を並べたいときは、必ず何か1つの要素で包む必要があります。
- `return (` の直後は、まだ JSX の「外」なので、コメントは普通の JavaScript と同じ `//` で書きます。`{/* */}` 形式のコメントが使えるのは、`<>` や `<div>` などのタグの「内側」だけです（外側に書くと構文エラーになります）。
- `<>` は **Fragment（フラグメント）** の開始タグです。`<div>` と違い「論理的に1つにまとめるが、実際のHTMLには何も出力しない」特殊な空タグです。
- `( )` で囲んでいるのは、JSXを複数行に分けて書くためです（returnの直後で改行すると別の意味になるため、括弧で1つの式にまとめています）。

> **用語: Fragment（フラグメント）** … 余計なタグを増やさずに複数の要素をまとめるためのReactの仕組み。`<> ... </>` と書く。

解説2: 中身の要素を並べて、空タグで閉じる

```
    <h1>タイトル</h1>
    <p>本文</p>
    {/* 終了タグの直前は JSX の中なので、この形でコメントを書く */}
  </>
);
```

- `<>` の中に、表示したい要素（見出しと段落）を好きなだけ並べられます。
- `</>` が Fragment の終了タグです。開始の `<>` と必ず対で書きます。
- `<div>` で包む方法との違いは「**HTMLに余計な箱（divタグ）が増えない**」こと。見た目は同じでも、出力されるHTMLがすっきりするのでこちらが推奨されます。

> **用語: ルート要素** … returnで返すJSXの一番外側の1つの要素。Fragmentやdivがこの役割を果たす。

HTML と JSX の違い

```
// ============================================================================
// HTML との細かい違い – JSX で書き換えが必要なポイント
// ============================================================================
function App() {
  return (
    <div>
      {/*
        (1) class は予約語のため className を使う
            → JS の class（クラス定義）と区別するため。
      */}
      <div className="container">

        {/*
          (2) <label for=""> は htmlFor に書き換える
              → for も JS の予約語（for ループ）だから。
        */}
        <label htmlFor="name">名前:</label>
        <input id="name" />

        {/*
          (3) style 属性はオブジェクトで渡す（文字列ではない！）
          – 外側の {} は「JSXに JS の式を埋め込む」記号
          – 内側の {} は「JS のオブジェクトリテラル」
          – キーはキャメルケース: font-size → fontSize, margin-top → marginTop
          – 値は文字列（"16px"）でもOK。数値だけ渡すと px が自動で付くものもある
        */}
        <p style={{ color: "red", fontSize: "16px", marginTop: "10px" }}>
          赤いテキスト
        </p>

        {/*
          (4) コメントは {/* ... */} 形式
          普通の HTML コメント <!-- ... --> は JSX では書けない。
        */}

        {/*
          (5) 自己閉じタグは「/」で必ず閉じる必要がある
          HTML では <br>, <img src="..."> でも OK だが、JSX は厳密。
        */}
        <br />
        <img src="/logo.png" alt="ロゴ" />
        <input type="text" />
      </div>
    </div>
  );
}
```

▼ ブラウザでの表示:

```
名前: [               ] ← input ボックス

赤いテキスト              ← color:red, font-size:16px で表示

[ロゴ画像]

[               ]        ← 別の input ボックス
```

▼ コードを1つずつ分解して解説

解説1: class ではなく className を使う

```
      <div className="container">
```

- HTMLでは `class="container"` と書きますが、JSXでは **`className`** と書きます。
- 理由は、`class` がJavaScriptの予約語（クラス定義に使う特別な単語）だからです。JSXは実体がJavaScriptなので、ぶつからないように名前を変えています。

> **用語: 予約語** … プログラミング言語があらかじめ用途を決めている特別な単語。変数名や属性名に使えない。

解説2: label の for は htmlFor に書き換える

```
        <label htmlFor="name">名前:</label>
        <input id="name" />
```

- HTMLの `<label for="name">` は、JSXでは **`htmlFor`** と書きます。これも `for`（forループ）がJavaScriptの予約語だからです。
- `htmlFor="name"` と `<input id="name">` を同じ値で結ぶと、ラベルをクリックしたとき対応する入力欄にカーソルが移ります。

> **用語: label（ラベル）** … 入力欄の見出し。for/htmlFor で入力欄と結びつける。

解説3: style はオブジェクトで渡す

```
<p style={{ color: "red", fontSize: "16px", marginTop: "10px" }}>
  赤いテキスト
</p>
```

- HTMLでは `style="color: red"` と文字列で書きますが、JSXでは **オブジェクト**で渡します。
- `{{ }}` の **外側の** `{ }` は「ここからJavaScriptを書く」という印、**内側の** `{ }` は「オブジェクト（キーと値の集まり）」です。
- CSSのプロパティ名は**キャメルケース**（単語の区切りを大文字に）で書きます。`font-size` → `fontSize`、`margin-top` → `marginTop`。

> **用語: キャメルケース** … `fontSize` のように2語目以降の先頭を大文字にする書き方。ラクダのコブに似ているのが由来。

解説4: 自己閉じタグは / で必ず閉じる

```
<br />
<img src="/logo.png" alt="ロゴ" />
<input type="text" />
```

- 中身を持たないタグ（改行・画像・入力欄など）は、JSXでは必ず末尾を `/>` で閉じます。
- HTMLでは `<br>` や `<img src="...">` のように閉じなくても許されますが、JSXは厳密なので `/` が必須です。
- `alt` は画像が表示できないときの代替テキスト、`src` は画像の場所（URL）を指定します。

> **用語: 自己閉じタグ（self-closing tag）** … 中身を持たず、1つのタグで完結する要素。末尾を `/>` で閉じる。

HTML	JSX/TSX
`class`	`className`
`for`	`htmlFor`
`style="color: red"`	`style={{ color: "red" }}`
`tabindex`	`tabIndex`
`onclick`	`onClick`
`<!-- コメント -->`	`{/* コメント */}`

2.3 JavaScript 式の埋め込み

JSX の中で `{}` を使うと、JavaScript の式を埋め込めます。

> **▼ このコードがやること（先に日本語で）**：見た目（JSX）の中に、**変数の値や計算結果を埋め込んで**表示します。やり方は「埋め込みたい場所を `{ }`（中カッコ）で囲む」だけ。`{ }` の中は"JavaScriptの世界"になり、変数・計算・関数の呼び出しなどを書けます。ただし**書けるのは「式（値になるもの）」だけ**で、`if` 文や `for` 文のような「文（命令）」は書けない、という1点に注意します（理由はコード内のコメントで説明します）。

```
// ============================================================================
// JSX に JavaScript の式を埋め込むサンプル
// ============================================================================
// JSX 中で { ... } を使うと、その中だけ「JavaScriptの式」を書ける。
// 注意: 文（statement）は書けない。式（expression）のみ。
//    ・OK : 変数、計算式、関数呼び出し、三項演算子、配列メソッド
//    ・NG : if 文、for 文、変数宣言（const x = ...）

function App() {
  // (1) コンポーネント関数の中（return より前）は普通の JS。
  //     ここで変数宣言や計算をしておく。
  // 文字列の変数
  const userName: string = "田中太郎";
  // 数値の変数
  const age: number = 25;
  // 現在時刻の Date オブジェクト
  const today: Date = new Date();

  // (2) return 内の JSX に { } で値を差し込む
  return (
    <div>
      {/* (a) 変数をそのまま埋め込む。
              <h1> の中身は「ようこそ、田中太郎さん！」になる。 */}
      <h1>ようこそ、{userName}さん！</h1>

      {/* (b) 計算式の結果を埋め込む。
              {age + 1} は 25+1 を計算してから 26 を表示する。 */}
      <p>年齢: {age}歳（来年は{age + 1}歳）</p>

      {/* (c) メソッド呼び出しも埋め込める。
              today.toLocaleDateString("ja-JP") は日付を「2026/3/16」形式に変換する。 */}
      <p>今日の日付: {today.toLocaleDateString("ja-JP")}</p>

      {/* (d) テンプレートリテラル（バッククォート）も式なのでOK。 */}
      <p>{`${userName}さんは${age}歳です`}</p>
    </div>
  );
}
```

▼ ブラウザでの表示（2026年5月10日に開いた場合の例）：

```
ようこそ、田中太郎さん！        ← <h1> 大きな見出し

年齢: 25歳（来年は26歳）

今日の日付: 2026/5/10

田中太郎さんは25歳です
```

▼ コードを1つずつ分解して解説

解説1: returnより前で変数を準備する

```
  // 文字列の変数
  const userName: string = "田中太郎";
  // 数値の変数
  const age: number = 25;
  // 現在時刻の Date オブジェクト
  const today: Date = new Date();
```

- コンポーネント関数の中で、`return` より**前**の部分は普通のJavaScriptです。ここで表示に使う変数を用意しておきます。
- `const 名前: 型 = 値;` は変数宣言です。`: string` や `: number` は「この変数の型」を表す型注釈です（書かなくても動きますが、書くと安全になります）。
- `new Date()` は「今この瞬間の日時」を表すオブジェクトを作ります。

> **用語: 変数** … 値に名前を付けて保管しておく入れ物。`const` は後から中身を変えられない変数。

解説2: 変数や計算結果を { } で埋め込む

```
      <h1>ようこそ、{userName}さん！</h1>
      <p>年齢: {age}歳（来年は{age + 1}歳）</p>
```

- JSXの中で `{ }`（**中カッコ**）を使うと、その中だけ「JavaScriptの世界」になり、変数や計算式を書けます。
- `{userName}` は変数 `userName` の中身（"田中太郎"）に置き換わります。
- `{age + 1}` のように**計算式**も書けます。`25 + 1` が計算されて `26` と表示されます。

> **用語: 埋め込み（補間）** … 文字の中に変数の値を差し込むこと。JSXでは `{ }` で行う。

解説3: メソッド呼び出しやテンプレートリテラルも埋め込める

```
      <p>今日の日付: {today.toLocaleDateString("ja-JP")}</p>
      <p>{`${userName}さんは${age}歳です`}</p>
```

- `{ }` の中には**関数（メソッド）の呼び出し**も書けます。`today.toLocaleDateString("ja-JP")` は日付を「2026/5/10」のような日本語形式の文字列に変換します。
- 2行目の `` `${userName}さんは${age}歳です` `` は**テンプレートリテラル**。バッククォート（`` ` ``）で囲み、`${ }` の中に変数を差し込める文字列です。これも「値になるもの（式）」なので埋め込めます。

> **用語: 式（expression）** … 計算すると1つの値になるもの（変数・計算・関数呼び出しなど）。`{ }` の中には式だけ書ける。`if` / `for` などの「文」は書けない。

重要: `{}` の中に書けるのは **式（expression）** だけです。`if` 文や `for` 文などの **文（statement）** は書けません。

```
// NG: if 文は式ではないので書けない
// 文（statement）は「値を持たない命令」。値を持たないものを { } の中に書くことはできない。
<p>{if (age >= 20) { "成人" }}</p>

// OK: 三項演算子は式なので書ける
// 「条件 ? 真の値 : 偽の値」は値を返す式。だから JSX 内に埋め込める。
<p>{age >= 20 ? "成人" : "未成年"}</p>
```

2.4 条件付きレンダリング

三項演算子

```
// ============================================================================
// 三項演算子 ?: で表示を切り替える
// ============================================================================
// 条件 ? 真の時の値 : 偽の時の値　という JS の式。
// JSX の中で「片方を表示・もう片方を表示」を切り替えるのに最適。

function UserStatus() {
  // (1) ユーザーがログイン済みかどうかを表す状態（ここでは固定値）
  const isLoggedIn: boolean = true;
  const userName: string = "田中太郎";

  return (
    <div>
      {/*
        (2) JSX 内の { ... } は JS の式。三項演算子は式なので使える。
            isLoggedIn が true なら <p>ようこそ...</p> を、
            false なら <p>ログインしてください。</p> を返す。
      */}
      {isLoggedIn ? (
        <p>ようこそ、{userName}さん！</p>
      ) : (
        <p>ログインしてください。</p>
      )}
    </div>
  );
}
```

▼ ブラウザでの表示:

- `isLoggedIn = true` のとき： ようこそ、田中太郎さん！
- `isLoggedIn = false` のとき： ログインしてください。

&&（論理AND）演算子

条件を満たしたときだけ表示したい場合に便利です。

```tsx
// ============================================================================
// && 演算子で「条件を満たしたときだけ」表示する
// ============================================================================
// JS では: A && B
//   A が truthy（真っぽい値）なら B が評価され、その結果が返る。
//   A が falsy（false, 0, "", null, undefined）なら A 自身が返る。
// JSX では false / null / undefined は「何も表示しない」扱いになる。
// よって「条件 && JSX」と書けば「条件が true のときだけ JSX を表示」になる。

function Notification() {
  // 数値の状態（未読件数）と真偽値の状態（エラー有無）
  const unreadCount: number = 5;
  const hasError: boolean = false;

  return (
    <div>
      <h1>ダッシュボード</h1>

      {/*
        (a) unreadCount > 0 の結果は boolean。
            - 5 > 0 → true なので <p>...</p> が表示される
            - 0 > 0 → false なので何も表示されない
      */}
      {unreadCount > 0 && (
        <p className="notification">
          未読メッセージが{unreadCount}件あります。
        </p>
      )}

      {/*
        (b) hasError は false なので右辺は評価されず、何も表示されない
      */}
      {hasError && (
        <p className="error">エラーが発生しました。</p>
      )}
    </div>
  );
}
```

▼ ブラウザでの表示:

```
ダッシュボード                          ← <h1>
未読メッセージが5件あります。            ← (a) は表示される
                                        ← (b) hasError=false なので非表示
```

注意: `&&` 演算子を使う際、左辺が数値の `0` だと `0` がそのまま画面に表示されてしまいます。

```tsx
// NG: count が 0 のとき、画面に「0」が表示されてしまう
// JSX は false/null/undefined は描画しないが、0 は数値として描画してしまう。
{count && <p>{count}件</p>}

// OK: 比較演算子を使えば安全
// count > 0 は必ず true/false の真偽値になるので、0件のときは何も表示されない。
{count > 0 && <p>{count}件</p>}
```

複雑な条件分岐

```tsx
// type 文は TypeScript の型定義。Status は "loading" / "success" / "error" のいずれか
// の文字列しか入らない「ユニオン型」になる。タイポしただけで VS Code が赤線で教えてくれる。
type Status = "loading" | "success" | "error";

// 関数コンポーネント DataDisplay の定義
function DataDisplay() {
  // 現在の状態を表す変数。: Status は型注釈で「Status 型しか入らない」と宣言。
  const status: Status = "success";
  // 表示するデータ本体（成功時に出す文字列）
  const data: string = "データの内容";
  // エラー時のメッセージ。今回はエラーがないので空文字
  const errorMessage: string = "";

  // 関数で条件分岐をまとめる
  // renderContent は「画面に出す中身を返す関数」。
  // 戻り値の型 JSX.Element は「JSXの要素1つ」を意味する。
  const renderContent = (): JSX.Element => {
    // switch 文: 1つの値を複数の case と比較して分岐する制御構造
    switch (status) {
      case "loading":
        // 状態が "loading" のとき表示するJSXを返す
        return <p>読み込み中...</p>;
      case "error":
        // 状態が "error" のとき。className="error" でCSSクラスを指定（赤字スタイル等）
        // { } の中は JS 式。errorMessage 変数の値が埋め込まれる。
        return <p className="error">エラー: {errorMessage}</p>;
      case "success":
        // 状態が "success" のとき。{data} で変数の中身を表示。
        return <p>{data}</p>;
    }
  };

  // 親JSXで <h2> と renderContent() の戻り値を並べる。
  // {renderContent()} の () は「関数を呼び出す」記号。
  // 関数の戻り値（=JSX）がここに展開される。
  return (
    <div>
      <h2>データ表示</h2>
      {renderContent()}
    </div>
  );
}
```

> `status` が `"success"` の場合、画面には「**データ表示**」という見出しと「**データの内容**」というテキストが表示されます。

2.5 リストのレンダリング（map）

配列データを画面に表示するには、`map` メソッドを使います。

> **▼ このコードがやること（先に日本語で）**：「りんご・バナナ…」という**文字列の配列**を、画面上の**箇条書きリスト（`<li>` の並び）**に変換して表示します。カギになるのが第2章で学んだ `map` ――「**配列の各要素を1つずつ別のものに作り替えて、新しい配列を作る**」メソッドです。ここでは「各フルーツ名（文字列）」を「`<li>フルーツ名</li>`（リスト項目のJSX）」に作り替えています。さらに、各項目には `key`（要素を見分けるための目印）を必ず付ける、という決まりも出てきます（理由は後述の 8.3 で詳しく解説します）。

```
// 関数コンポーネント FruitList を定義
function FruitList() {
  // fruits: 4つの文字列が入った配列。型 string[] は「文字列の配列」を意味する。
  const fruits: string[] = ["りんご", "バナナ", "みかん", "ぶどう"];

  return (
    <div>
      <h2>フルーツ一覧</h2>
      {/* <ul> は順序なしリスト（unordered list）。中に <li>（list item）を並べる */}
      <ul>
        {/*
          {fruits.map(...)} = JSX 中に「配列の各要素を JSX に変換した結果の配列」を埋め込む。
          map((要素, インデックス) => 戻り値) で全要素に対して関数を呼び、新しい配列を作る。
            fruit  = 配列の各要素（"りんご" など）
            index  = 配列内での位置（0, 1, 2, 3）
          アロー関数の () => (...) は「JSXを返すアロー関数」。
          { } の中で () で囲んでいるのは「return できる単一の式」にするため。
        */}
        {fruits.map((fruit, index) => (
          // key={index} は React がリスト要素を識別するための特別な属性。
          // 後述のとおり、本来は配列の index ではなく一意なIDを使うのが望ましい。
          // ここでは「並び替えがない静的なリスト」なので index でも問題ない。
          <li key={index}>{fruit}</li>
        ))}
      </ul>
    </div>
  );
}
```

この結果、画面には「**フルーツ一覧**」という見出しと、箇条書きで以下のリストが表示されます:

- りんご
- バナナ
- みかん
- ぶどう

▼ コードを1つずつ分解して解説

解説1: 表示元になる配列を用意する

```
const fruits: string[] = ["りんご", "バナナ", "みかん", "ぶどう"];
```

- `fruits` は4つの文字列が入った**配列**（複数の値を順番に並べた入れ物）です。
- `: string[]` は型注釈で「**文字列の配列**」という意味。`string` が「文字列」、`[]` が「配列」を表します。
- この配列の各要素を、画面のリスト項目に変換していきます。

> **用語: 配列（array）** … 複数の値を `[ ]` で並べて1つにまとめたもの。`["a", "b"]` のように書く。

**解説2: map で各要素を
に変換する**

```
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li>
      ))}
```

- `fruits.map(...)` は「**配列の各要素を、別のものに作り替えて新しい配列を作る**」メソッドです。ここでは「フルーツ名（文字列）」を「`<li>フルーツ名</li>`（リスト項目）」に作り替えています。
- `(fruit, index) => (...)` はアロー関数で、`fruit` が各要素（"りんご"など）、`index` が並び順の番号（0,1,2,3）です。
- `<li key={index}>{fruit}</li>` で、各フルーツ名を箇条書き項目にしています。`{fruit}` で要素の値を表示します。
- `key={index}` は、Reactがリストの各項目を見分けるための目印です（詳しくは後の 8.3 で解説）。

> **用語: map（マップ）** … 配列の全要素を1つずつ変換して新しい配列を作るメソッド。リスト表示の定番。

オブジェクト配列のレンダリング

```
// ============================================================================
// オブジェクト配列を map で展開して表示するサンプル
// ============================================================================

// (1) Book 型を定義（書籍データ1件分の形）
//     type を使うのが React/TS では一般的。interface でも同じことができる。
type Book = {
  // 一意なID（key に使う）
  id: number;
  // 書籍タイトル
  title: string;
  // 著者名
  author: string;
  // 税込価格
  price: number;
  // 在庫があるか（true=あり, false=なし）
  isAvailable: boolean;
};

function BookList() {
  // (2) Book 型の配列を作る。
  //     型注釈 Book[] で「Book型のオブジェクトしか入らない配列」と宣言。
  //     1件でもプロパティが欠けていると VS Code が赤線で教えてくれる。
  const books: Book[] = [
    { id: 1, title: "React入門",       author: "田中太郎", price: 2800, isAvailable: true
},
    { id: 2, title: "TypeScript実践",  author: "鈴木花子", price: 3200, isAvailable:
false },
    { id: 3, title: "Next.js徹底解説", author: "佐藤一郎", price: 3500, isAvailable: true
},
  ];

  return (
    <div>
      <h2>書籍一覧</h2>

      {/*
        (3) books.map((book) => JSX) で「配列の各要素を JSX に変換」する。
            戻り値は「JSX要素の配列」になり、それを React がそのまま並べて描画する。

            (4) key={book.id} は「リスト項目を識別するため」の特別な属性。
                React がリストの差分検出に使う。配列内で一意である必要がある。
                添え字 i ではなく、データのID（DBのprimary key）を使うのが鉄則。
      */}
      {books.map((book) => (
        <div key={book.id} className="book-card">
          <h3>{book.title}</h3>
```

```
            <p>著者: {book.author}</p>

            {/*
              (5) book.price.toLocaleString() は数値を「3桁ごとにカンマで区切った
                  文字列」に変換するメソッド。2800 → "2,800"
            */}
            <p>価格: ¥{book.price.toLocaleString()}</p>

            {/*
              (6) 三項演算子で表示文字列と色を切り替える
                  {" "} は意図的に空白を1文字入れたいとき使う書き方
            */}
            <p>
              状態:{" "}
              <span style={{ color: book.isAvailable ? "green" : "red" }}>
                {book.isAvailable ? "在庫あり" : "在庫なし"}
              </span>
            </p>
          </div>
        ))}
      </div>
    );
}
```

▼ ブラウザでの表示:

```
書籍一覧
──────────────────────────────────
React入門
著者: 田中太郎
価格: ¥2,800
状態: 在庫あり          ← 緑色
──────────────────────────────────
TypeScript実践
著者: 鈴木花子
価格: ¥3,200
状態: 在庫なし          ← 赤色
──────────────────────────────────
Next.js徹底解説
著者: 佐藤一郎
価格: ¥3,500
状態: 在庫あり          ← 緑色
```

▼ コードを1つずつ分解して解説

解説1: 書籍データ1件分の「形」を型で決める

```
type Book = {
  // 一意なID（key に使う）
  id: number;
  // 書籍タイトル
  title: string;
  // 著者名
  author: string;
  // 税込価格
  price: number;
  // 在庫があるか（true=あり, false=なし）
  isAvailable: boolean;
};
```

- `type Book = { ... }` は「**Book型という、データの形のひな型**」を定義しています。
- この型は「本1件は `id`（数値）・`title`（文字列）・`author`（文字列）・`price`（数値）・`isAvailable`（真偽値）を持つ」と決めています。
- こう決めておくと、プロパティを書き忘れたり型を間違えたりしたとき、VS Codeが赤線で教えてくれます。

> **用語: 型（type）** … データの形（どんな項目をどんな種類で持つか）の決まり。TypeScriptで定義する。

解説2: Book型の配列を作る

```
  const books: Book[] = [
    { id: 1, title: "React入門",       author: "田中太郎", price: 2800, isAvailable: true
},
    { id: 2, title: "TypeScript実践",  author: "鈴木花子", price: 3200, isAvailable:
false },
    { id: 3, title: "Next.js徹底解説", author: "佐藤一郎", price: 3500, isAvailable: true
},
  ];
```

- `: Book[]` は「**Book型のオブジェクトだけが入る配列**」という意味です。
- `{ }` で囲んだ1つ1つが書籍オブジェクト（解説1の Book 型に沿ったデータ）です。
- 文字列の配列と違い、各要素が複数の項目を持つ「オブジェクト」になっている点がポイントです。

> **用語: オブジェクト配列** … 配列の各要素がオブジェクト（複数項目の集まり）になっているもの。

解説3: map でオブジェクトをカードに変換し、key を付ける

```
      {books.map((book) => (
        <div key={book.id} className="book-card">
          <h3>{book.title}</h3>
          <p>著者: {book.author}</p>
```

- `books.map((book) => (...))` で、各書籍オブジェクトを1枚のカード（`<div>`）に作り替えます。`book` には各要素のオブジェクトが入ります。
- `{book.title}` のように `オブジェクト.プロパティ名` と書くと、そのオブジェクトの中の項目を取り出せます。
- `key={book.id}` には、配列の番号ではなく**データ固有のID**を使っています。これが推奨される付け方です。

> **用語: プロパティアクセス** … `book.title` のように `.` でオブジェクトの中の項目を取り出すこと。

解説4: メソッドや三項演算子で表示を整える

```
          <p>価格: ¥{book.price.toLocaleString()}</p>
          <p>
            状態:{" "}
            <span style={{ color: book.isAvailable ? "green" : "red" }}>
              {book.isAvailable ? "在庫あり" : "在庫なし"}
            </span>
          </p>
```

- `book.price.toLocaleString()` は数値を「3桁ごとにカンマで区切った文字列」にします。`2800` → `"2,800"`。
- `{" "}` は「ここに半角スペースを1つ入れたい」というときの書き方です。
- `book.isAvailable ? "在庫あり" : "在庫なし"` は**三項演算子**で、`isAvailable` が `true` なら「在庫あり」、`false` なら「在庫なし」を表示します。文字色も同じ仕組みで緑/赤に切り替えています。

> **用語: 三項演算子** … `条件 ? Aの値 : Bの値` で、条件によって2つの値を切り替える式。

重要: `key` プロパティ（key prop：キー プロップ。Reactがリスト要素を識別するための特別な属性）には、配列内で一意（ユニーク）な値を指定します。データベースから取得した `id` がベストです。`index` は最後の手段です（要素の並び替え・追加・削除で不具合の原因になります）。`key` は React 内部の差分計算（reconciliation）専用で、子コンポーネントから `props.key` として読むことはできません。

つまずきポイント（map の key 忘れ）：`map` で並べた要素に `key` を付け忘れても、画面には普通に表示されます。そのため気づきにくいのですが、ブラウザの開発者ツールの Console には、React 19 では次のような警告が出ています（`FruitList` の部分は、あなたのコンポーネント名になります）。「画面は出ているから大丈夫」と放置すると、8.3 で見るような「入力した文字が別の行に移る」バグの原因になります。

```
Each child in a list should have a unique "key" prop.

Check the render method of `FruitList`. See https://react.dev/link/warning-keys for
more information.
```

```jsx
// ❌ NG: map で返す <li> に key が無い（表示はされるが Console に警告が出る）
{fruits.map((fruit) => (
  <li>{fruit}</li>
))}
```

```jsx
// ✅ OK: map で返す「一番外側の要素」に、重複しない値を key として付ける
{fruits.map((fruit) => (
  <li key={fruit}>{fruit}</li>
))}
```

key は「`map` の中で `return` している一番外側の要素」に付けます。`<li>` の中の `<span>` に付けても意味がありません。また、ここでは果物の名前が重複しない前提で `fruit` を key にしていますが、データに `id` があるなら `id` を使うのが一番安全です。

フィルタリングと map の組み合わせ

```
// 「在庫があるものだけ」を表示する関数コンポーネント
function AvailableBookList() {
  // Book型の配列を用意（前のサンプルと同じ Book 型を使用）
  const books: Book[] = [
    { id: 1, title: "React入門", author: "田中太郎", price: 2800, isAvailable: true },
    { id: 2, title: "TypeScript実践", author: "鈴木花子", price: 3200, isAvailable: false
},
    { id: 3, title: "Next.js徹底解説", author: "佐藤一郎", price: 3500, isAvailable: true
},
  ];

  return (
    <div>
      <h2>在庫のある書籍</h2>
      {/*
        メソッドチェーン: 「.filter() の戻り値（新しい配列）」に「.map() を呼ぶ」と続けて書く形。
        - filter((要素) => 条件) は「条件が true の要素だけ残した新しい配列」を返す。
          ここでは book.isAvailable が true の書籍だけを残す。
        - 続く map((要素) => JSX) で「残った書籍を JSX に変換」。
        - filter/map とも元の配列を変更しない。新しい配列を返す（破壊的ではない）。
      */}
      {books
        .filter((book) => book.isAvailable)
        .map((book) => (
          // key={book.id} は必須。データのID（一意な値）を使うのがベスト。
          <div key={book.id}>
            <p>
              {/* {book.title} は文字列。- は普通の文字。
                  ¥ は通貨記号（普通の文字）。
                  {book.price.toLocaleString()} は「数値→3桁カンマ区切り文字列」変換。 */}
              {book.title} - ¥{book.price.toLocaleString()}
            </p>
          </div>
        ))}
    </div>
  );
}
```

この結果、画面には「**在庫のある書籍**」という見出しと、在庫のある2冊だけが表示されます:

React入門 - ¥2,800

Next.js徹底解説 - ¥3,500

▼ コードを1つずつ分解して解説

解説1: filter で条件に合う要素だけ残す

```
      {books
        .filter((book) => book.isAvailable)
```

- `filter`（フィルター）は「**条件が true の要素だけを残した新しい配列**」を作るメソッドです。
- `(book) => book.isAvailable` は「`book.isAvailable` が `true` の本だけ残す」という条件です。在庫ありの本だけが残ります。
- 元の `books` 配列は変更されず、新しい配列が作られます（**非破壊的**）。

> **用語: filter（フィルター）** … 配列から条件に合う要素だけを抜き出して新しい配列を作るメソッド。

解説2: メソッドチェーンで filter のあとに map をつなぐ

```
        .map((book) => (
          <div key={book.id}>
            <p>
              {book.title} - ¥{book.price.toLocaleString()}
            </p>
          </div>
        ))}
```

- `.filter(...)` のすぐ後ろに `.map(...)` を続けて書いています。これを **メソッドチェーン**（メソッドを数珠つなぎにする書き方）と呼びます。
- 流れは「①filter で在庫ありだけ残す → ②map でその各本をJSXに変換する」の2段階です。
- `{book.title} - ¥{book.price.toLocaleString()}` で「タイトル - ¥価格」の形に表示しています。`-` や `¥` はそのままの文字、`{ }` の中だけがJavaScriptの値です。

> **用語: メソッドチェーン** … `配列.filter(...).map(...)` のようにメソッドを連続して呼ぶ書き方。filterの結果にmapが続けて適用される。

---

3. コンポーネント

3.1 関数コンポーネントの書き方

React では、**関数コンポーネント**（Function Component：関数として書かれたコンポーネント）が標準的な書き方です（**クラスコンポーネント**（Class Component：class 構文で書かれた古い書き方）は現在の React では推奨されていません。本書では扱いません）。

> **▼ このコードがやること（先に日本語で）**：同じ「こんにちは！と表示する部品」を、**3通りの書き方**で示します。中身はどれも同じで、書き方（見た目）が違うだけです——① `function` を使う書き方、② `const 名前 = () => {...}` のアロー関数で書く書き方、③1行で返せるときに `return` と `{ }` を省く短い書き方。どれを使っても結果は同じなので、「いろんな書き方があるが、やっていることは同じ」と分かれば十分です。実務では②③のアロー関数の形をよく見かけます。

```
// 最もシンプルな関数コンポーネント
// function 文で「JSXを返す関数」を作るだけでコンポーネントになる。
// 関数名 Greeting は大文字始まり（PascalCase）にする。これが React の決まり。
function Greeting() {
  // return = この関数の戻り値を指定。<h1>...</h1> の JSX をそのまま返す。
  return <h1>こんにちは！</h1>;
}

// アロー関数でも書ける
// const Greeting = ... で「Greeting という変数にアロー関数を入れる」書き方。
// () は引数（このコンポーネントは props を取らないので空）。
// => の右側 { ... } が関数本体。
const Greeting = () => {
  return <h1>こんにちは！</h1>;
};

// 1行で返せる場合は return を省略できる
// アロー関数で「=> 式」と書くと、その式が自動的に戻り値になる（即時 return）。
// 中括弧 { } を書かないことで「式を直接返す」モードになる。
const Greeting = () => <h1>こんにちは！</h1>;
```

> いずれの書き方でも、画面には「**こんにちは！**」という見出しが表示されます。

▼ コードを1つずつ分解して解説

解説1: function を使う書き方

```
function Greeting() {
  return <h1>こんにちは！</h1>;
}
```

- `function 名前() { ... }` は、もっとも基本的な関数の書き方です。
- `return` で見た目（JSX）を返せば、それだけでコンポーネントになります。
- 関数名 `Greeting` は**大文字始まり**（PascalCase）にします。これがReactの決まりです。

> **用語: PascalCase（パスカルケース）** … `Greeting` のように先頭を大文字にする命名法。Reactのコンポーネント名はこれを使う。

解説2: アロー関数を使う書き方

```
const Greeting = () => {
  return <h1>こんにちは！</h1>;
};
```

- `const 名前 = () => { ... }` は**アロー関数**という書き方で、「Greetingという変数に関数を入れる」形です。
- `()` が引数、`=>` の右の `{ }` が関数の中身です。やっていることは解説1とまったく同じです。
- 実務ではこのアロー関数の形がよく使われます。

> **用語: アロー関数** … `() => { ... }` という形で書く関数。`function` より短く書ける。

解説3: 1行で返せるときは return と { } を省略

```
const Greeting = () => <h1>こんにちは！</h1>;
```

- アロー関数で `=> 式` と書くと、その式が**自動的に戻り値**になります（returnを書かなくてよい）。
- 中括弧 `{ }` を書かないことで「式を直接返す」モードになります。
- 1行で済むシンプルな部品のとき、この短い書き方がよく使われます。3つとも結果は同じです。

> **用語: 暗黙のreturn** … アロー関数で `{ }` を省くと、右側の式が自動的に戻り値になる仕組み。

コンポーネントの使い方

```
// 親コンポーネント App から子コンポーネント Greeting を呼び出す例
function App() {
  return (
    // <div> で全体を1つにまとめる（JSXの「ルート要素は1つ」ルール）
    <div>
      {/* <Greeting /> は自己閉じタグ。HTMLの <br /> と同じく / で閉じる。
          中身がない子コンポーネントは自己閉じタグで書くのが慣習。
          同じコンポーネントは何度でも書けて、それぞれが独立したインスタンスになる。 */}
      <Greeting />
      <Greeting />
      <Greeting />
    </div>
  );
}
```

> この結果、画面には「**こんにちは！**」が3回表示されます。同じコンポーネントを何度でも再利用できます。**1つのコンポーネント定義（設計図）から、いくつでもインスタンス（実際の表示）を作れる**のがコンポーネント指向のメリットです。

命名規則: コンポーネント名は必ず **大文字始まり（PascalCase：パスカルケース。単語の先頭をすべて大文字にする命名法）** にします。小文字で始まると、HTML タグとして認識されてしまいます。

```
// OK: 大文字始まり → React コンポーネント
// JSX → JS変換時に React.createElement(Greeting, ...) と「変数 Greeting」として扱われる
<Greeting />
<BookCard />
<UserProfile />

// NG: 小文字始まり → HTML タグとして扱われる
// React.createElement("greeting", ...) になり、不明なHTMLタグとしてDOMに出力される
// <greeting> という存在しない HTML タグになる
<greeting />
```

3.2 Props の受け渡し（TypeScript での型定義含む）

Props（プロパティ：Properties。「親が子に渡す入力データ」のこと。HTMLタグの属性に似ている）は、親コンポーネントから子コンポーネントにデータを渡す仕組みです。値は **読み取り専用**で、子の中で書き換えてはいけません（書き換えても親には反映されないし、Reactの想定外の動作になります）。

propsドリリング（Props Drilling：プロップスドリリング）について: 親 → 子 → 孫… と何階層も同じ props を「素通し」で渡し続ける現象です。階層が深くなるとコード保守が辛くなるため、後の章で扱う **Context**（コンテキスト）や状態管理ライブラリで解決します。

リフトアップ（Lifting State Up：状態を持ち上げる）について: 兄弟コンポーネント間で同じ state を共有したいとき、共通の親に state を「持ち上げ」、props と更新関数を子に配るパターンです。例えば「親が `items` を持ち、子A が読み出し、子B が更新」という構図にします（state を学んだあと、5.5 で図と動くサンプルを使って詳しく説明します）。

基本的な Props

```tsx
// ============================================================================
// Props の最小サンプル – 親が「name」という値を子に渡す
// ============================================================================

// (1) 子コンポーネントが受け取る Props の「形」を type で定義する。
//     ここでは「name という string が必須で1つ」と宣言。
type GreetingProps = {
  name: string;
};

// (2) Greeting 関数コンポーネント
//     関数の引数で「props オブジェクト」を受け取る。
//     ここでは ({ name }: GreetingProps) と分割代入 + 型注釈で書いている。
//     等価なのは:
//       function Greeting(props: GreetingProps) {
//         const name = props.name;
//         ...
//       }
//     props.name と書かずに済むので分割代入のほうが好まれる。
function Greeting({ name }: GreetingProps) {
  // (3) 受け取った name を JSX 内に { } で埋め込んで使う
  return <h1>こんにちは、{name}さん！</h1>;
}

// (4) 親コンポーネント（App）が Greeting を3回使う
//     <Greeting name="田中" /> の name="田中" の部分が Props として子に渡る。
//     文字列リテラルは "..." または '...' で書く（{} はJS式埋め込み用）。
function App() {
  return (
    <div>
      <Greeting name="田中" />
      <Greeting name="鈴木" />
      <Greeting name="佐藤" />
    </div>
  );
}
```

▼ ブラウザでの表示:

```
こんにちは、田中さん！        ← <h1>
こんにちは、鈴木さん！        ← <h1>
こんにちは、佐藤さん！        ← <h1>
```

> 同じ Greeting コンポーネントが3回呼ばれ、それぞれ異なる name を受け取って描画されている。コンポーネントの再利用性がこれで実現できる。

▼ コードを1つずつ分解して解説

解説1: 受け取るPropsの「形」を型で決める

```
type GreetingProps = {
  name: string;
};
```

- `GreetingProps` は「この子コンポーネントが親から受け取るデータの形」を表す型です。
- ここでは「`name` という**文字列**を1つ受け取る」と決めています。
- こうしておくと、親が `name` を渡し忘れたり数値を渡したりしたとき、TypeScriptが警告してくれます。

> **用語: Props（プロパティ）** … 親コンポーネントから子コンポーネントへ渡すデータ。子の中では読み取り専用。

解説2: 分割代入でPropsを受け取る

```
function Greeting({ name }: GreetingProps) {
  return <h1>こんにちは、{name}さん！</h1>;
}
```

- 関数の引数 `{ name }` は**分割代入**で、親から渡されたPropsオブジェクトから `name` だけを取り出しています。
- `: GreetingProps` は「受け取るPropsはGreetingProps型ですよ」という型注釈です。
- 取り出した `name` を `{name}` でJSXに埋め込んで表示しています。

> **用語: 分割代入** … オブジェクトから必要な項目だけを取り出して変数にする書き方。`{ name } = props` のように書く。

解説3: 親がPropsを渡してコンポーネントを使う

```
function App() {
  return (
    <div>
      <Greeting name="田中" />
      <Greeting name="鈴木" />
      <Greeting name="佐藤" />
    </div>
  );
}
```

- `<Greeting name="田中" />` の `name="田中"` の部分が、子に渡すPropsです。
- 文字列は `"..."` で渡します（数値や変数を渡すときは `{ }` を使います）。
- 同じ `Greeting` を3回使い、それぞれ違う `name` を渡すことで、3つの異なる挨拶を表示しています。これがPropsによる**再利用**です。

> **用語: 再利用** ... 同じコンポーネントに違うPropsを渡して、何度も使い回すこと。

さまざまな型の Props

```
// Props の型を定義。1つのオブジェクトの「形」を表す type を作る。
type UserCardProps = {
  // 必須の文字列
  name: string;
  // 必須の数値
  age: number;
  // ? はオプショナル（省略可能）。値が無いと型は string | undefined になる
  email?: string;
  // 必須の真偽値（true / false）
  isAdmin: boolean;
  // 文字列の配列。[] は「配列」を表す
  hobbies: string[];
  // 関数型。「引数なし、戻り値なし（void）の関数」を表す
  onClickProfile: () => void;
};

// 分割代入で props のプロパティを直接取り出す。
// 元の書き方: function UserCard(props: UserCardProps) { const { name, ... } = props; }
function UserCard({
  name,
  age,
  email,
  isAdmin,
  hobbies,
  onClickProfile,
}: UserCardProps) {
  return (
    // className は HTML の class 属性の JSX 版（class は JS の予約語のため）
    <div className="user-card">
      {/* { } で JS の式を JSX に埋め込む。name 変数の値が表示される */}
      <h2>{name}</h2>
      {/* 「年齢: 25歳」のように文字列と変数が混ざる */}
      <p>年齢: {age}歳</p>

      {/* オプショナルな props は存在チェック
          email が undefined（=falsy）なら && の右辺が評価されず、何も表示されない。
          email が文字列（=truthy）なら <p>...</p> が表示される。 */}
      {email && <p>メール: {email}</p>}

      {/* 三項演算子 条件 ? 真の値 : 偽の値。式なので JSX 内に書ける */}
      <p>権限: {isAdmin ? "管理者" : "一般ユーザー"}</p>

      <div>
        趣味:
        <ul>
          {/* hobbies.map で配列を <li> の配列に変換。
              key には今回 index を使っているが、本来はデータ固有のIDが望ましい。 */}
```

```
          {hobbies.map((hobby, index) => (
            <li key={index}>{hobby}</li>
          ))}
        </ul>
      </div>

      {/* onClick に「関数自身」を渡す。()を付けると即実行になりバグの元なので注意。
          onClickProfile は親から受け取った関数で、ボタンクリック時に呼び出される。 */}
      <button onClick={onClickProfile}>プロフィールを見る</button>
    </div>
  );
}

// 使い方
function App() {
  // ボタンクリック時の処理を関数として用意。alert はブラウザの組み込み関数。
  const handleClick = () => {
    alert("プロフィールページへ移動します");
  };

  return (
    // 子コンポーネントに props を渡す。
    // - 文字列は "..." または '...' で書く（ダブルクオート/シングルクオート）
    // - 数値・真偽値・配列・関数は { } で囲んでJS式として渡す
    <UserCard
      name="田中太郎"
      age={25}
      email="tanaka@example.com"
      isAdmin={false}
      hobbies={["読書", "プログラミング", "映画鑑賞"]}
      onClickProfile={handleClick}
    />
  );
}
```

この結果、画面には以下のように表示されます:

田中太郎

年齢: 25歳

メール: tanaka@example.com

権限: 一般ユーザー

趣味: - 読書 - プログラミング - 映画鑑賞

【プロフィールを見る】ボタン

▼ コードを1つずつ分解して解説

解説1: いろいろな型のPropsを定義する

```
type UserCardProps = {
  // 必須の文字列
  name: string;
  // 必須の数値
  age: number;
  // ? はオプショナル（省略可能）
  email?: string;
  // 必須の真偽値（true / false）
  isAdmin: boolean;
  // 文字列の配列
  hobbies: string[];
  // 関数型（引数なし・戻り値なし）
  onClickProfile: () => void;
};
```

- Propsは文字列だけでなく、**数値・真偽値・配列・関数**など、いろいろな型を受け取れます。
- `email?` の `?` は「**オプショナル（省略可能）**」の印で、親が渡さなくてもよい項目です。
- `onClickProfile: () => void` は**関数を受け取る**型で、`() => void` は「引数なし・戻り値なしの関数」を表します。親が用意した処理を子で呼び出せます。

> **用語: オプショナル（ `?` ）** … 「あってもなくてもよい」項目を表す印。省略すると値は `undefined` になる。

解説2: 受け取った各Propsを使って表示する

```
      <h2>{name}</h2>
      <p>年齢: {age}歳</p>
      {email && <p>メール: {email}</p>}
      <p>権限: {isAdmin ? "管理者" : "一般ユーザー"}</p>
```

- `{name}` `{age}` のように、受け取ったPropsを `{ }` で埋め込んで表示します。
- `{email && <p>...</p>}` は「`email` に値があるときだけ表示」という条件付き表示です（省略可能なPropsの定番の扱い方）。
- `{isAdmin ? "管理者" : "一般ユーザー"}` は三項演算子で、真偽値によって表示文字を切り替えています。

> **用語: 条件付きレンダリング** … 条件に応じて表示する/しないを切り替えること。`&&` や三項演算子で書く。

解説3: 配列Propsを map で展開・関数Propsをボタンに渡す

```
        {hobbies.map((hobby, index) => (
          <li key={index}>{hobby}</li>
        ))}
    ...
    <button onClick={onClickProfile}>プロフィールを見る</button>
```

- 配列のProps `hobbies` は `map` で `<li>` に変換して箇条書き表示しています。
- `onClick={onClickProfile}` は、親から受け取った**関数をそのまま**ボタンのクリック処理に渡しています。`()` を付けない点に注意（付けると即実行されてしまう）。

> **用語: コールバック関数** … 親が用意し、子が「ボタンが押されたとき」などに呼び出す関数。

解説4: 親が各型の値を渡す

```
  <UserCard
    name="田中太郎"
    age={25}
    email="tanaka@example.com"
    isAdmin={false}
    hobbies={["読書", "プログラミング", "映画鑑賞"]}
    onClickProfile={handleClick}
  />
```

- 文字列は `"..."` でそのまま渡せます。
- 数値・真偽値・配列・関数は `{ }` で囲んで「JavaScriptの値」として渡します（`age={25}`, `isAdmin={false}`, `hobbies={[...]}`, `onClickProfile={handleClick}`）。
- この渡し方の違い（文字列だけ `"..."`、それ以外は `{ }`）が初心者のつまずきどころです。

> **用語: Propsの渡し方** … 文字列は `"..."`、それ以外（数値・真偽値・配列・オブジェクト・関数）は `{ }` で渡す。

デフォルト値を持つ Props

```
// Button が受け取る Props の型定義
type ButtonProps = {
  // ボタンに表示する文字（必須）
  label: string;
  // ? を付けると省略可能（オプショナル）
  color?: string;
  // ユニオン型: 3つの文字列のどれかしか入らない
  size?: "small" | "medium" | "large";
  // クリック不可にするかどうか（省略可能）
  disabled?: boolean;
};

// 分割代入で props を取り出し、同時に「= 値」でデフォルト値を設定。
// 親が color を指定しなかった場合は "blue" が使われる。
function Button({
  label,
  color = "blue",
  size = "medium",
  disabled = false,
}: ButtonProps) {
  // Record<キーの型, 値の型> は「オブジェクトの型」。
  // ここでは「キー: string, 値: string のオブジェクト」を表す。
  // ボタンサイズごとに padding（内側余白）の値を持たせている。
  const sizeStyles: Record<string, string> = {
    small: "8px 16px",
    medium: "12px 24px",
    large: "16px 32px",
  };

  return (
    <button
      // style はオブジェクトで指定。外側の {} が JSX、内側の {} が JSオブジェクトリテラル。
      // CSSプロパティ名はキャメルケース（background-color → backgroundColor）。
      style={{
        // 背景色（props 由来）
        backgroundColor: color,
        // size に対応する余白文字列
        padding: sizeStyles[size],
        // 文字色
        color: "white",
        // 枠線なし
        border: "none",
        // 角を丸める
        borderRadius: "4px",
        // 無効化されてたら半透明
        opacity: disabled ? 0.5 : 1,
        // マウスカーソルの形
```

```
        cursor: disabled ? "not-allowed" : "pointer",
      }}
      // disabled={disabled} は「JS変数の値をHTML属性にセット」する書き方。
      // boolean を渡せば true のとき disabled 属性が付き、false なら付かない。
      disabled={disabled}
    >
      {/* {label} は props の文字列をボタンの中身として表示 */}
      {label}
    </button>
  );
}

function App() {
  return (
    <div>
      {/* color, size, disabled を省略 → デフォルト値が使われる */}
      <Button label="デフォルト" />
      {/* color="red" size="large" を指定 */}
      <Button label="赤い大きなボタン" color="red" size="large" />
      {/* disabled だけ書くと自動的に disabled={true} と同じ意味になる（JSXのショートハンド）
*/}
      <Button label="無効化" disabled />
      <Button label="緑の小さなボタン" color="green" size="small" />
    </div>
  );
}
```

この結果、画面には4つのボタンが表示されます:

1. 「**デフォルト**」- 青色・中サイズのボタン
2. 「**赤い大きなボタン**」- 赤色・大サイズのボタン
3. 「**無効化**」- 青色・中サイズだが半透明でクリック不可のボタン
4. 「**緑の小さなボタン**」- 緑色・小サイズのボタン

▼ コードを1つずつ分解して解説

解説1: 省略可能なPropsとユニオン型を定義する

```
type ButtonProps = {
  // ボタンに表示する文字（必須）
  label: string;
  // ? を付けると省略可能
  color?: string;
  // ユニオン型: 3つの文字列のどれか
  size?: "small" | "medium" | "large";
  // クリック不可にするか（省略可能）
  disabled?: boolean;
};
```

- `label` だけ必須で、`color` `size` `disabled` は `?` 付きの**省略可能**なPropsです。
- `size?: "small" | "medium" | "large"` の `|` は「**または**」を表す**ユニオン型**で、この3つの文字列のどれかしか入れられません。タイポすると警告が出ます。

> **用語: ユニオン型** … `"a" | "b" | "c"` のように「いくつかのうちのどれか」を表す型。`|` でつなぐ。

解説2: 分割代入と同時にデフォルト値を設定する

```
function Button({
  label,
  color = "blue",
  size = "medium",
  disabled = false,
}: ButtonProps) {
```

- 分割代入で各Propsを取り出すと同時に、`= 値` で**デフォルト値**を指定しています。
- 親が `color` を渡さなかった場合は、自動的に `"blue"` が使われます。同様に `size` は `"medium"`、`disabled` は `false` になります。
- これにより「省略されたときの安全な初期値」を保証できます。

> **用語: デフォルト値（既定値）** … Propsが渡されなかったときに使われる初期値。`= 値` で指定する。

解説3: Propsの値からスタイルを組み立てる

```
      style={{
        // 背景色（props 由来）
        backgroundColor: color,
        // size に対応する余白文字列
        padding: sizeStyles[size],
        // 無効化されてたら半透明
        opacity: disabled ? 0.5 : 1,
        // マウスカーソルの形
        cursor: disabled ? "not-allowed" : "pointer",
      }}
      disabled={disabled}
```

- 受け取ったProps（`color` `size` `disabled`）を使って、見た目を動的に組み立てています。
- `sizeStyles[size]` は、サイズ名（"medium"など）をキーにして対応する余白の値を取り出しています。
- `disabled ? 0.5 : 1` のように三項演算子で、無効化されているかどうかで見た目を変えています。
- `disabled={disabled}` で、ボタンを実際にクリック不可にしています。

> **用語: 動的スタイル** … Propsやstateの値に応じて変化する見た目。三項演算子などで切り替える。

---

children Props

コンポーネントのタグで囲んだ中身は、`children`（子要素）として渡されます。これにより「外側の枠だけ用意して、中身は呼び出し側が自由に決められる」コンポーネントが作れます。

```tsx
// CardProps の型定義
type CardProps = {
  title: string;
  // React.ReactNode = 「React で描画できるもの全部」を表す特別な型。
  // 文字列、数値、JSX要素、その配列、null、undefined… すべて受け入れる。
  // children という名前は React の特別な予約名。タグで囲んだ中身が自動的にここに入る。
  children: React.ReactNode;
};

function Card({ title, children }: CardProps) {
  return (
    // インラインスタイルでカードの見た目を作る
    <div
      style={{
        // 1px、実線、薄いグレーの枠線
        border: "1px solid #ddd",
        // 角丸
        borderRadius: "8px",
        // 内側の余白
        padding: "16px",
        // 外側の余白
        margin: "8px",
      }}
    >
      {/* タイトル部分。borderBottom で下線、paddingBottom で下線との余白を確保 */}
      <h2 style={{ borderBottom: "1px solid #eee", paddingBottom: "8px" }}>
        {title}
      </h2>
      {/* {children} と書くと、親で <Card>...</Card> の間に書いた要素がここに展開される */}
      <div>{children}</div>
    </div>
  );
}

function App() {
  return (
    <div>
      {/* 開始タグと終了タグで囲んだ中身（<p>2つ）が children として渡される */}
      <Card title="お知らせ">
        <p>新機能がリリースされました！</p>
        <p>詳しくはこちらをご覧ください。</p>
      </Card>

      {/* 別の呼び出し。中身に画像と段落を入れている。 */}
      <Card title="プロフィール">
        {/* <img> は自己閉じタグ。src は画像URL、alt は代替テキスト（読み上げ・画像未表示時用）
*/}
        <img src="/avatar.png" alt="アバター" />
        <p>田中太郎</p>
```

```
      </Card>
    </div>
  );
}
```

この結果、画面には2つのカードが表示されます:

【お知らせカード】枠線で囲まれた領域に「お知らせ」という見出しと、2行のテキスト

【プロフィールカード】枠線で囲まれた領域に「プロフィール」という見出しと、画像・名前

▼ コードを1つずつ分解して解説

解説1: children を受け取る型を定義する

```
type CardProps = {
  title: string;
  children: React.ReactNode;
};
```

- `children`（チルドレン）は**特別な予約名**で、コンポーネントのタグで囲んだ中身が自動的にここに入ります。
- `React.ReactNode` は「**Reactで描画できるものすべて**」を表す型です。文字列・数値・JSX要素・その配列など、何でも受け取れます。
- これにより「外側の枠だけ用意し、中身は使う側が自由に決める」部品が作れます。

> **用語: children（子要素）** … コンポーネントの開始タグと終了タグで囲んだ中身。`<Card>ここ</Card>` の「ここ」の部分。

解説2: 受け取った children を表示位置に置く

```
      <h2 style={{ borderBottom: "1px solid #eee", paddingBottom: "8px" }}>
        {title}
      </h2>
      <div>{children}</div>
```

- `{title}` でPropsのタイトルを見出しに表示しています。
- `{children}` と書くと、親が `<Card>...</Card>` の間に書いた要素が、ちょうどこの位置に展開されます。
- 枠やタイトルは固定で、中身だけ差し替えられる「テンプレート」のような部品になります。

> **用語: コンポジション** … 枠コンポーネントの中に別の要素を差し込んで組み合わせる設計。childrenで実現する。

解説3: 親が中身を入れて使う

```
<Card title="お知らせ">
  <p>新機能がリリースされました！</p>
  <p>詳しくはこちらをご覧ください。</p>
</Card>
```

- `<Card>...</Card>` の**開始タグと終了タグで囲んだ中身**（`<p>` 2つ）が、`children` として子に渡されます。
- `title="お知らせ"` は通常のPropsとして渡されます。
- 同じ `Card` でも、中身を変えれば「お知らせカード」「プロフィールカード」など、いろいろなカードが作れます。

> **用語: ラッパーコンポーネント** … 中身を囲んで装飾や枠を付ける役割のコンポーネント。Cardが典型例。

3.3 コンポーネントの分割の考え方

コンポーネントを分割する判断基準:

1. **再利用されるか**: 複数箇所で使う UI は別コンポーネントにする
2. **責務が明確か**: 1つのコンポーネントが1つの役割を持つ
3. **複雑すぎないか**: 1ファイルが 100行を超えたら分割を検討する
4. **独立してテストできるか**: テストしやすい単位に分ける

```
src/                          # ソースコードのルートフォルダ
├── components/               # 再利用可能なUI部品を置く
│   ├── common/               # 汎用コンポーネント（プロジェクト横断で使う）
│   │   ├── Button.tsx        # ボタン
│   │   ├── Card.tsx          # 枠付きカード
│   │   ├── Input.tsx         # 入力欄
│   │   └── Modal.tsx         # モーダル（ポップアップ）
│   ├── layout/               # 画面レイアウト系（ページ全体の骨組み）
│   │   ├── Header.tsx        # 上部ヘッダー
│   │   ├── Footer.tsx        # 下部フッター
│   │   └── Sidebar.tsx       # サイドバー
│   └── book/                 # 書籍機能専用のコンポーネント群
│       ├── BookCard.tsx      # 書籍1件分のカード
│       ├── BookList.tsx      # 書籍リスト全体
│       ├── BookDetail.tsx    # 詳細表示
│       └── BookForm.tsx      # 登録・編集フォーム
├── pages/                    # 1ページ＝1ファイル（ルーティング先）
│   ├── HomePage.tsx          # トップページ
│   ├── BookListPage.tsx      # 書籍一覧ページ
│   └── BookDetailPage.tsx    # 書籍詳細ページ
└── App.tsx                   # アプリ全体のルート（最上位コンポーネント）
```

3.4 書籍カードコンポーネントの例

実際のアプリで使うような、少し本格的なコンポーネントを作ってみましょう。

> **▼ このコードがやること（先に日本語で）**：これまで学んだ要素を全部使って、**1冊分の「書籍カード」部品**を作ります。流れは――①本1冊の「形」を `Book` 型で決める、②この部品が親から受け取る材料（props）の「形」を `BookCardProps` 型で決める（表示する本のデータ＋「カートに追加」「お気に入り切替」されたときに呼ぶ関数など）、③受け取った props を**分割代入**で取り出し、④星評価などを計算して、⑤カードの見た目（JSX）を返す。少し長いですが、**型 → props → 表示** という、これまでの章の集大成です。コードの後ろで、初めて見る書き方を分解して解説します。

```tsx
// types.ts - 型定義
// 書籍データ1件分の「形」を表す型
type Book = {
  // 一意なID（DBの主キーに相当）
  id: number;
  // 書名
  title: string;
  // 著者名
  author: string;
  // 価格（円）
  price: number;
  // 評価1〜5
  rating: number;
  // 在庫があるか
  isAvailable: boolean;
  // ? でオプショナル: 画像URLが無い書籍もある
  coverImage?: string;
  // 出版日（ISO形式の文字列 "2025-01-15"）
  publishedDate: string;
  // タグの配列（複数の文字列）
  tags: string[];
};

// BookCard.tsx - 書籍カードコンポーネント
// このコンポーネントが親から受け取る props の型
type BookCardProps = {
  // 表示する書籍データ
  book: Book;
  // カート追加時のコールバック関数
  onAddToCart: (bookId: number) => void;
  // お気に入り切替時のコールバック関数
  onToggleFavorite: (bookId: number) => void;
  // 現在お気に入り状態か
  isFavorite: boolean;
};

// 分割代入で props を取り出す
function BookCard({ book, onAddToCart, onToggleFavorite, isFavorite }: BookCardProps) {
  // 星の表示を作る関数
  // ★を rating 個、☆を (5 - rating) 個並べて文字列を作る。
  // "★".repeat(3) は "★★★"（3回繰り返した文字列）になる JS の文字列メソッド。
  const renderStars = (rating: number): string => {
    return "★".repeat(rating) + "☆".repeat(5 - rating);
  };

  return (
    // カード全体の枠
    <div
      style={{
```

```
        // 薄い枠線
        border: "1px solid #ddd",
        // 大きめ角丸
        borderRadius: "12px",
        // 内側の余白
        padding: "16px",
        // 最大幅
        maxWidth: "300px",
        // ふんわり影
        boxShadow: "0 2px 8px rgba(0,0,0,0.1)",
      }}
    >
      {/* 表紙画像
          三項演算子 条件 ? A : B で、coverImage の有無により表示を切り替え。
          coverImage が "string" → truthy → 画像を表示
          coverImage が undefined → falsy → No Image プレースホルダーを表示 */}
      {book.coverImage ? (
        <img
          // 画像URL
          src={book.coverImage}
          // 代替テキスト（テンプレートリテラル）
          alt={`${book.title}の表紙`}
          // 横幅100%・角丸
          style={{ width: "100%", borderRadius: "8px" }}
        />
      ) : (
        <div
          style={{
            width: "100%",
            height: "200px",
            // 薄いグレー
            backgroundColor: "#f0f0f0",
            borderRadius: "8px",
            // 中身を flex レイアウトに
            display: "flex",
            // 縦方向中央寄せ
            alignItems: "center",
            // 横方向中央寄せ
            justifyContent: "center",
            color: "#999",
          }}
        >
          No Image
        </div>
      )}

      {/* 書籍情報。h3 のマージンを上12px・下4pxに調整 */}
      <h3 style={{ margin: "12px 0 4px" }}>{book.title}</h3>
      <p style={{ color: "#666", margin: "0 0 8px" }}>{book.author}</p>
```

```
      {/* 評価。色は山吹色（#f39c12）。星と数値を並べて表示 */}
      <p style={{ color: "#f39c12", margin: "0 0 8px" }}>
        {renderStars(book.rating)} ({book.rating}/5)
      </p>

      {/* タグ一覧
          display:flex でタグを横並びに、flexWrap:wrap で幅が足りなければ折り返す */}
      <div style={{ display: "flex", gap: "4px", flexWrap: "wrap", marginBottom: "8px"
}}>
        {/* タグ配列を map で <span> に変換。key にタグ文字列を使う（タグは重複しない前提） */}
        {book.tags.map((tag) => (
          <span
            key={tag}
            style={{
              // 淡い青背景
              backgroundColor: "#e8f4fd",
              // 青文字
              color: "#1a73e8",
              padding: "2px 8px",
              // 強めの角丸でピル状に
              borderRadius: "12px",
              fontSize: "12px",
            }}
          >
            {tag}
          </span>
        ))}
      </div>

      {/* 価格と在庫状態を左右に配置（space-between） */}
      <div style={{ display: "flex", justifyContent: "space-between", alignItems:
"center" }}>
        <span style={{ fontSize: "20px", fontWeight: "bold" }}>
          {/* 3桁カンマ区切り */}
          ¥{book.price.toLocaleString()}
        </span>
        {/* 在庫がある=緑、ない=赤 */}
        <span style={{ color: book.isAvailable ? "green" : "red", fontSize: "14px" }}>
          {book.isAvailable ? "在庫あり" : "在庫なし"}
        </span>
      </div>

      {/* アクションボタン */}
      <div style={{ display: "flex", gap: "8px", marginTop: "12px" }}>
        <button
          // アロー関数で包む理由: onAddToCart に book.id を引数として渡したいから。
          // onClick={onAddToCart(book.id)} と書くとレンダー中に即実行されてしまう。
          onClick={() => onAddToCart(book.id)}
          // 在庫なし → disabled=true（クリック不可）
          disabled={!book.isAvailable}
```

```
          style={{
            // 残りスペースを埋める
            flex: 1,
            padding: "8px",
            // 状態で色変化
            backgroundColor: book.isAvailable ? "#1a73e8" : "#ccc",
            color: "white",
            border: "none",
            borderRadius: "6px",
            cursor: book.isAvailable ? "pointer" : "not-allowed",
          }}
        >
          カートに追加
        </button>
        <button
          // クリックでお気に入り切り替え。コールバックに book.id を渡す。
          onClick={() => onToggleFavorite(book.id)}
          style={{
            padding: "8px 12px",
            // 背景なし
            backgroundColor: "transparent",
            border: "1px solid #ddd",
            borderRadius: "6px",
            cursor: "pointer",
            fontSize: "18px",
          }}
        >
          {/* お気に入り中は塗りつぶしハート、未登録は中抜きハート */}
          {isFavorite ? "♥" : "♡"}
        </button>
      </div>
    </div>
  );
}

// 使い方
function App() {
  // サンプルデータ。: Book で型を明示してプロパティ漏れを防ぐ。
  const sampleBook: Book = {
    id: 1,
    title: "React入門ガイド",
    author: "田中太郎",
    price: 2800,
    rating: 4,
    isAvailable: true,
    publishedDate: "2025-01-15",
    tags: ["React", "TypeScript", "フロントエンド"],
  };

  return (
```

```
    // 親 → 子へ props を渡す
    // book={sampleBook} はオブジェクトを渡している（{} の中の {} ではない、変数を渡しているだけ）
    <BookCard
      book={sampleBook}
      // インラインでコールバック関数を渡す。alert はブラウザの組み込み関数。
      // テンプレートリテラル `...${id}...` は文字列内に変数を埋め込む書き方。
      onAddToCart={(id) => alert(`書籍ID: ${id} をカートに追加しました`)}
      onToggleFavorite={(id) => alert(`書籍ID: ${id} のお気に入りを切り替えました`)}
      isFavorite={false}
    />
  );
}
```

この結果、画面にはカード型のUIが表示されます:

【No Image】（灰色のプレースホルダー）

React入門ガイド

田中太郎

★★★★☆ (4/5)

React　TypeScript　フロントエンド　（タグバッジ）

¥2,800 在庫あり（緑色）

【カートに追加】ボタン【♡】ボタン

▼ コードを1つずつ分解して解説

解説1: 書籍データの型を定義する

```
type Book = {
  // 一意なID（DBの主キーに相当）
  id: number;
  // 書名
  title: string;
  // 著者名
  author: string;
  // 価格（円）
  price: number;
  // 評価1〜5
  rating: number;
  // 在庫があるか
  isAvailable: boolean;
  // ? でオプショナル：画像URLが無い書籍もある
  coverImage?: string;
  // 出版日
  publishedDate: string;
  // タグの配列
  tags: string[];
};
```

- まず「本1冊のデータの形」を `Book` 型で決めます。これまでより項目が多いですが、考え方は同じです。
- `coverImage?` の `?` は省略可能の印で、「表紙画像が無い本もある」ことを表しています。
- `tags: string[]` はタグの配列（複数の文字列）です。

> **用語: 型定義** … データがどんな項目をどんな種類で持つかを決めること。表示や受け渡しの安全性が増す。

解説2: この部品が受け取るPropsを定義する

```
type BookCardProps = {
  // 表示する書籍データ
  book: Book;
  // カート追加時のコールバック
  onAddToCart: (bookId: number) => void;
  // お気に入り切替時のコールバック
  onToggleFavorite: (bookId: number) => void;
  // 現在お気に入り状態か
  isFavorite: boolean;
};
```

- この部品は「**表示する本のデータ（`book`）**」と、「**ボタンが押されたとき親に知らせる関数（`onAddToCart` / `onToggleFavorite`）**」、「**お気に入り状態（`isFavorite`）**」を受け取ります。
- `onAddToCart: (bookId: number) => void` は「数値を1つ受け取り、戻り値なしの関数」を表します。どの本かを `bookId` で親に伝えます。
- 「データ＋操作を知らせる関数」をまとめて受け取るのが、実用的なコンポーネントの典型です。

> **用語: イベント用コールバック** … `onXxx` という名前で親から渡される関数。子で操作が起きたとき呼んで親に通知する。

解説3: Propsを分割代入し、表示用の関数を用意する

```
function BookCard({ book, onAddToCart, onToggleFavorite, isFavorite }: BookCardProps) {
  const renderStars = (rating: number): string => {
    return "★".repeat(rating) + "☆".repeat(5 - rating);
  };
```

- 引数の `{ book, onAddToCart, ... }` で、4つのPropsを一度に分割代入で取り出しています。
- `renderStars` は「評価の数だけ★、残りを☆で埋めた文字列」を作る関数です。
- `"★".repeat(3)` は `"★★★"` のように「文字を指定回数くり返す」メソッドです。評価4なら `★★★★☆` になります。

> **用語: repeat** … 文字列を指定した回数だけくり返した新しい文字列を作るメソッド。

解説4: 表紙画像の有無で表示を切り替える

```
      {book.coverImage ? (
        <img
          src={book.coverImage}
          alt={`${book.title}の表紙`}
          style={{ width: "100%", borderRadius: "8px" }}
        />
      ) : (
        <div style={{ /* ...グレーの箱... */ }}>
          No Image
        </div>
      )}
```

- 三項演算子で「`book.coverImage` **があれば画像、なければ「No Image」の箱**」を切り替えています。
- `alt={` ${book.title}の表紙 `}` はテンプレートリテラルで、「○○の表紙」という代替テキストを動的に作っています。

- 省略可能なProps（画像URL）の「ある/ない」を安全に扱う典型パターンです。

> **用語: プレースホルダー** … 本来の内容がないときに代わりに表示する仮の表示。ここでは「No Image」の箱。

解説5: ボタンにアロー関数で包んだコールバックを渡す

```
        onClick={() => onAddToCart(book.id)}
        disabled={!book.isAvailable}
    ...
        onClick={() => onToggleFavorite(book.id)}
```

- `onClick={() => onAddToCart(book.id)}` のように**アロー関数で包む**のは、「**クリックされたときに `book.id` を引数として渡したい**」からです。
- `onClick={onAddToCart(book.id)}` と書くと、表示の瞬間に即実行されてしまうので誤りです。
- `disabled={!book.isAvailable}` は「在庫が無ければボタンを押せなくする」指定です。`!` は「否定（〜でない）」を表します。

> **用語: 引数付きイベント** … クリック時に値を渡したいとき、`() => 関数(値)` とアロー関数で包んで渡す。

---

4. State (useState)

4.1 state とは何か

state（ステート：状態。コンポーネントが内部で保持する「変化するデータ」）は、コンポーネントが持つ「変化するデータ」です。state が変わると、React は自動的にそのコンポーネントを**再レンダリング**（さいレンダリング：Re-render。state や props が変わったときに、コンポーネント関数が再度実行されて新しい JSX が作られ、画面が更新されること）します。

> **なぜ普通の変数ではダメか:** React は「state が変わったかどうか」だけを監視しています。普通のローカル変数（ `let count = 0;` のようなもの）は React の管理外なので、いくら値を変えても画面は更新されません。値を「画面に反映される形で覚える」には `useState` という **フック**（Hook：フック。`use` で始まる React の特別な関数。コンポーネントに機能を追加する）を使う必要があります。

普通の変数と state の違いを見てみましょう。

```
// NG: 普通の変数は変更しても再レンダリングされない
function Counter() {
  // let は再代入できる変数を作るキーワード。
  // ただしこの変数は React の管理外なので、変更しても画面更新は起きない。
  // しかも関数が再実行されるたびに 0 に戻されてしまう。
  let count = 0;

  // ボタンが押されたときに呼ぶ関数
  const handleClick = () => {
    // 値は変わるが、画面は更新されない！
    count += 1;
    // コンソールには 1, 2, 3... と出る
    console.log(count);
  };

  return (
    <div>
      {/* ずっと 0 のまま */}
      <p>カウント: {count}</p>
      <button onClick={handleClick}>+1</button>
    </div>
  );
}
```

> この結果、ボタンをクリックしても画面には「**カウント: 0**」のまま変わりません。コンソールには値が増えていきますが、React は変数の変化を検知できないのです。

なぜこうなるのかを、たとえ話で考えてみましょう。コンポーネント関数は、画面を描くたびに**最初から最後まで実行し直される**「使い捨てのメモ用紙」のようなものです。`let count = 0;` と書いた変数はメモ用紙に書いた数字なので、描き直しのたびに新しい用紙になり、また 0 から始まります。一方 `useState` で作った state は、**React が用意したロッカーに預けた値**です。メモ用紙（関数）が何度新しくなっても、React がロッカーから最新の値を取り出して渡してくれます。



図3-4: 普通の変数と state の違い。state の値は React が預かっていて、再実行のたびに最新の値を返してくれます

ポイントは2つです。

- 普通の変数を書き換えても、React には「画面を描き直して」という合図が届きません（図の左②）。
- `setCount` は「値を預け直す」と同時に「描き直して」という合図を React に送る関数です（図の右②）。だから画面が変わります。

4.2 useState の使い方

> **▼ このコードがやること（先に日本語で）**：「ボタンを押すと数字が増えたり減ったりする**カウンター**」を作ります。これはReactで最も基本的な「動く部品」です。カギは `useState` ——「**コンポーネントが値を覚えておく**ための道具」です。`const [count, setCount] = useState(0)` の一行で「今の値（`count`）」と「値を変える関数（`setCount`）」をセットで受け取り、ボタンが押されたら `setCount` を呼んで値を変えます。**`setCount` を呼ぶと、Reactが自動で画面を描き直してくれる**——この「値を変えたら画面も自動更新」が `useState` の最大のポイントです。

```tsx
// ============================================================================
// シンプルなカウンターコンポーネント（詳細コメント版）
// ----------------------------------------------------------------------------
// 「ボタンを押すと数値が +1 / -1 され、画面が自動更新される」
// React の最も基本的なインタラクティブ部品。
// ============================================================================

// React の useState フックを使うので import で取り込む。
// `useState` は「コンポーネントが値を覚えておく」ための関数。
import { useState } from "react";

// 関数コンポーネント = 大文字始まりの関数で、JSXを返すもの。
function Counter() {
  // ──────────────────────────────────────────────────────────────
  // (1) state（状態）を作る
  // ──────────────────────────────────────────────────────────────
  // useState<number>(0) は「数値型の状態を初期値0で作って」という意味。
  //
  // 戻り値は配列で2つの要素が入っている:
  //   [現在の値, 値を更新するための関数]
  //
  // この配列を「分割代入」で count, setCount に取り出している。
  //   → const count = 戻り値[0];  ← 現在の数値
  //   → const setCount = 戻り値[1]; ← 数値を変える関数
  //
  // 重要: count の値を直接変えることはできない（const なので再代入不可）。
  //       必ず setCount(...) を呼ぶ。setCount を呼ぶと、Reactが
  //       「state が変わったぞ」と認識し、自動で画面を再描画してくれる。
  const [count, setCount] = useState<number>(0);

  // ──────────────────────────────────────────────────────────────
  // (2) ボタンが押されたときの処理（イベントハンドラ）
  // ──────────────────────────────────────────────────────────────
  // アロー関数で「+1する処理」を定義。
  // setCount を呼ぶことで、count が 0 → 1 → 2 ... と更新される。
  const handleIncrement = () => {
    // 「今の count + 1」を新しい値として設定
    setCount(count + 1);
  };

  // 同じく「-1する処理」
  const handleDecrement = () => {
    setCount(count - 1);
  };

  // 「0 に戻す処理」
  const handleReset = () => {
    setCount(0);
  };
```

```
  // ──────────────────────────────────────────────────────────
  // (3) 画面に出すJSXを返す
  // ──────────────────────────────────────────────────────────
  // {count} の波カッコはJSX中にJavaScriptの値を埋め込む書き方。
  // count が 5 なら、「カウント: 5」と表示される。
  //
  // onClick={handleDecrement} は「-1ボタンが押されたら handleDecrement を呼んで」の意味。
  // 注意: onClick={handleDecrement()} と () を付けると「即実行」になりバグの元。
  //       関数自体を渡す（= 関数名のみ書く）のが正解。
  return (
    <div>
      <h2>カウンター</h2>
      <p>カウント: {count}</p>
      <button onClick={handleDecrement}>-1</button>
      <button onClick={handleReset}>リセット</button>
      <button onClick={handleIncrement}>+1</button>
    </div>
  );
}
```

▼ ブラウザでの見た目（初期表示）:

```
┌──────────────────────────┐
│  カウンター              │ ← <h2>
│  カウント: 0             │ ← {count} に 0 が入る
│ [ -1 ] [ リセット ] [ +1 ] │ ← 3つの <button>
└──────────────────────────┘
```

▼ 動作の流れ:

操作	count の変化	画面表示
初期表示	0	カウント: 0
「+1」を押す	0 → 1	カウント: 1
「+1」をもう1回	1 → 2	カウント: 2
「-1」を押す	2 → 1	カウント: 1
「リセット」を押す	1 → 0	カウント: 0

▼ **重要ポイント**: 1. `count` は**読み取り専用**。直接 `count = 5;` とは書けない（書いても画面に反映されない） 2. 値を変えたいときは必ず `setCount(...)` を呼ぶ 3. `setCount` を呼ぶと、React が裏で関数 `Counter` をもう一度実行し、新しい count で画面を再描画する

▼ コードを1つずつ分解して解説

解説1: useStateをimportする

```
import { useState } from "react";
```

- `import { useState } from "react";` は「Reactから `useState` という道具を取り込む」という宣言です。
- `useState` を使うコンポーネントのファイルには、必ずこの1行が必要です。
- `useState` は「**コンポーネントが値を覚えておく**」ための関数（フック）です。

> **用語: フック（Hook）** … `use` で始まるReactの特別な関数。コンポーネントに「状態を持つ」などの機能を追加する。

解説2: stateを作る

```
  const [count, setCount] = useState<number>(0);
```

- `useState<number>(0)` は「**数値の状態を、初期値0で作って**」という意味です。`<number>` が型、`(0)` が初期値です。
- 戻り値は配列で2つ入っています：`[現在の値, 値を変える関数]`。これを分割代入で `count` と `setCount` に取り出しています。
- `count` が今の数値、`setCount` がその数値を変える関数です。`count` は直接書き換えられず、必ず `setCount` を使います。

> **用語: state（状態）** … コンポーネントが内部で覚えておく「変化する値」。変わると画面が自動で更新される。

解説3: ボタンが押されたときの処理を書く

```
  const handleIncrement = () => {
    // 「今の count + 1」を新しい値として設定
    setCount(count + 1);
  };
```

- `handleIncrement` は「+1ボタンが押されたときの処理」を表す関数です。
- `setCount(count + 1)` で「今の値 + 1」を新しい値として設定します。
- **`setCount` を呼ぶと、Reactが自動で画面を描き直す**——これが `useState` の最大のポイントです。同様に `setCount(count - 1)` で-1、`setCount(0)` でリセットができます。

> **用語: イベントハンドラ** … ボタンクリックなどの操作が起きたときに呼ばれる関数。`handleXxx` という名前をよく使う。

解説4: stateを画面に表示し、ボタンに処理を結びつける

```
      <p>カウント: {count}</p>
      <button onClick={handleDecrement}>-1</button>
      <button onClick={handleReset}>リセット</button>
      <button onClick={handleIncrement}>+1</button>
```

- `{count}` で、今の数値を画面に表示します。`count` が5なら「カウント: 5」になります。
- `onClick={handleIncrement}` で、ボタンと処理を結びつけます。**関数名だけを渡す**点に注意（`handleIncrement()` と `()` を付けると即実行されてバグになります）。
- 数値が変わると `setCount` が画面を描き直すので、表示も自動で更新されます。

> **用語: 再レンダリング** … stateが変わったとき、コンポーネント関数が再実行されて画面が更新されること。

---

useState の型推論

```
// 型を明示的に指定するパターン（<>はジェネリクス: 型を1つ渡す）
// useState<number>(0) は「number型の状態を初期値0で作る」と書いている。
const [count, setCount] = useState<number>(0);
const [name, setName] = useState<string>("");
const [isVisible, setIsVisible] = useState<boolean>(false);

// 初期値から型推論される（明示しなくてもOK）
// 初期値 0 を見て TypeScript が「number 型だな」と自動で判断する。
// number と推論
const [count, setCount] = useState(0);
// string と推論
const [name, setName] = useState("");
// boolean と推論
const [isVisible, setIsVisible] = useState(false);

// null を使う場合は明示的な型指定が必要
// 理由: 初期値が null だけだと「null 型」と推論されてしまい、
//         後から User オブジェクトを入れられなくなる。
// User | null は「User型 または null」のユニオン型。
const [user, setUser] = useState<User | null>(null);
```

> **つまずきポイント（フックを if の中で呼ぶ）**：`useState` などのフックは、**コンポーネントの一番上の階層で、毎回同じ順番で**呼ぶ決まりがあります（Rules of Hooks：フックのルール）。React は「1番目の useState」「2番目の useState」…と**呼ばれた順番**でロッカー（図3-4）を割り当てているので、`if` の中で呼ぶと、条件によって順番がずれて、別の state の値を取り出してしまうからです。第6章で作る Next.js プロジェクト（ESLint を有効にしたもの）では、ESLint（コードの書き方チェッカー）が次のエラーで教えてくれます。

```
React Hook "useState" is called conditionally. React Hooks must be called in the exact same order in every component render
```

```tsx
// ❌ NG: 条件によって useState が呼ばれたり呼ばれなかったりする
function Greeting({ isLoggedIn }: { isLoggedIn: boolean }) {
  if (isLoggedIn) {
    const [name, setName] = useState("");
  }
  const [count, setCount] = useState(0);
  return <p>{count}</p>;
}
```

```tsx
// ✅ OK: フックは必ず一番上で、毎回同じ順番で呼ぶ
function Greeting({ isLoggedIn }: { isLoggedIn: boolean }) {
  const [name, setName] = useState("");
  const [count, setCount] = useState(0);
  // 条件分岐は、フックを呼び終わったあと（表示の部分）で行う
  return <p>{isLoggedIn ? `${name}さん: ${count}` : "ログインしてください"}</p>;
}
```

4.3 state の更新とリレンダリング



重要なポイント: `setCount` を呼んでも、その場では `count` の値は変わりません。次の再レンダリング時に新しい値になります。これは React の「state は1回の関数実行中は同じ値で固定される」というルールによるものです（**state は再レンダリング時に最新化される**）。

```tsx
function Counter() {
  // count, setCount を useState で作る
  const [count, setCount] = useState<number>(0);

  const handleClick = () => {
    // 1回目: 「次のレンダリングで count を (現在の count=0) + 1 = 1 にして」と予約
    setCount(count + 1);
    // この時点で count はまだ 0（更新は予約されただけ）
    // まだ 0 のまま！（次のレンダリングで 1 になる）
    console.log(count);
    // 2回目: count はまだ 0 → setCount(0 + 1) になる → 結局 1 を予約しているだけ
    // count はまだ 0 なので、0 + 1 = 1 になる（2 にはならない！）
    setCount(count + 1);
  };

  return (
    <div>
      <p>カウント: {count}</p>
      <button onClick={handleClick}>+2 のつもり（実際は+1）</button>
    </div>
  );
}
```

> この結果、ボタンをクリックすると「**カウント: 1**」になります。+2 になると期待しますが、同じレンダリング内では `count` は古い値（0）のままなので、両方とも `setCount(0 + 1)` = `setCount(1)` になります。

この動きは「**レンダーごとのスナップショット（写真）**」と考えると分かりやすくなります。コンポーネント関数が1回実行されるたびに、その時点の state を写真が1枚撮られ、その回の関数の中（クリック時の処理も含む）では、ずっとその写真の値を見ています。`setCount` は写真を書き換えるのではなく、「次の写真ではこの値にしてね」と**予約**するだけです。

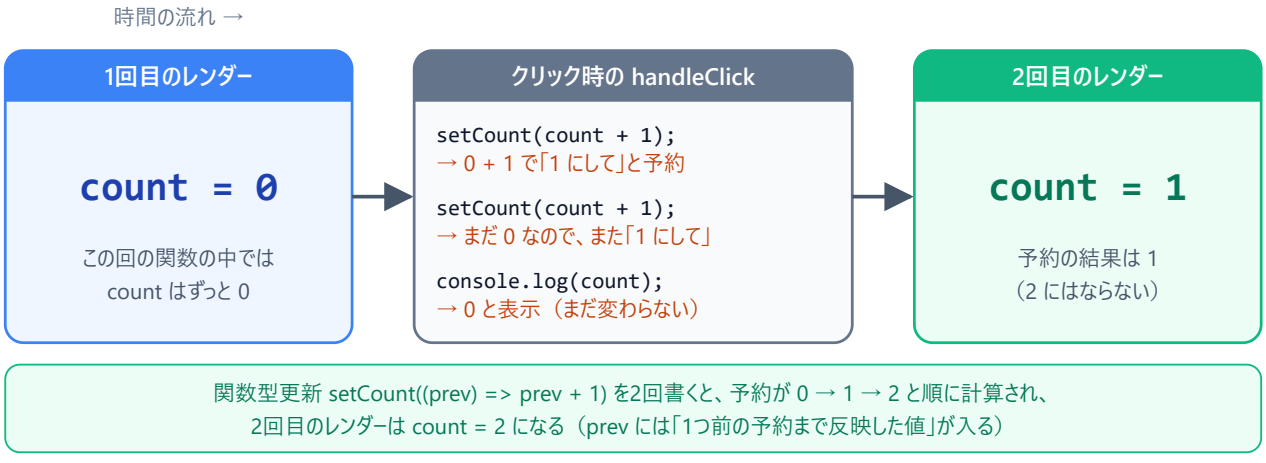


図3-5: レンダーごとの「スナップショット」。1回のレンダーの中で、state の値は写真のように固定されています

解決策: 関数型更新を使う

`setXxx(値)` の代わりに `setXxx((prev) => 新しい値)` と書くと、React は「予約された変更を順番に適用」してくれます。`prev` には「これまでに予約された変更を全部反映した最新値」が渡されます。

```
function Counter() {
  const [count, setCount] = useState<number>(0);

  const handleClick = () => {
    // prev には常に「最新の値」が渡される
    // 関数型更新: setCount に「現在値 → 次の値」という関数を渡す
    // 0 → 1
    setCount((prev) => prev + 1);
    // ↑の予約が反映された結果が prev に渡る → 1 + 1 = 2
    // 1 → 2
    setCount((prev) => prev + 1);
  };

  return (
    <div>
      <p>カウント: {count}</p>
      <button onClick={handleClick}>+2</button>
    </div>
  );
}
```

この結果、ボタンをクリックすると「**カウント: 2**」→「**カウント: 4**」→ … と2ずつ増えます。関数型更新なら、直前の最新値に基づいて計算されるため、正しく動作します。

「予約」の処理順を Node.js で確かめる

▼ このコードがやること（先に日本語で）：React の中で起きている「state 更新の予約」を、React を使わずに小さな模型で再現します。予約には「次の値そのもの（例: `1`）」と「前の値から次の値を作る関数（例: `(prev) => prev + 1`）」の2種類があり、React は予約を**書いた順番どおりに**処理します。3つのパターンで、次のレンダーの `count` がいくつになるかを計算します。

```ts
// ファイル名: state-queue.ts
// React の「state 更新の予約」をまねした小さな模型
//  （React 本体のコードではなく、考え方をつかむためのものです）

// 予約できるのは「次の値そのもの」か「前の値から次の値を作る関数」
type Update = number | ((prev: number) => number);

// 予約を前から順番に処理して、次のレンダーで使う値を求める
function applyUpdates(current: number, queue: Update[]): number {
  let value = current;
  for (const update of queue) {
    // 関数なら「ここまでの最新の値」を渡して計算し、値ならそのまま置き換える
    value = typeof update === "function" ? update(value) : update;
  }
  return value;
}

// このレンダーの count は 0 で固定されている（スナップショット）
const count = 0;

// ---- パターンA: setCount(count + 1) を2回 ----
const queueA: Update[] = [];
// 1回目: count は 0 なので「1 にして」と予約
queueA.push(count + 1);
// 2回目: count はまだ 0 なので、また「1 にして」と予約
queueA.push(count + 1);
console.log("A: 予約の中身 =", queueA);
console.log("A: 次のレンダーの count =", applyUpdates(count, queueA));

// ---- パターンB: setCount((prev) => prev + 1) を2回 ----
const queueB: Update[] = [];
queueB.push((prev) => prev + 1);
queueB.push((prev) => prev + 1);
console.log("B: 次のレンダーの count =", applyUpdates(count, queueB));

// ---- パターンC: setCount(5) のあとに setCount((prev) => prev * 2) ----
const queueC: Update[] = [];
queueC.push(5);
queueC.push((prev) => prev * 2);
console.log("C: 次のレンダーの count =", applyUpdates(count, queueC));
```

▼ 実行結果（`node state-queue.ts`）

```
A: 予約の中身 = [ 1, 1 ]
A: 次のレンダーの count = 1
B: 次のレンダーの count = 2
C: 次のレンダーの count = 10
```

この3つの結果は、本物の React で同じボタン処理を書いたときの結果（A は 1、B は 2、C は 10）と一致します。

▼ コードを1つずつ分解して解説

解説1: 予約の2つの形を型で表す

```
type Update = number | ((prev: number) => number);
```

- `setCount(1)` のように**値**を渡す予約と、`setCount((prev) => prev + 1)` のように**関数**を渡す予約の2種類があることを、ユニオン型（`|`）で表しています。
- 関数のほうの `prev` は「previous（前の）」の略で、「1つ前の予約まで反映した値」が入ります。

解説2: 予約を順番に処理する

```
for (const update of queue) {
  value = typeof update === "function" ? update(value) : update;
}
```

- `typeof update === "function"` で「関数かどうか」を調べ、関数なら**ここまでの最新の値**を渡して呼び出し、値ならその値で置き換えます。
- パターンA は予約が `[1, 1]` なので「1 にする → 1 にする」で結果は 1。値の予約は、それまでの計算を**上書き**してしまうことが分かります。
- パターンC は「5 にする → 前の値（5）を2倍」で 10 になります。値と関数を混ぜても、書いた順に処理されます。

解説3: count がずっと 0 なのがポイント

```
const count = 0;
queueA.push(count + 1);
queueA.push(count + 1);
```

- 模型でも `count` は `const` で 0 に固定しています。これが「1回のレンダーの中では state はスナップショット（写真）」という意味です。
- だから `count + 1` は何回書いても `1` になります。前の値をもとに何度も更新したいときは、関数型更新 `(prev) => prev + 1` を使います。

> **用語: 関数型更新（updater function）** … `setXxx((prev) => 次の値)` のように、値の代わりに「前の値から次の値を計算する関数」を渡す書き方。

つまずきポイント（set した直後に値を使う）： `setCount` を呼んだ直後に `count` を使うと、まだ古い値のままです。「保存しました: 1件」のように**新しい値を使いたいときは、先に変数に計算しておき**、その変数を state 更新と表示の両方に使います。

```
// ❌ NG: setCount の直後でも count は古い値（0 件のとき「0 件になりました」と出る）
const handleAdd = () => {
  setCount(count + 1);
  alert(`${count} 件になりました`);
};
```

```
// ✅ OK: 次の値を先に変数に入れて、state 更新と表示の両方に使う
const handleAdd = () => {
  const nextCount = count + 1;
  setCount(nextCount);
  alert(`${nextCount} 件になりました`);
};
```

4.4 オブジェクトの state

▼ このコードがやること（先に日本語で）：1つの値（数字など）ではなく、「名前・メール・年齢…」を**まとめて持つオブジェクト**を state にする例です。ここで一番大事なのは更新のしかた――オブジェクトの一部だけを変えたいときも、**古いオブジェクトを直接書き換えてはいけません。**第2章で学んだ**スプレッド構文** `...` で「今の全項目をコピー」してから、変えたい項目だけを上書きした**新しいオブジェクトを作って** `setXxx` に渡します。「直接いじらず、新しく作り替えて差し替える」がReactのstate更新の鉄則です（理由はコード内で説明します）。

```tsx
// useState フックを取り込む
import { useState } from "react";

// プロフィール情報の「形」を表す型
type UserProfile = {
  name: string;
  email: string;
  age: number;
  // 自己紹介
  bio: string;
};

function ProfileEditor() {
  // オブジェクト state を作る。複数の関連プロパティをまとめて管理できる。
  const [profile, setProfile] = useState<UserProfile>({
    name: "田中太郎",
    email: "tanaka@example.com",
    age: 25,
    bio: "React を勉強中です",
  });

  // 名前を変更する関数
  // 引数 e の型 React.ChangeEvent<HTMLInputElement> は「<input>のChangeイベント」を表す。
  // e.target.value で入力欄の現在の文字列を取得できる。
  const handleNameChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    // スプレッド構文 ...profile で「現在のprofileの全プロパティを展開」してコピーし、
    // 続けて name: e.target.value で name だけ新しい値に上書きする。
    // この結果、新しい profile オブジェクトが作られる（=参照が変わる=Reactが変化を検知）。
    setProfile({
      ...profile,
      name: e.target.value,
    });
  };

  // メールを変更する関数（パターンは同じ）
  const handleEmailChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    setProfile({
      ...profile,
      email: e.target.value,
    });
  };

  // 年齢を変更する関数
  // <input type="number"> でも e.target.value は文字列なので Number() で数値に変換する。
  const handleAgeChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    setProfile({
      ...profile,
      age: Number(e.target.value),
    });
```

```
};

// 自己紹介を変更する関数
// <textarea> の場合は型が HTMLTextAreaElement に変わる点に注意。
const handleBioChange = (e: React.ChangeEvent<HTMLTextAreaElement>) => {
  setProfile({
    ...profile,
    bio: e.target.value,
  });
};

return (
  <div>
    <h2>プロフィール編集</h2>

    {/* 制御コンポーネント（Controlled Component）パターン:
        value と onChange を両方セットで指定し、表示値を state で完全に制御する。
        これにより state が「唯一の正解」となり、画面と data が常に同期する。 */}
    <div>
      <label>名前: </label>
      <input value={profile.name} onChange={handleNameChange} />
    </div>

    <div>
      <label>メール: </label>
      <input value={profile.email} onChange={handleEmailChange} />
    </div>

    <div>
      <label>年齢: </label>
      <input
        // 数値入力モード
        type="number"
        value={profile.age}
        onChange={handleAgeChange}
      />
    </div>

    <div>
      <label>自己紹介: </label>
      {/* textarea は HTML では <textarea>中身</textarea> だが、
          React では value 属性で中身を制御する。 */}
      <textarea value={profile.bio} onChange={handleBioChange} />
    </div>

    <h3>プレビュー</h3>
    <p>名前: {profile.name}</p>
    <p>メール: {profile.email}</p>
    <p>年齢: {profile.age}歳</p>
    <p>自己紹介: {profile.bio}</p>
```

```
    </div>
  );
}
```

> **制御コンポーネント / 非制御コンポーネント**: value と onChange の両方で state とつなぐ書き方が**制御コンポーネント**（Controlled Component：Reactのstateが入力値の真実の出どころ）。逆に value を指定せず DOM 任せにする書き方は**非制御コンポーネント**（Uncontrolled Component：DOM自身が値を保持し、必要なときだけ ref で読み取る）。本書では基本的に制御コンポーネントを使います。

> この結果、画面には4つの入力欄と、その下にプレビューが表示されます。入力欄に文字を入力すると、**リアルタイムにプレビュー部分が更新**されます。例えば名前の入力欄を「鈴木花子」に変更すると、プレビューの名前も即座に「**名前: 鈴木花子**」に変わります。

▼ コードを1つずつ分解して解説

解説1: 複数の項目を1つのオブジェクトstateで持つ

```
const [profile, setProfile] = useState<UserProfile>({
  name: "田中太郎",
  email: "tanaka@example.com",
  age: 25,
  bio: "React を勉強中です",
});
```

- 名前・メール・年齢・自己紹介を、**1つのオブジェクト**として state に持っています。バラバラに4つの state を作るより管理しやすくなります。
- `useState<UserProfile>(...)` の `<UserProfile>` で「この state の中身はUserProfile型」と指定しています。
- 初期値として、オブジェクトをそのまま渡しています。

> **用語: オブジェクトstate** … 複数の関連する値を1つのオブジェクトにまとめて持つstate。

解説2: スプレッド構文で「新しいオブジェクト」を作って更新する

```
const handleNameChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  setProfile({
    ...profile,
    name: e.target.value,
  });
};
```

- `e.target.value` は入力欄の現在の文字列です。`e` はイベント情報、`.target` が入力欄そのもの、`.value` が中身です。
- `...profile` は**スプレッド構文**で、「今の `profile` の全項目をコピーして展開する」という意味です。
- そのあと `name: e.target.value` で `name` だけを上書きします。結果として「**他の項目はそのまま・名前だけ新しい、新しいオブジェクト**」ができ、それを `setProfile` に渡します。

> **用語: スプレッド構文（ `...` ）** … オブジェクトや配列の中身を展開してコピーする記法。state更新で「新しいオブジェクトを作る」のに使う。

解説3: 入力の種類に応じてイベントの型を変える

```
const handleAgeChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  setProfile({ ...profile, age: Number(e.target.value) });
};
const handleBioChange = (e: React.ChangeEvent<HTMLTextAreaElement>) => {
  setProfile({ ...profile, bio: e.target.value });
};
```

- `e.target.value` は、たとえ `<input type="number">` でも**必ず文字列**です。数値が必要なら `Number(...)` で変換します。
- `<textarea>`（複数行入力）の場合は、イベントの型が `HTMLInputElement` から `HTMLTextAreaElement` に変わります。
- どのハンドラも「スプレッドでコピー＋1項目だけ上書き」という同じパターンです。

> **用語: 型変換** … `Number("25")` のように、文字列を数値などへ変換すること。入力値は文字列なので必要になる。

解説4: 制御コンポーネントで入力欄とstateを同期する

```
      <input value={profile.name} onChange={handleNameChange} />
```

- `value={profile.name}` で入力欄の表示を state の値に固定し、`onChange={handleNameChange}` で入力のたびに state を更新します。
- この「**value と onChange をセットで指定する**」書き方を**制御コンポーネント**と呼びます。state が「唯一の正解」になり、画面と値が常に一致します。
- だから入力するとプレビューもリアルタイムに更新されます。

> **用語: 制御コンポーネント** … 入力欄の値をstateで完全に管理する書き方。value と onChange を必ずセットで使う。

重要: オブジェクトの state を更新するときは、必ず**新しいオブジェクトを作成**してください。プロパティを直接変更してはいけません。

```
// NG: 直接変更（React が変化を検知できない）
profile.name = "新しい名前";
// 同じオブジェクト参照なので再レンダリングされない！
setProfile(profile);

// OK: 新しいオブジェクトを作成
setProfile({ ...profile, name: "新しい名前" });
```

なぜ直接書き換えるとダメなのでしょうか。React は state が変わったかどうかを、**中身を1つずつ見比べるのではなく、「前と同じ箱（オブジェクト）かどうか」だけで判断している**からです。たとえるなら、宅配便の受付係が「箱の中身」ではなく「箱の伝票番号」だけを見て「新しい荷物かどうか」を決めているようなものです。同じ箱の中身をこっそり入れ替えても、伝票番号が同じなので「新しい荷物は来ていない」と判断されてしまいます。

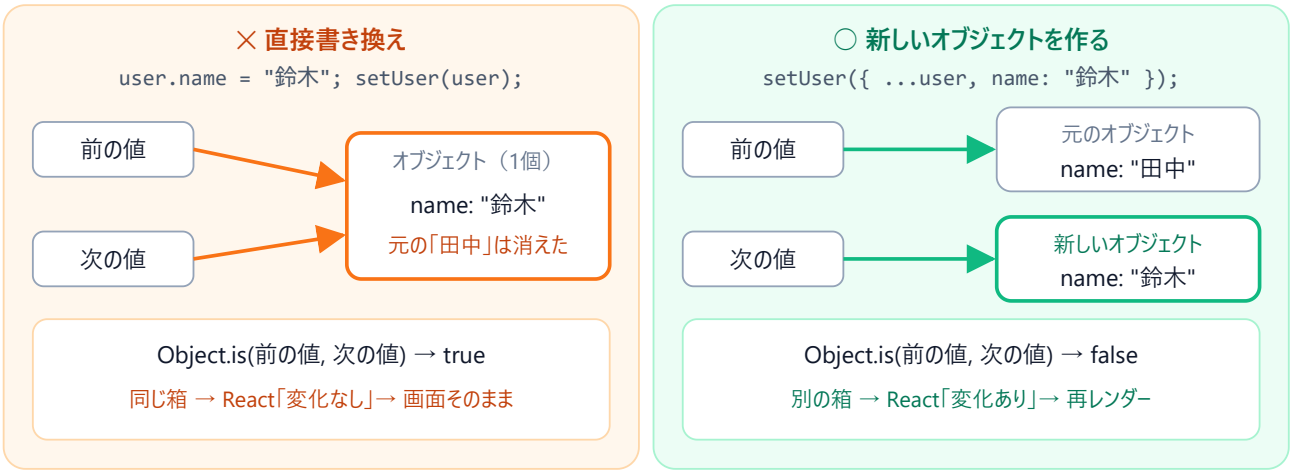


図3-6: React は「中身」ではなく「同じ箱（参照）かどうか」で変化を判断します

「同じ箱かどうか」を Node.js で確かめる

> **▼ このコードがやること（先に日本語で）**：React が state の変化を判断するときに使う `Object.is`（2つの値が「同じもの」かを調べる関数）を使って、①直接書き換えた場合、②スプレッド構文で新しいオブジェクトを作った場合、③入れ子（ネスト）のオブジェクトの場合、で結果がどう変わるかを確かめます。

```
// ファイル名: reference.ts
// 本のデータの型（info の中にもう1段オブジェクトがある）
type Book = {
  title: string;
  info: { pages: number };
};

// ---- NG: 直接書き換える ----
const before1: Book = { title: "React入門", info: { pages: 200 } };
//  「新しい変数」に入れても、指している箱は同じ
const after1 = before1;
after1.title = "React入門 第2版";
// React は Object.is(前の値, 次の値) で「変わったか」を判定する
console.log("NG: 同じ箱か →", Object.is(before1, after1));
console.log("NG: 元の title →", before1.title);

// ---- OK: スプレッド構文で新しい箱を作る ----
const before2: Book = { title: "React入門", info: { pages: 200 } };
const after2 = { ...before2, title: "React入門 第2版" };
console.log("OK: 同じ箱か →", Object.is(before2, after2));
console.log("OK: 元の title →", before2.title);

// ---- 注意: スプレッド構文は「1段目」しかコピーしない ----
const after3 = { ...before2 };
console.log("info は同じ箱か →", Object.is(before2.info, after3.info));

// ネストした値を変えるときは、その段もスプレッドで作り直す
const after4 = { ...before2, info: { ...before2.info, pages: 250 } };
console.log("作り直した info は同じ箱か →", Object.is(before2.info, after4.info));
console.log("元の pages →", before2.info.pages, "/ 新しい pages →", after4.info.pages);
```

▼ 実行結果（`node reference.ts`）

```
NG: 同じ箱か → true
NG: 元の title → React入門 第2版
OK: 同じ箱か → false
OK: 元の title → React入門
info は同じ箱か → true
作り直した info は同じ箱か → false
元の pages → 200 / 新しい pages → 250
```

▼ コードを1つずつ分解して解説

解説1: 別の変数に入れても「同じ箱」

```
const after1 = before1;
after1.title = "React入門 第2版";
```

- `const after1 = before1;` はオブジェクトをコピーしているように見えますが、実際は**同じ箱を指す名札をもう1枚付けただけ**です。
- そのため `after1.title` を変えると、`before1.title` も変わってしまいます（実行結果の2行目）。
- `Object.is(before1, after1)` が `true`（同じもの）なので、これを `setXxx` に渡しても React は「変化なし」と判断し、画面を更新しません。

> **用語: 参照（reference）** … オブジェクトや配列が「どの箱か」を指し示す情報。変数に入っているのは箱そのものではなく、この参照（箱の場所）。

解説2: スプレッド構文なら「新しい箱」

```
const after2 = { ...before2, title: "React入門 第2版" };
```

- `{ ...before2 }` で中身を新しい箱にコピーし、`title` だけ上書きしています。
- `Object.is` が `false`（別もの）になるので、React は「変化あり」と判断して再レンダーします。元の `before2` も壊れていません。

解説3: スプレッドは1段目だけ

```
const after3 = { ...before2 };
console.log("info は同じ箱か →", Object.is(before2.info, after3.info));
```

- スプレッド構文がコピーするのは**1段目だけ**です。`info` のような入れ子のオブジェクトは、コピー後も元と**同じ箱**を指しています（これを「浅いコピー」と呼びます）。
- 入れ子の値を変えたいときは、`info: { ...before2.info, pages: 250 }` のように、**変えたい段まで全部スプレッドで作り直します**。このあと見る「ネストしたオブジェクトの更新」は、まさにこの書き方です。

> **用語: 浅いコピー（shallow copy）** … 1段目だけをコピーし、入れ子の中身は元と共有したままのコピー。スプレッド構文 `...` は浅いコピー。

ネストしたオブジェクトの更新

```
// 住所の型（小さな部品）
type Address = {
  // 都道府県
  prefecture: string;
  // 市区町村
  city: string;
  // 番地
  street: string;
};

// ユーザー型（Address を中に含む = ネストした構造）
type UserWithAddress = {
  name: string;
  address: Address;
};

function AddressEditor() {
  // ネストしたオブジェクト state
  const [user, setUser] = useState<UserWithAddress>({
    name: "田中太郎",
    address: {
      prefecture: "東京都",
      city: "渋谷区",
      street: "道玄坂1-1-1",
    },
  });

  const handleCityChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    // ネストしたオブジェクトもスプレッド構文で展開する
    // 外側の {...user} で「user の全プロパティ」を展開してコピー、
    // address プロパティだけは別の新しいオブジェクトに置き換える。
    // その内側でも {...user.address} で展開し、city だけ上書きする。
    // → user も user.address も新しい参照になるので React がきちんと変化を検知する。
    setUser({
      ...user,
      address: {
        ...user.address,
        city: e.target.value,
      },
    });
  };

  return (
    <div>
      <p>
        {/* {} で JS式を埋め込み、文字列を連結 */}
        住所: {user.address.prefecture}
```

```
        {user.address.city}
        {user.address.street}
      </p>
      <label>市区町村: </label>
      <input value={user.address.city} onChange={handleCityChange} />
    </div>
  );
}
```

この結果、画面には「**住所: 東京都渋谷区道玄坂1-1-1**」と表示され、入力欄で市区町村を変更すると住所表示がリアルタイムに更新されます。例えば「新宿区」と入力すると、「**住所: 東京都新宿区道玄坂1-1-1**」に変わります。

つまずきポイント（1段目だけコピーして、中を書き換える）：「スプレッドでコピーしたから大丈夫」と思って、コピーの**中の段**を直接書き換えてしまう失敗です。外側の箱は新しいので画面は一見ちゃんと更新されますが、`address` は元と同じ箱なので、**前回の state まで書き換わって**しまいます。「前の値と比べる処理」や `React.memo`（6.8）が正しく動かなくなる、見つけにくいバグの原因です。

```
// ❌ NG: 外側だけコピーして、中の address を直接書き換える
const handleCityChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const nextUser = { ...user };
  // nextUser.address は user.address と同じ箱なので、元の state も変わってしまう
  nextUser.address.city = e.target.value;
  setUser(nextUser);
};
```

```
// ✅ OK: 変えたい段（address）もスプレッドで新しく作る
const handleCityChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  setUser({
    ...user,
    address: { ...user.address, city: e.target.value },
  });
};
```

4.5 配列の state

配列の state も、直接変更せず、常に**新しい配列を作成**して更新します。

▼ **このコードがやること（先に日本語で）**：「TODOリスト（やることの一覧）」を題材に、**配列の state を追加・削除・変更する**基本パターンを学びます。ここでもオブジェクトのときと同じ鉄則――**元の配列を直接いじらない**――が効いてきます。具体的には、追加は**スプレッド構文 `[...todos, 新しい項目]`**（今の全部＋末尾に1つ）、削除は **`filter`**（消したいもの以外を残す）、変更は **`map`**（対象だけ作り替える）を使い、いずれも「**新しい配列を作って差し替える**」形にします。第2章で学んだ `map` ／ `filter` が、ここで実戦投入されます。

```tsx
import { useState } from "react";

// TODO項目1件の型
type Todo = {
  // 一意なID（key に使う）
  id: number;
  // タスク本文
  text: string;
  // 完了済みか
  completed: boolean;
};

function TodoApp() {
  // 配列 state。初期値は2件のサンプル。
  const [todos, setTodos] = useState<Todo[]>([
    { id: 1, text: "React を学ぶ", completed: false },
    { id: 2, text: "TypeScript を学ぶ", completed: true },
  ]);
  // 入力欄の現在値（制御コンポーネント用）
  const [inputValue, setInputValue] = useState<string>("");
  // 次に使う一意なID。新規追加するたびに1増やす。
  const [nextId, setNextId] = useState<number>(3);

  // ── 追加 ──
  const handleAdd = () => {
    // .trim() は前後の空白を取り除いた新しい文字列を返す。
    // 空文字なら何もせず即 return（早期リターン）。
    if (inputValue.trim() === "") return;

    // 新しい Todo オブジェクトを作る
    const newTodo: Todo = {
      id: nextId,
      text: inputValue,
      completed: false,
    };
    // [...todos, newTodo] で「既存の全要素＋末尾に新項目」の新しい配列を作って渡す。
    // 元の todos 配列は変更しない（イミュータブル＝不変な更新）。
    // 既存の配列を展開して新しい要素を追加
    setTodos([...todos, newTodo]);
    // 入力欄をクリア
    setInputValue("");
    // 次回ID用に+1
    setNextId(nextId + 1);
  };

  // ── 削除 ──
  const handleDelete = (id: number) => {
    // filter は「条件が true の要素だけ残した新しい配列」を返す。
    // ここでは「ID が一致しないもの = 削除対象でないもの」だけを残す。
```

```
  // id が一致しない要素だけ残す
  setTodos(todos.filter((todo) => todo.id !== id));
};

// ── 完了/未完了の切り替え ──
const handleToggle = (id: number) => {
  setTodos(
    // map は「各要素を変換して新しい配列を作る」メソッド。
    // 一致する Todo だけ「completed を反転した新しいオブジェクト」に置き換える。
    // 一致しない Todo はそのまま返す。
    todos.map((todo) =>
      todo.id === id ? { ...todo, completed: !todo.completed } : todo
    )
  );
};

// ── テキスト更新 ──
const handleUpdate = (id: number, newText: string) => {
  setTodos(
    // 一致する Todo の text プロパティだけを上書き
    todos.map((todo) =>
      todo.id === id ? { ...todo, text: newText } : todo
    )
  );
};

return (
  <div>
    <h2>TODOリスト</h2>

    {/* 入力欄 */}
    <div>
      <input
        // 表示値を state に紐づけ
        value={inputValue}
        // 入力ごとに state を更新
        onChange={(e) => setInputValue(e.target.value)}
        // 未入力時の薄いガイド文字
        placeholder="新しいタスクを入力"
      />
      <button onClick={handleAdd}>追加</button>
    </div>

    {/* TODOリスト */}
    <ul>
      {todos.map((todo) => (
        // key には DBの主キーに当たる一意なIDを使う
        <li key={todo.id}>
          <input
            // チェックボックス入力
```

```
              type="checkbox"
              // 制御コンポーネント
              checked={todo.completed}
              // クリックで切替
              onChange={() => handleToggle(todo.id)}
            />
            <span
              style={{
                // 完了済みなら取り消し線、未完了ならなし
                textDecoration: todo.completed ? "line-through" : "none",
                // 完了済みは薄い灰色、未完了は黒
                color: todo.completed ? "#999" : "#000",
              }}
            >
              {todo.text}
            </span>
            {/* アロー関数で包んで id を引数に渡す */}
            <button onClick={() => handleDelete(todo.id)}>削除</button>
          </li>
        ))}
      </ul>

      {/* 統計
          todos.length は配列の要素数。
          filter(...).length で「条件を満たす要素の数」を計算できる。 */}
      <p>
        合計: {todos.length}件 / 完了: {todos.filter((t) => t.completed).length}件
        / 未完了: {todos.filter((t) => !t.completed).length}件
      </p>
    </div>
  );
}
```

この結果、画面には以下のように表示されます:

TODOリスト

【入力欄】【追加ボタン】

- [ ] React を学ぶ【削除】
- [x] ~~TypeScript を学ぶ~~【削除】

合計: 2件 / 完了: 1件 / 未完了: 1件

「テストを書く」と入力して【追加】を押すと:

- [ ] React を学ぶ【削除】
- [x] ~~TypeScript を学ぶ~~【削除】
- [ ] テストを書く【削除】

合計: 3件 / 完了: 1件 / 未完了: 2件

「React を学ぶ」のチェックボックスをクリックすると:

- [x] ~~React を学ぶ~~【削除】

合計: 3件 / 完了: 2件 / 未完了: 1件

▼ コードを1つずつ分解して解説

解説1: 配列stateと補助のstateを用意する

```
const [todos, setTodos] = useState<Todo[]>([
  { id: 1, text: "React を学ぶ", completed: false },
  { id: 2, text: "TypeScript を学ぶ", completed: true },
]);
const [inputValue, setInputValue] = useState<string>("");
const [nextId, setNextId] = useState<number>(3);
```

- `todos` は TODO の**配列**state です。`<Todo[]>` で「Todo型の配列」と指定しています。
- `inputValue` は入力欄の現在値（制御コンポーネント用）、`nextId` は新規追加するときの一意なID用です。
- このように、1つの機能でも複数のstateを組み合わせて使うことがよくあります。

> **用語: 配列state** ... 複数の項目をリストとして持つstate。追加・削除・変更を非破壊的に行う。

解説2: 追加はスプレッド構文で「末尾に足した新しい配列」を作る

```
    if (inputValue.trim() === "") return;
    const newTodo: Todo = { id: nextId, text: inputValue, completed: false };
    setTodos([...todos, newTodo]);
    setInputValue("");
    setNextId(nextId + 1);
```

- `inputValue.trim() === ""` で空入力なら**早期return**して何もしません。`.trim()` は前後の空白を除く処理です。
- `[...todos, newTodo]` は「**今の全要素＋末尾に新項目**」の新しい配列です。元の `todos` は変更しません。
- 追加後は入力欄を空に戻し（`setInputValue("")`）、次回のIDを1増やします。

> **用語: イミュータブル更新** … 元の配列・オブジェクトを変更せず、新しいものを作って差し替える更新方法。

解説3: 削除はfilter、変更はmapで新しい配列を作る

```
const handleDelete = (id: number) => {
  setTodos(todos.filter((todo) => todo.id !== id));
};
const handleToggle = (id: number) => {
  setTodos(
    todos.map((todo) =>
      todo.id === id ? { ...todo, completed: !todo.completed } : todo
    )
  );
};
```

- **削除**は `filter` で「IDが一致しないもの（＝消す対象でないもの）だけ残す」新しい配列を作ります。
- **変更**は `map` で「IDが一致するものだけ作り替え、他はそのまま」の新しい配列を作ります。`{ ...todo, completed: !todo.completed }` で完了状態を反転しています。
- どちらも元の配列を直接いじらず、新しい配列を `setTodos` に渡すのがポイントです。

> **用語: filter / map** … filterは「条件で絞る」、mapは「各要素を作り替える」。配列stateの更新で多用する。

解説4: 配列を map でリスト表示し、統計を計算する

```
        {todos.map((todo) => (
          <li key={todo.id}>
            <input type="checkbox" checked={todo.completed} onChange={() =>
handleToggle(todo.id)} />
            ...
            <button onClick={() => handleDelete(todo.id)}>削除</button>
          </li>
        ))}
      ...
        合計: {todos.length}件 / 完了: {todos.filter((t) => t.completed).length}件
```

- `todos.map(...)` で各TODOを `<li>` に変換します。`key={todo.id}` には一意なIDを使います。
- チェックボックスや削除ボタンは `() => handleToggle(todo.id)` のようにアロー関数で包み、どのTODOかをIDで渡します。
- `todos.length` で件数、`todos.filter(...).length` で「条件を満たす件数」を計算して統計を表示しています。

> **用語: length** … 配列の要素数。`filter(...).length` で「条件に合う数」を数えられる。

配列操作の早見表

操作	使わない（直接変更）	使う（新しい配列を作成）
追加	`push`, `unshift`	`[...arr, newItem]`, `[newItem, ...arr]`
削除	`splice`	`filter`
置換	`arr[i] = value`	`map`
並び替え	`sort`, `reverse`	`[...arr].sort()`, `[...arr].reverse()`

---

5. イベントハンドリング

5.1 onClick

▼ **このコードがやること（先に日本語で）**：ボタンを押したときに処理を動かす `onClick` の使い方を、いくつかのパターンで示します。一番大事なのは「**関数を"渡す"のであって、"実行"してはいけない**」という点です。`onClick={handleClick}`（関数そのものを渡す＝押したとき呼ばれる）は正しく、`onClick={handleClick()}`（その場で実行してしまう）は誤り。引数を渡したいときは `onClick={() => handleClick(値)}` のように**アロー関数で包んで「押されたら実行する」形にする**——この違いがこの節のヤマです（コード内とこの後で詳しく解説します）。

```tsx
function ClickExamples() {
  // 基本的なクリックハンドラ
  // イベントハンドラ（Event Handler）= イベント発生時に呼ばれる関数。
  const handleClick = () => {
    alert("ボタンがクリックされました！");
  };

  // 引数を受け取るクリックハンドラ
  // 通常のJS関数なので、引数は自由に定義できる。
  const handleItemClick = (itemName: string) => {
    // テンプレートリテラル: ` ` で囲み、${} の中に式を埋め込める文字列。
    alert(`${itemName}がクリックされました`);
  };

  // イベントオブジェクトを受け取るクリックハンドラ
  // React.MouseEvent<HTMLButtonElement> は「<button>に対する Mouseイベント」の型。
  //   e.clientX / e.clientY = クリックされた画面座標
  //   e.currentTarget = ハンドラを付けた要素そのもの
  //   e.target = 実際にクリックされた要素（子要素の場合がある）
  const handleButtonClick = (e: React.MouseEvent<HTMLButtonElement>) => {
    console.log("クリック位置:", e.clientX, e.clientY);
    console.log("クリックされた要素:", e.currentTarget.textContent);
  };

  return (
    <div>
      {/* 基本的な使い方
          onClick={関数名} で「関数の参照」を渡す。React がクリック時に呼んでくれる。 */}
      <button onClick={handleClick}>クリック</button>

      {/* 引数を渡す場合はアロー関数で包む
          () => handleItemClick("Apple") は「引数なしの関数」で、その中で
          handleItemClick("Apple") を呼ぶ。React はこの匿名関数をクリック時に呼ぶ。 */}
      <button onClick={() => handleItemClick("Apple")}>Apple</button>
      <button onClick={() => handleItemClick("Banana")}>Banana</button>

      {/* イベントオブジェクトを使う
          関数参照を直接渡すと、React がクリック時に第1引数として event を渡してくれる。 */}
      <button onClick={handleButtonClick}>位置を表示</button>

      {/* インラインで直接書く
          わざわざ関数を別に定義しなくても、その場でアロー関数を書ける。 */}
      <button onClick={() => console.log("インラインハンドラ")}>
        インライン
      </button>
    </div>
  );
}
```

この結果、画面には4つのボタンが表示されます。

【クリック】を押すと、アラートで「**ボタンがクリックされました！**」と表示されます。

【Apple】を押すと、「**Appleがクリックされました**」と表示されます。

【位置を表示】を押すと、コンソールにクリック座標が出力されます。

▼ コードを1つずつ分解して解説

解説1: クリック時に呼ぶ関数を用意する

```
const handleClick = () => {
  alert("ボタンがクリックされました！");
};
const handleItemClick = (itemName: string) => {
  alert(`${itemName}がクリックされました`);
};
```

- `handleClick` は引数なしのシンプルなクリック処理です。`alert(...)` はブラウザにメッセージを表示する組み込み関数です。
- `handleItemClick` は**引数を受け取る**処理で、押された項目名を受け取って表示します。
- イベントハンドラはただのJavaScript関数なので、引数も自由に決められます。

> **用語: イベントハンドラ** … クリックなどの操作が起きたときに呼ばれる関数。

解説2: イベントオブジェクトを受け取る

```
const handleButtonClick = (e: React.MouseEvent<HTMLButtonElement>) => {
  console.log("クリック位置:", e.clientX, e.clientY);
  console.log("クリックされた要素:", e.currentTarget.textContent);
};
```

- 関数の引数 `e` には、Reactが「何が起きたか」の情報（**イベントオブジェクト**）を渡してくれます。
- `e: React.MouseEvent<HTMLButtonElement>` は「`<button>` に対するマウスイベント」の型です。
- `e.clientX` / `e.clientY` でクリックされた座標、`e.currentTarget` でハンドラを付けた要素そのものを取得できます。

> **用語: イベントオブジェクト** … 発生したイベントの詳細情報（座標・対象要素など）が入った箱。慣例で `e` と書く。

解説3: 関数の「渡し方」を使い分ける

```
      <button onClick={handleClick}>クリック</button>
      <button onClick={() => handleItemClick("Apple")}>Apple</button>
      <button onClick={handleButtonClick}>位置を表示</button>
      <button onClick={() => console.log("インラインハンドラ")}>
        インライン
      </button>
```

- 引数が不要なら `onClick={handleClick}` のように**関数名だけ**を渡します（`()` を付けない）。
- 引数を渡したいときは `onClick={() => handleItemClick("Apple")}` のように**アロー関数で包み**ます。
- イベントオブジェクトを使いたいときは関数名だけ渡せば、Reactが自動で `e` を渡してくれます。
- その場限りの処理なら、インラインでアロー関数を直接書くこともできます。

> **用語: 関数参照を渡す** … `onClick={関数名}` のように、関数を実行せず「関数そのもの」を渡すこと。Reactがクリック時に呼ぶ。

注意: `onClick` に関数を渡すときは、**関数を実行してはいけません。**

```
// NG: 関数を実行してしまっている（レンダリング時に即座に実行される）
// () を付けると「今すぐ呼び出す」意味になり、戻り値（多くの場合 undefined）が onClick に渡る。
// 結果として、画面表示の瞬間に handleClick が動いてしまい、無限ループや誤動作になる。
<button onClick={handleClick()}>クリック</button>

// OK: 関数の参照を渡す
// () を付けないことで「関数そのもの」を渡せる。React がクリック時に呼ぶ。
<button onClick={handleClick}>クリック</button>

// OK: アロー関数で包む
// 「クリック時に handleClick() を呼ぶ」新しい関数を作って渡している。
// 引数を渡したい場合はこの書き方が必要。
<button onClick={() => handleClick()}>クリック</button>
```

`()` が1つ付くかどうかの違いですが、起きることはまったく違います。たとえるなら、`onClick={関数}` は「ボタンが押されたら、この電話番号にかけてください」と**電話番号のメモを渡す**こと。`onClick={関数()}` は、メモを渡す代わりに**今この場で電話をかけてしまう**ことです。電話はすぐにつながってしまい、ボタンには「通話の結果（多くは何もない `undefined`）」しか残りません。

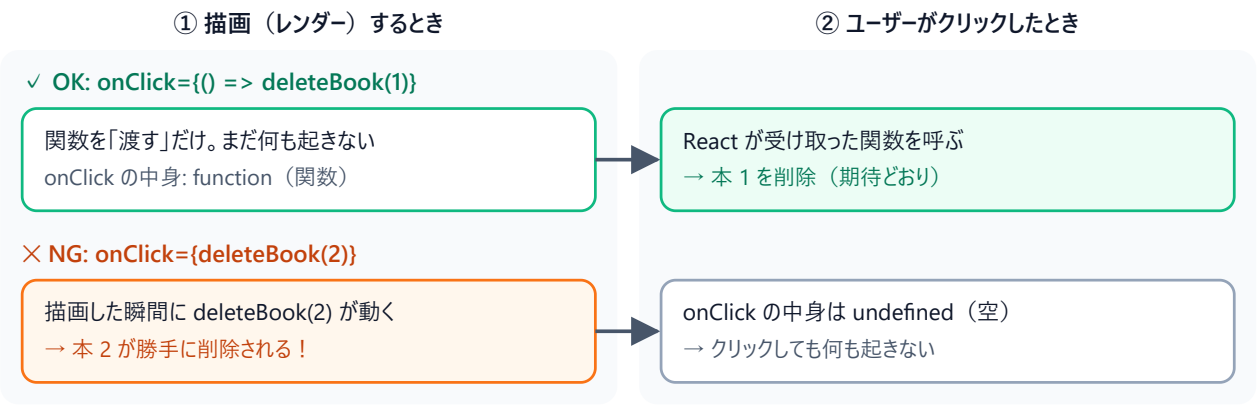


図3-7: 関数を「渡す」か「その場で呼ぶ」か。() を付けると描画のタイミングで実行されてしまいます

「渡す」と「呼ぶ」の違いを Node.js で確かめる

▼ このコードがやること（先に日本語で）： `<button>` の代わりになる小さな模型（描画のときに `onClick` を受け取って覚えておき、クリックされたら呼ぶ）を作り、①アロー関数で包んで渡した場合と、② `deleteBook(2)` と `()` 付きで書いた場合で、「いつ」削除処理が動くかを比べます。

```ts
// ファイル名: onclick.ts
// クリックされたときに呼ぶ関数の型（関数 か、何もなし）
type ClickHandler = (() => void) | undefined;

// <button> の代わりになる小さな模型
// 「描画」のときに onClick を受け取って覚えておき、クリックされたら呼ぶ
function renderButton(label: string, onClick: ClickHandler) {
  console.log(`[描画] ${label} ボタン（onClick の中身: ${typeof onClick}）`);
  return {
    click() {
      console.log(`[クリック] ${label} ボタン`);
      if (onClick) {
        onClick();
      } else {
        console.log("  → onClick が空なので何も起きない");
      }
    },
  };
}

// 本を削除する関数（ここではメッセージを出すだけ）
function deleteBook(id: number): void {
  console.log(`  → 本 ${id} を削除しました`);
}

// OK: アロー関数で包んで「クリックされたら deleteBook(1) を呼ぶ関数」を渡す
const okButton = renderButton("OK", () => deleteBook(1));
// NG: deleteBook(2) を「今ここで」呼び、その戻り値（undefined）を渡してしまう
const ngButton = renderButton("NG", deleteBook(2));

console.log("--- ここでユーザーがクリック ---");
okButton.click();
ngButton.click();
```

▼ 実行結果（`node onclick.ts`）

```
[描画] OK ボタン（onClick の中身: function）
  → 本 2 を削除しました
[描画] NG ボタン（onClick の中身: undefined）
--- ここでユーザーがクリック ---
[クリック] OK ボタン
  → 本 1 を削除しました
[クリック] NG ボタン
  → onClick が空なので何も起きない
```

▼ コードを1つずつ分解して解説

解説1: 描画のときに onClick を受け取る模型

```
function renderButton(label: string, onClick: ClickHandler) {
  console.log(`[描画] ${label} ボタン（onClick の中身: ${typeof onClick}）`);
```

- React の `<button onClick={...}>` も、**描画のときに** `onClick` の値を受け取って覚えておき、クリックされたときにそれを呼びます。模型はこの動きをまねしています。
- `typeof onClick` は値の種類を文字列で返します。関数なら `"function"`、何もなければ `"undefined"` です。

解説2: () を付けると「その場で」実行される

```
const ngButton = renderButton("NG", deleteBook(2));
```

- `deleteBook(2)` と書いた瞬間に関数が実行されます。実行結果を見ると、**まだ誰もクリックしていないのに**「本 2 を削除しました」が `[描画] NG ボタン` より先に出ています。
- `deleteBook` は何も返さない（`void`）ので、`renderButton` に渡るのは `undefined` です。だからクリックしても何も起きません。
- 「削除されてほしくない時に削除され、削除してほしい時に何も起きない」——これが `onClick={f()}` の怖さです。

解説3: TypeScript は事前に教えてくれる

- 実はこの NG の行は、TypeScript の型チェックでエラーになります（Node.js の `node onclick.ts` は型チェックをせずに型注釈を取り除くだけなので、あえて動かして結果を見ています）。React のコードで `onClick={handleClick()}` と書いた場合も、VS Code が次のように赤線で知らせてくれます。

```
Type 'void' is not assignable to type 'MouseEventHandler<HTMLButtonElement> |
undefined'.
```

- 「`void`（何もない）を、クリック時の関数を入れる場所には入れられません」という意味です。このエラーを見たら「`()` を付けていないか？」を疑いましょう。

> **つまずきポイント（onClick={setCount(count + 1)}）**：state を更新する関数を `()` 付きで渡すと、描画のたびに `setCount` が実行され、それがまた描画を起こし…と無限にくり返されます。React は途中で止めて、画面が真っ白になり次のエラーを出します。

```
Too many re-renders. React limits the number of renders to prevent an infinite loop.
```

```
// ❌ NG: 描画のたびに setCount が実行され、無限ループになる
<button onClick={setCount(count + 1)}>+1</button>
```

```
// ✅ OK: 「クリックされたら setCount を呼ぶ関数」をアロー関数で作って渡す
<button onClick={() => setCount(count + 1)}>+1</button>
```

5.2 onChange

> ▼ **このコードがやること（先に日本語で）**：入力欄（テキスト・チェックボックス・ラジオボタン・セレクト）に何か入力・選択されたときに反応する **onChange** の使い方です。基本の流れは「**入力された値を state に保存し、その state を入力欄の表示に反映する**」というワンセット。これにより「**画面の見た目（入力欄）と、プログラムが持つ値（state）が常に一致する**」状態を作れます（これを"制御された入力"と呼びます）。入力の種類ごとに「値の取り出し方」が少し違う点も、コード内で1つずつ確認します。

```tsx
import { useState } from "react";

function InputExamples() {
  // 4つの独立した state を用意
  const [text, setText] = useState<string>("");
  const [selectedColor, setSelectedColor] = useState<string>("red");
  const [isChecked, setIsChecked] = useState<boolean>(false);
  const [selectedSize, setSelectedSize] = useState<string>("M");

  return (
    <div>
      {/* テキスト入力 */}
      <div>
        <label>名前: </label>
        <input
          // 1行のテキスト入力
          type="text"
          // 表示値は state と同期
          value={text}
          onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
            // e.target = 入力欄のDOM要素。.value で現在の文字列を取得
            setText(e.target.value)
          }
          // 未入力時のガイド文字
          placeholder="名前を入力してください"
        />
        <p>入力値: 「{text}」</p>
      </div>

      {/* セレクトボックス（プルダウンメニュー） */}
      <div>
        <label>色: </label>
        <select
          // 現在の選択値
          value={selectedColor}
          onChange={(e: React.ChangeEvent<HTMLSelectElement>) =>
            // <select>専用の型
            setSelectedColor(e.target.value)
          }
        >
          {/* <option> の value 属性が選択時に e.target.value として返る */}
          <option value="red">赤</option>
          <option value="blue">青</option>
          <option value="green">緑</option>
        </select>
        {/* 選択中の色名で文字色を変える */}
        <p style={{ color: selectedColor }}>
          選択した色: {selectedColor}
        </p>
```

```
      </div>

      {/* チェックボックス */}
      <div>
        <label>
          <input
            type="checkbox"
            // チェックボックスでは value ではなく checked プロパティを使う
            checked={isChecked}
            onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
              // e.target.checked は真偽値（true/false）。文字列ではない！
              setIsChecked(e.target.checked)
            }
          />
          利用規約に同意する
        </label>
        <p>同意状態: {isChecked ? "同意済み" : "未同意"}</p>
      </div>

      {/* ラジオボタン
          複数のラジオボタンを「同じグループ」として扱うには、name 属性を揃える。 */}
      <div>
        <p>サイズ:</p>
        {/* ["S","M","L","XL"] の各要素を <label> に展開 */}
        {["S", "M", "L", "XL"].map((size) => (
          <label key={size} style={{ marginRight: "12px" }}>
            <input
              type="radio"
              // 同じ name でグループ化
              name="size"
              value={size}
              // この選択肢が現在の値と同じか
              checked={selectedSize === size}
              onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
                setSelectedSize(e.target.value)
              }
            />
            {size}
          </label>
        ))}
        <p>選択サイズ: {selectedSize}</p>
      </div>
    </div>
  );
}
```

この結果、画面には4つの入力セクションが表示されます:

1. テキスト入力欄に「こんにちは」と入力すると、下に「**入力値: 「こんにちは」**」とリアルタイム表示
2. セレクトボックスで「青」を選ぶと、「**選択した色: blue**」が青色で表示
3. チェックボックスをオンにすると、「**同意状態: 同意済み**」に変化
4. ラジオボタンで「L」を選ぶと、「**選択サイズ: L**」に変化

▼ コードを1つずつ分解して解説

解説1: 入力の種類ごとにstateを用意する

```
const [text, setText] = useState<string>("");
const [selectedColor, setSelectedColor] = useState<string>("red");
const [isChecked, setIsChecked] = useState<boolean>(false);
const [selectedSize, setSelectedSize] = useState<string>("M");
```

- テキスト・セレクト・チェックボックス・ラジオの4つに対し、それぞれ独立した state を用意しています。
- テキストやセレクトは文字列（`string`）、チェックボックスは真偽値（`boolean`）の state です。
- 入力の種類によって「覚えておく値の型」が変わる点に注目してください。

> **用語: onChange** … 入力欄の内容が変化したときに呼ばれるイベント。値をstateに保存するのに使う。

解説2: テキスト入力は value をstateで取り出す

```
      <input
        type="text"
        value={text}
        onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
          setText(e.target.value)
        }
      />
```

- `value={text}` で表示を state に固定し、`onChange` で入力のたびに更新する**制御コンポーネント**です。
- `e.target.value` でその入力欄の現在の文字列を取り出し、`setText` で state を更新します。
- これにより「画面の入力欄」と「state の値」が常に一致します。

> **用語: e.target.value** … 入力欄の現在の値。テキストやセレクトの値の取り出しに使う。

解説3: チェックボックスは value ではなく checked を使う

```
        <input
          type="checkbox"
          checked={isChecked}
          onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
            setIsChecked(e.target.checked)
          }
        />
```

- チェックボックスは文字列ではなく**オン/オフ**なので、`value` ではなく `checked` プロパティを使います。
- 値の取り出しも `e.target.value` ではなく `e.target.checked`（true/false の真偽値）を使います。
- 入力の種類によって「使うプロパティ」が違う、という点が初心者のつまずきどころです。

> **用語: checked** … チェックボックスやラジオボタンの「選択されているか」を表す真偽値プロパティ。

解説4: ラジオボタンは name でグループ化し checked で判定

```
      {["S", "M", "L", "XL"].map((size) => (
        <label key={size} style={{ marginRight: "12px" }}>
          <input
            type="radio"
            name="size"
            value={size}
            checked={selectedSize === size}
            onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
              setSelectedSize(e.target.value)
            }
          />
          {size}
        </label>
      ))}
```

- 選択肢の配列を `map` で複数のラジオボタンに展開しています。
- `name="size"` をすべて同じにすることで「**1つだけ選べるグループ**」になります。
- `checked={selectedSize === size}` で「この選択肢が今の値と同じなら選択状態」にしています。選ぶと `onChange` で state が更新されます。

> **用語: name属性** … 同じ name のラジオボタンは1つのグループになり、同時に1つだけ選択できる。

制御コンポーネントの「1文字ごとのループ」

ここまでの入力欄は、すべて「`value`（表示する値）は state から、変更は `onChange` で state へ」という形でした。これを**制御コンポーネント**と呼びます。1文字入力するたびに、裏では次のループが1周しています。

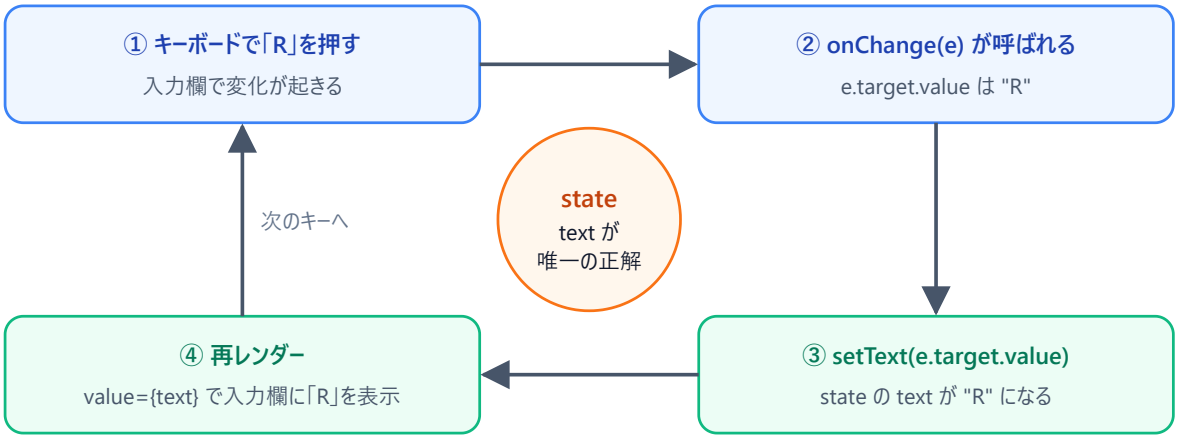


図3-8: 制御コンポーネントの入力ループ。1文字打つたびに ①→②→③→④ が1周します

たとえるなら、入力欄は「自分では何も覚えていない電光掲示板」で、表示する文字はすべて state（司令室）が決めています。キーが押されると掲示板は司令室に「R が押されました」と報告し（②）、司令室が記録を書き換え（③）、その記録どおりに掲示板の表示を更新します（④）。表示と記録がずれることがないので、「入力中の値をそのままプレビューに出す」「文字数を数える」「送信前にチェックする」といった処理が簡単に書けます。

> **つまずきポイント（value だけ書いて onChange を忘れる）**：`value={text}` だけを書いて `onChange` を書き忘れると、キーを押しても文字が入りません。図のループの ③ が起きないので、④ で毎回「元の値（空文字）」に戻されてしまうからです。Console には次の警告が出ます。

```
You provided a `value` prop to a form field without an `onChange` handler. This will
render a read-only field. If the field should be mutable use `defaultValue`. Otherwise,
set either `onChange` or `readOnly`.
```

```jsx
// ❌ NG: value はあるのに onChange が無い → 何を打っても入力欄は空のまま
const [text, setText] = useState("");
return <input value={text} />;
```

```jsx
// ✅ OK: value と onChange をセットで書く
const [text, setText] = useState("");
return <input value={text} onChange={(e) => setText(e.target.value)} />;
```

> 警告文にあるとおり、「最初の値だけ決めて、あとはブラウザ任せでよい」場合は `defaultValue`（非制御コンポーネント）、「表示するだけで入力させない」場合は `readOnly` を使います。本書のフォームは基本的に `value` + `onChange` の制御コンポーネントで書きます。

5.3 onSubmit（フォーム送信）

ここでは「書籍登録フォーム」を作ります。少しコードが長く、初めて見る書き方も多いので、**最初に完成形を載せたあと、難しい部分を1つずつ分解して、初心者向けに徹底的に解説**します。コードを今すぐ全部理解できなくても大丈夫です。「あとで解説があるんだな」と思って、まずは雰囲気を眺めてください。

> **このフォームがやることの全体像（先に日本語で）**：1. ユーザーが「タイトル・著者・価格・カテゴリ・説明」を入力する。2. 入力した内容は `formData` という1つの箱（state）にまとめて保存する。3. 入力するたびに、その内容を `formData` に反映する（ `handleChange` ）。4.「登録する」ボタンを押したら（ `handleSubmit` ）、まず**入力チェック**（ `validate` ）をする。5. 必須項目が空なら、赤字でエラーメッセージを出す。6. すべてOKなら「登録完了！」画面に切り替える。
>
> この「入力 → チェック → 送信 → 完了画面」という流れは、ほぼ全てのアプリのフォームに共通する基本パターンです。

▼ まずは完成形（このあと部品ごとに分解します）

```tsx
import { useState } from "react";

// フォーム入力データの型
// 注意: price は文字列にしている。<input type="number"> でも DOM 上の値は文字列なので、
//       入力中の "" や "1." のような中間状態を表現しやすい。
type BookFormData = {
  title: string;
  author: string;
  price: string;
  category: string;
  description: string;
};

function BookForm() {
  // フォーム全体のデータを1つのオブジェクト state で持つ
  const [formData, setFormData] = useState<BookFormData>({
    title: "",
    author: "",
    price: "",
    // デフォルトカテゴリ
    category: "programming",
    description: "",
  });

  // エラー情報を「フィールド名 → エラーメッセージ」の形で持つ。
  // Partial<X> は X の全プロパティをオプショナル化した型。
  // Record<K, V> は「キーKのオブジェクトに値Vが入る」型。
  // keyof BookFormData は "title" | "author" | ... の文字列リテラルユニオン。
  const [errors, setErrors] = useState<Partial<Record<keyof BookFormData, string>>>
({});
  // 送信完了フラグ
  const [isSubmitted, setIsSubmitted] = useState<boolean>(false);

  // バリデーション関数
  // 戻り値が true なら「全項目OK」。
  const validate = (): boolean => {
    const newErrors: Partial<Record<keyof BookFormData, string>> = {};

    // .trim() で前後空白を取り除いた文字列が "" なら未入力。
    if (formData.title.trim() === "") {
      newErrors.title = "タイトルは必須です";
    }
    if (formData.author.trim() === "") {
      newErrors.author = "著者は必須です";
    }
    // Number("") は NaN、Number("100") は 100。<= 0 で「未入力 or 0以下」を弾く。
    if (formData.price === "" || Number(formData.price) <= 0) {
```

```
      newErrors.price = "価格は0より大きい数値を入力してください";
    }

    // エラーオブジェクトを state にセット
    setErrors(newErrors);
    // Object.keys でプロパティ名の配列を取得。長さ0なら「エラーなし」=true。
    return Object.keys(newErrors).length === 0;
  };

  // 汎用的な入力ハンドラ
  // ChangeEvent の型を3種類のユニオンにして、input/select/textarea を1関数で処理。
  const handleChange = (
    e: React.ChangeEvent<HTMLInputElement | HTMLSelectElement | HTMLTextAreaElement>
  ) => {
    // 分割代入で name と value を取り出す。
    // name は <input name="..."> 属性、value は現在値。
    const { name, value } = e.target;
    setFormData({
      ...formData,
      // [name]: value は「動的なキー」で、name の値そのものをプロパティ名として使う。
      // 例えば name="title" なら { ...formData, title: value } と書くのと同じ。
      [name]: value,
    });

    // 入力時にエラーをクリア
    // as keyof BookFormData は「string を BookFormData のキー型として扱う」型アサーション。
    if (errors[name as keyof BookFormData]) {
      setErrors({
        ...errors,
        [name]: undefined,
      });
    }
  };

  // フォーム送信
  const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
    // <form> のデフォルト動作（ページリロード）を抑止
    // ページ遷移（デフォルト動作）を防ぐ
    e.preventDefault();

    if (validate()) {
      console.log("送信データ:", formData);
      setIsSubmitted(true);
      // 実際のアプリではここで API を呼ぶ
    }
  };

  // 早期 return で「送信完了画面」と「フォーム画面」を切り替える
  if (isSubmitted) {
    return (
```

```
      <div>
        <h2>登録完了！</h2>
        <p>「{formData.title}」を登録しました。</p>
        {/* 完了画面のボタン: フラグを false に戻すとフォーム画面に戻る */}
        <button onClick={() => setIsSubmitted(false)}>もう1冊登録する</button>
      </div>
    );
  }

  return (
    // <form> の onSubmit に handleSubmit を指定。
    // フォーム内の submit ボタンクリック または Enter キーで発火する。
    <form onSubmit={handleSubmit}>
      <h2>書籍登録</h2>

      <div>
        {/* htmlFor は HTML の for 属性のJSX版。クリックで対応する input にフォーカス */}
        <label htmlFor="title">タイトル *</label>
        <input
          id="title"
          // handleChange の中で動的キーとして使う
          name="title"
          type="text"
          value={formData.title}
          onChange={handleChange}
        />
        {/* エラーがある（truthy）ときだけ赤字でメッセージを表示 */}
        {errors.title && <p style={{ color: "red" }}>{errors.title}</p>}
      </div>

      <div>
        <label htmlFor="author">著者 *</label>
        <input
          id="author"
          name="author"
          type="text"
          value={formData.author}
          onChange={handleChange}
        />
        {errors.author && <p style={{ color: "red" }}>{errors.author}</p>}
      </div>

      <div>
        <label htmlFor="price">価格 *</label>
        <input
          id="price"
          name="price"
          // 数値入力（スマホで数字キーボードが出る）
          type="number"
          value={formData.price}
```

```
          onChange={handleChange}
        />
        {errors.price && <p style={{ color: "red" }}>{errors.price}</p>}
      </div>

      <div>
        <label htmlFor="category">カテゴリ</label>
        <select
          id="category"
          name="category"
          value={formData.category}
          onChange={handleChange}
        >
          <option value="programming">プログラミング</option>
          <option value="design">デザイン</option>
          <option value="business">ビジネス</option>
          <option value="other">その他</option>
        </select>
      </div>

      <div>
        <label htmlFor="description">説明</label>
        <textarea
          id="description"
          name="description"
          value={formData.description}
          onChange={handleChange}
          // 表示行数の目安
          rows={4}
        />
      </div>

      {/* type="submit" のボタンを押すと <form> の onSubmit が走る */}
      <button type="submit">登録する</button>
    </form>
  );
}
```

▼ ここからが本題：難しい部分を1つずつ分解して解説

上のコードには、初心者がつまずきやすい書き方がいくつも入っています。順番に、**1つずつ**ていねいに見ていきましょう。

解説1: フォームの入力内容を「1つの箱」にまとめて持つ

```
const [formData, setFormData] = useState<BookFormData>({
  title: "",
  author: "",
  price: "",
  category: "programming",
  description: "",
});
```

- `useState`（ユーズ・ステート）は、この章の 4.2 で学んだ「**変化する値を持つための道具**」です。`const [今の値, 値を変える関数] = useState(初期値)` の形で使います。
- ここでは「今の値」を `formData`、「値を変える関数」を `setFormData` という名前にしています。
- 入力欄は5つ（タイトル・著者・価格・カテゴリ・説明）ありますが、**5個バラバラにstateを作るのではなく、1つのオブジェクト（`{ }` でまとめたもの）にまとめて**持っています。こうすると管理が楽になります。
- 初期値はすべて空文字 `""`（からの文字列）。ただしカテゴリだけは最初から `"programming"` を選んだ状態にしています。
- `<BookFormData>` の部分は「この箱の中身は `BookFormData` という型ですよ」とTypeScriptに教えている部分です（`BookFormData` 型はコードの上のほうで定義しています）。

> **なぜ価格(`price`)も文字列 `""` なの？数字じゃないの？** Webの入力欄（`<input>`）は、たとえ「数値入力欄」でも、**中身は必ず文字列として扱われる**という決まりがあるためです。入力途中の `"1."`（まだ打ちかけ）のような状態も文字列なら素直に表せます。数値が必要になった時だけ、あとで数値に変換します（解説4で出てきます）。

解説2: エラーメッセージを入れる箱（ここが一番むずかしく見える部分）

```
const [errors, setErrors] = useState<Partial<Record<keyof BookFormData, string>>>({});
```

`Partial<Record<keyof BookFormData, string>>` という型が、**初見だと一番こわく見える部分**です。でも分解すれば難しくありません。内側から順に読み解きましょう。

1. **`keyof BookFormData`** … `keyof`（キーオブ）は「その型が持っているキー（項目名）を全部集める」という意味です。`BookFormData` は `title / author / price / category / description` という項目を持つので、`keyof BookFormData` は「**`"title"` または `"author"` または … `"description"`**」という「項目名の一覧」を表します。
2. **`Record<キー, 値>`** … `Record`（レコード）は「**こういうキーに、こういう値が入るオブジェクト**」を表す型です。`Record<keyof BookFormData, string>` は「`title` や `author` などの各項目名をキーにして、値が**文字列（エラーメッセージ）**のオブジェクト」という意味になります。つまり `{ title: "タイトルは必須です", price: "価格を…" }` のような形です。

3. `Partial<...>` … `Partial`（パーシャル＝部分的）は「**全部の項目を"あってもなくてもよい"扱いにする**」型です。エラーは「ある項目だけ」発生することが多い（タイトルだけエラー、など）ので、全項目そろっていなくてOKにしたいのです。
4. 最後に `useState<...>({})` の `{}` は「初期値は**空っぽのオブジェクト**（エラーが1つもない状態）」という意味です。

> **まとめると:** `errors` は「**エラーが出た項目だけ、その項目名→エラーメッセージ という形で入る箱**」です。最初は空 `{}`。たとえばタイトルが未入力なら `{ title: "タイトルは必須です" }` のようになります。むずかしい型に見えますが、やりたいことは「エラーメッセージを項目ごとに覚えておく箱」というだけです。

```
const [isSubmitted, setIsSubmitted] = useState<boolean>(false);
```

- `isSubmitted`（イズ・サブミテッド＝送信済みか）は「**送信が完了したかどうか**」を表す `true` / `false` の値です。最初は `false`（まだ送信していない）。送信が成功したら `true` にして、画面を「完了画面」に切り替えるのに使います（解説6）。

解説3: 入力するたびに内容を保存する `handleChange`

入力欄に文字を打つたびに呼ばれる関数です。ここに**スプレッド構文**と**動的なキー**という、2つの大事な書き方が出てきます。

```
const handleChange = (
  e: React.ChangeEvent<HTMLInputElement | HTMLSelectElement | HTMLTextAreaElement>
) => {
  const { name, value } = e.target;
  setFormData({
    ...formData,
    [name]: value,
  });
  // （エラーを消す処理は後述）
};
```

- `e`（**イベントオブジェクト**） … 入力が変化したとき、Reactが「何が起きたか」の情報を `e` という箱に入れて渡してくれます。`e: React.ChangeEvent<...>` はその `e` の型で、「`<input>` か `<select>` か `<textarea>` の変化イベント」という意味です（`|` は「または」）。**この型指定はおまじない**と思って大丈夫です。1つの関数で3種類の入力欄すべてを処理できるようにしています。
- `const { name, value } = e.target;` … `e.target`（イー・ターゲット）は「**変化が起きた入力欄そのもの**」を指します。そこから `name`（その欄の名前）と `value`（今入力されている値）を取り出しています。`const { name, value } = ...` は第2章で学んだ「**分割代入**」（オブジェクトから必要な項目だけ取り出す書き方）です。

- `name` は、下の `<input name="title" ... />` の `name="title"` の部分から来ます。つまり「どの欄が変化したか」が文字列で分かります。
- `...formData`（**スプレッド構文**） … `...`（ドット3つ）は「**今ある `formData` の中身を、まるごとコピーして展開する**」という意味です。これを「スプレッド構文（spread＝展開）」と呼びます。

> **なぜわざわざ全部コピーするの？** Reactでは、stateを更新するとき「**古いものを直接書き換えず、新しいオブジェクトを作って丸ごと差し替える**」のがルールだからです。 `...formData` で「今の全項目をコピー」してから、変化した1項目だけを上書きすることで、「他の項目はそのまま・変わった欄だけ新しい値」という新しいオブジェクトを作っています。

- `[name]: value`（**動的なキー**） … キー名を `[ ]`（角カッコ）で囲むと、「**変数 `name` の中身を、そのままキー（項目名）として使う**」という意味になります。これを「動的なキー（computed property name）」と呼びます。
- 例えばタイトル欄が変化したなら `name` は `"title"` なので、`[name]: value` は `title: value` と書いたのと同じになります。著者欄なら `author: value` になります。
- **これがこの関数のキモです。**たった1つの `handleChange` で、5つのどの入力欄が変わっても、その欄だけを正しく更新できるのです。「もし `[name]: value` と書かず `title: value` と固定で書いてしまうと、どの欄を触ってもタイトルだけが書き換わってしまう」と考えると、ありがたみが分かります。

つまり `setFormData({ ...formData, [name]: value })` は、日本語にすると「**今の全項目はそのままに、今変化した欄（name）だけを新しい値（value）にした、新しいformDataを作ってセットして**」という意味です。

続いて、同じ `handleChange` の中の後半部分です。

```
if (errors[name as keyof BookFormData]) {
  setErrors({
    ...errors,
    [name]: undefined,
  });
}
```

- これは「**入力し直したら、その欄のエラー表示を消す**」ための処理です。一度エラーが出ても、ユーザーが入力を直したら赤字を消してあげる、という親切機能です。
- `errors[name as keyof BookFormData]` … 「`errors` という箱の中に、今の欄（`name`）のエラーが入っているか？」を確認しています。
- `name as keyof BookFormData` の `as` は「**型アサーション**」と呼ばれ、「TypeScriptさん、この `name`（ただの文字列）を、`BookFormData` の項目名のどれかとして扱ってね」と**お願い（言い換え）する**書き方です。`name` はそのままだと「ただの文字列」扱いで、`errors` のキーとして使うとTypeScriptが警告するため、`as` で「ちゃんと項目名ですよ」と伝えています。
- `if ( ... )` … カッコの中が「**中身がある（エラーが存在する）**」と判断できるときだけ、`{ }` の中を実行します。エラーが無いなら何もしません。

- `[name]: undefined` … その欄のエラーを `undefined`（＝値が無い状態）にして、実質的に消しています。ここでも `...errors` で他のエラーは残しつつ、その欄だけ消しています（スプレッド構文と動的キーの再利用です）。

解説4: 入力チェック（バリデーション） `validate`

「送信してよい内容か」をチェックする関数です。`validate`（バリデート）は「検証する」という意味の英単語です。

```
const validate = (): boolean => {
  const newErrors: Partial<Record<keyof BookFormData, string>> = {};

  if (formData.title.trim() === "") {
    newErrors.title = "タイトルは必須です";
  }
  if (formData.author.trim() === "") {
    newErrors.author = "著者は必須です";
  }
  if (formData.price === "" || Number(formData.price) <= 0) {
    newErrors.price = "価格は0より大きい数値を入力してください";
  }

  setErrors(newErrors);
  return Object.keys(newErrors).length === 0;
};
```

- `(): boolean =>` … この関数は最後に `true` か `false`（＝`boolean` 型）を返します、という宣言です。チェックの結果「OKだったか(true)／ダメだったか(false)」を呼び出し元に伝えるためです。
- `const newErrors = {}` … まず「**今回のエラーを集める、空っぽの箱**」を用意します。これからチェックして、エラーがあればこの箱に入れていきます。
- `formData.title.trim() === ""` … `.trim()`（トリム）は「文字列の**前後の空白を取り除く**」働きをします。`=== ""` は「**取り除いた結果がからっぽか**」の判定です。
- つまり「スペースだけ入力した」場合も「未入力」とみなせます。`=== ""` の `===`（イコール3つ）は第2章で学んだ「**厳密に等しいか**」の比較です（`=` 1つは代入なので別物）。
- 未入力なら `newErrors.title = "タイトルは必須です";` で、箱にエラーメッセージを入れます。
- `Number(formData.price)` … `Number(...)` は「**文字列を数値に変換する**」関数です。`Number("100")` は数値の `100` になります。`Number("")`（空文字）は `NaN`（ナン＝Not a Number、数値ではない、を表す特別な値）になります。
- 条件 `formData.price === "" || Number(formData.price) <= 0` は、`||`（または）で2つをつないで「**価格が空 または 0以下**」のときにエラー、という意味です。マイナスや0、未入力の価格をはじいています。
- `setErrors(newErrors)` … 集めたエラーを `errors`（解説2の箱）にセットします。これで画面の赤字メッセージが更新されます。
- `Object.keys(newErrors).length === 0` … ここが「OKだったか」を返す部分です。

- `Object.keys(オブジェクト)` は「そのオブジェクトの**キー（項目名）を配列にして取り出す**」関数です。`{ title: "...", price: "..." }` なら `["title", "price"]` を返します。
- `.length` はその配列の**個数**。`=== 0` は「**エラーが1個もない**」という意味です。
- つまりこの関数は「エラーが0個なら `true`（全部OK）、1個でもあれば `false`」を返します。

解説5: 送信ボタンを押したときの `handleSubmit`

```
const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault();

  if (validate()) {
    console.log("送信データ:", formData);
    setIsSubmitted(true);
  }
};
```

- **`e.preventDefault();`** … これは**フォームでとても重要なおまじない**です。
- HTMLの `<form>` は、送信ボタンを押すと**もともと「ページ全体を再読み込みして別ページへ移動する」という昔ながらの動作**を持っています。
- `e.preventDefault()`（プリベント・デフォルト＝デフォルト動作を防ぐ）は、その「**勝手なページ再読み込みを止めて**」と指示する命令です。これを書かないと、ボタンを押した瞬間に画面がリロードされ、Reactの処理が動く前にすべてリセットされてしまいます。
- **フォームの `onSubmit` では、ほぼ必ず最初に `e.preventDefault()` を書く**と覚えてください。
- **`if (validate())`** … 解説4の `validate()` を実行し、戻り値が `true`（＝全項目OK）のときだけ `{ }` の中を実行します。エラーがあれば（`false`）何もせず、赤字メッセージが表示されたままになります。
- **`console.log("送信データ:", formData);`** … 開発者向けに、送信内容をコンソール（開発者ツールの画面）に表示しています。実際のアプリでは、ここで**サーバーにデータを送る処理**を書きます（このチュートリアルでは第7章のSupabase連携で実装します）。
- **`setIsSubmitted(true);`** … 「送信済み」の印を `true` にします。これで次の解説6の画面切り替えが起きます。

解説6: 画面の出し分け（早期 return と 条件付き表示）

```
if (isSubmitted) {
  return (
    <div>
      <h2>登録完了！</h2>
      <p>「{formData.title}」を登録しました。</p>
      <button onClick={() => setIsSubmitted(false)}>もう1冊登録する</button>
    </div>
  );
}

return (
  <form onSubmit={handleSubmit}>
    {/* ...フォーム本体... */}
  </form>
);
```

- これは「**送信が済んでいるかどうかで、表示する画面を切り替える**」しくみです。
- `if (isSubmitted) { return (...) }` … もし送信済み（`isSubmitted` が `true`）なら、ここで「登録完了！」画面を `return`（返す）して、関数を**ここで終わらせます**。この「条件を満たしたら途中で `return` して抜ける」書き方を「**早期 return（early return）**」と呼びます。
- 早期 return で抜けなかった場合（まだ送信していない場合）だけ、下の `return (<form ...>)` まで進み、フォーム画面が表示されます。
- `<button onClick={() => setIsSubmitted(false)}>` … 完了画面のボタンを押すと `isSubmitted` を `false` に戻すので、もう一度フォーム画面に戻れます。

最後に、フォーム本体の中にあるエラー表示も見ておきましょう。

```
{errors.title && <p style={{ color: "red" }}>{errors.title}</p>}
```

- これは「`errors.title` **に中身があるときだけ、赤字のエラーメッセージを表示する**」という書き方です。
- `A && B` … `&&`（アンド）を使った `A && B` は「**Aに中身があれば B を表示、Aがからっぽなら何も表示しない**」という、Reactでとてもよく使う「条件付き表示」のテクニックです。
- `errors.title` にエラーメッセージ（文字列）が入っていれば、その右の `<p style={{ color: "red" }}>...</p>`（赤い文字の段落）が表示されます。
- エラーが無ければ（`undefined`）何も表示されません。
- `style={{ color: "red" }}` の `{{ }}`（中カッコ2つ）は、この章の 2.2 で説明した「**JSXの中にスタイルのオブジェクトを書く**」書き方です（外側は「JSの世界に入る」印、内側は「スタイルのオブジェクト」）。

> **ここまでのまとめ:** 難しく見えたコードも、分解すれば「①入力を1つの箱に保存し（スプレッド構文＋動的キー）、②送信時にページ再読み込みを止めて（preventDefault）、③入力チェックし（validate）、④OKなら完了画面に切り替える（早期return）」という流れでした。`Partial<Record<keyof ...>>` のような型も、「エラーを項目ごとに覚える箱」という目的さえ分かれば怖くありません。**一度で全部覚えなくて大丈夫**です。フォームを作るたびにこの節へ戻ってくれば、少しずつ手に馴染んでいきます。

> この結果、画面には書籍登録フォームが表示されます:
>
> **書籍登録**
>
> - タイトル *:【入力欄】
> - 著者 *:【入力欄】
> - 価格 *:【入力欄】
> - カテゴリ:【セレクトボックス（プログラミング）】
> - 説明:【テキストエリア】
>
> 【登録する】ボタン（「*」は必須項目の印）
>
> 何も入力せずに【登録する】を押すと、各必須項目の下に赤字で「**タイトルは必須です**」等のエラーメッセージが表示されます。
>
> すべての必須項目を入力して送信すると、「**登録完了！**」画面に切り替わり、「「**React入門**」を登録しました。」と表示されます。

タイトルを空のまま、著者に「田中」、価格に「-100」と入力して【登録する】を押したときの画面と、そのときの state の中身を並べると次のようになります。



図3-9: 書籍登録フォームの画面イメージ。タイトルを空、価格を -100 にして「登録する」を押したときの表示です

`errors` オブジェクトに入っている項目（`title` と `price`）の下にだけ、赤いメッセージが出ていることが分かります。`{errors.title && <p>...</p>}` は「`errors.title` に文字が入っていれば表示する」という意味でした（解説6）。

入力チェックだけを Node.js で試す

> **▼ このコードがやること（先に日本語で）**：上のフォームの `validate` から「チェックする部分」だけを取り出し、画面を作らずに Node.js で動かします。3種類の入力（全部空・空白だけのタイトルと負の価格・正しい入力）を渡して、どの項目にエラーが付くかを表示します。画面と切り離した「ただの関数」にしておくと、こうして**画面を操作せずに動作を確かめられる**のが利点です。

```ts
// ファイル名: validate.ts
// 5.3 のフォームと同じ形の入力データ
type BookFormData = {
  title: string;
  author: string;
  price: string;
  category: string;
  description: string;
};

// 「エラーが出た項目だけ、項目名 → メッセージ」が入る型
type FormErrors = Partial<Record<keyof BookFormData, string>>;

// 5.3 の validate から「チェックする部分」だけを取り出した関数
// （setErrors を呼ぶ代わりに、見つけたエラーを return する）
function findErrors(formData: BookFormData): FormErrors {
  const newErrors: FormErrors = {};
  if (formData.title.trim() === "") {
    newErrors.title = "タイトルは必須です";
  }
  if (formData.author.trim() === "") {
    newErrors.author = "著者は必須です";
  }
  if (formData.price === "" || Number(formData.price) <= 0) {
    newErrors.price = "価格は0より大きい数値を入力してください";
  }
  return newErrors;
}

// ラベルと入力データを受け取り、チェック結果を表示する
function check(label: string, formData: BookFormData): void {
  const errors = findErrors(formData);
  // エラーの数が 0 なら送信してよい
  const canSubmit = Object.keys(errors).length === 0;
  console.log(`【${label}】 送信できる: ${canSubmit}`);
  console.log(errors);
}

// 何も入力していないフォーム（カテゴリだけ最初から選ばれている）
const emptyForm: BookFormData = {
  title: "",
  author: "",
  price: "",
  category: "programming",
  description: "",
};

check("全部空", emptyForm);
check("空白だけのタイトルと負の価格", {
```

```
  ...emptyForm,
  title: "   ",
  author: "田中",
  price: "-100",
});
check("正しい入力", {
  ...emptyForm,
  title: "React入門",
  author: "田中",
  price: "2800",
});
```

▼ 実行結果（`node validate.ts`）

```
【全部空】 送信できる: false
{
  title: 'タイトルは必須です',
  author: '著者は必須です',
  price: '価格は0より大きい数値を入力してください'
}
【空白だけのタイトルと負の価格】 送信できる: false
{ title: 'タイトルは必須です', price: '価格は0より大きい数値を入力してください' }
【正しい入力】 送信できる: true
{}
```

▼ コードを1つずつ分解して解説

解説1: エラーを集める箱の型

```
type FormErrors = Partial<Record<keyof BookFormData, string>>;
```

- 5.3 の解説2で分解した `Partial<Record<keyof BookFormData, string>>` に、`FormErrors` という短い名前を付けています。長い型は名前を付けると読みやすくなります。
- 実行結果の `{ title: '...', price: '...' }` のように、**エラーがある項目だけ**が入ります。正しい入力では空のオブジェクト `{}` です。

解説2: 空白だけの入力もはじける

```
if (formData.title.trim() === "") {
```

- 2つ目の入力ではタイトルにスペースを3つ入れていますが、`.trim()` で前後の空白を取り除くと空文字になるので、ちゃんとエラーになっています。

- 価格の `"-100"` は `Number("-100")` で `-100` になり、`<= 0` に当てはまるのでエラーです。

解説3: エラーの数で「送信してよいか」を決める

```
const canSubmit = Object.keys(errors).length === 0;
```

- `Object.keys(errors)` はエラーの項目名の配列（例: `["title", "price"]`）を返します。その数（`length`）が 0 なら送信 OK です。
- 5.3 の `validate` も、最後に同じ式を `return` しています。チェックのロジックを関数に切り出しておくと、こうして**画面を操作しなくても動作を確かめられる**ので、バグを早く見つけられます。

> **ヒント:** 画面のコンポーネントの中に処理を全部書くのではなく、「計算やチェックはただの関数に切り出し、コンポーネントはそれを呼ぶだけ」にしておくと、コードが読みやすく、確かめやすくなります。

5.4 イベントオブジェクトの型 一覧

イベント	型
`onClick`（ボタン）	`React.MouseEvent<HTMLButtonElement>`
`onClick`（div）	`React.MouseEvent<HTMLDivElement>`
`onChange`（input）	`React.ChangeEvent<HTMLInputElement>`
`onChange`（select）	`React.ChangeEvent<HTMLSelectElement>`
`onChange`（textarea）	`React.ChangeEvent<HTMLTextAreaElement>`
`onSubmit`（form）	`React.FormEvent<HTMLFormElement>`
`onKeyDown`	`React.KeyboardEvent<HTMLInputElement>`
`onFocus` / `onBlur`	`React.FocusEvent<HTMLInputElement>`

5.5 データは下へ、イベントは上へ（state のリフトアップ）

3.2 で「props は親から子への一方通行」と学びました。では、**子で起きた出来事（入力やクリック）を親に伝えたい**ときはどうするのでしょうか。答えは「**親が関数を props で渡しておき、子はそれを呼ぶ**」です。

会社にたとえると、親は「上司」、子は「部下」です。上司は部下に資料（データ）を配り（props）、「何かあったらこの内線番号に電話して」と内線番号（関数）も渡しておきます。部下は自分で資料を書き換えず、何かあれば内線で上司に報告します。上司が資料を更新して配り直すと、部下の手元の資料も新しくなります。

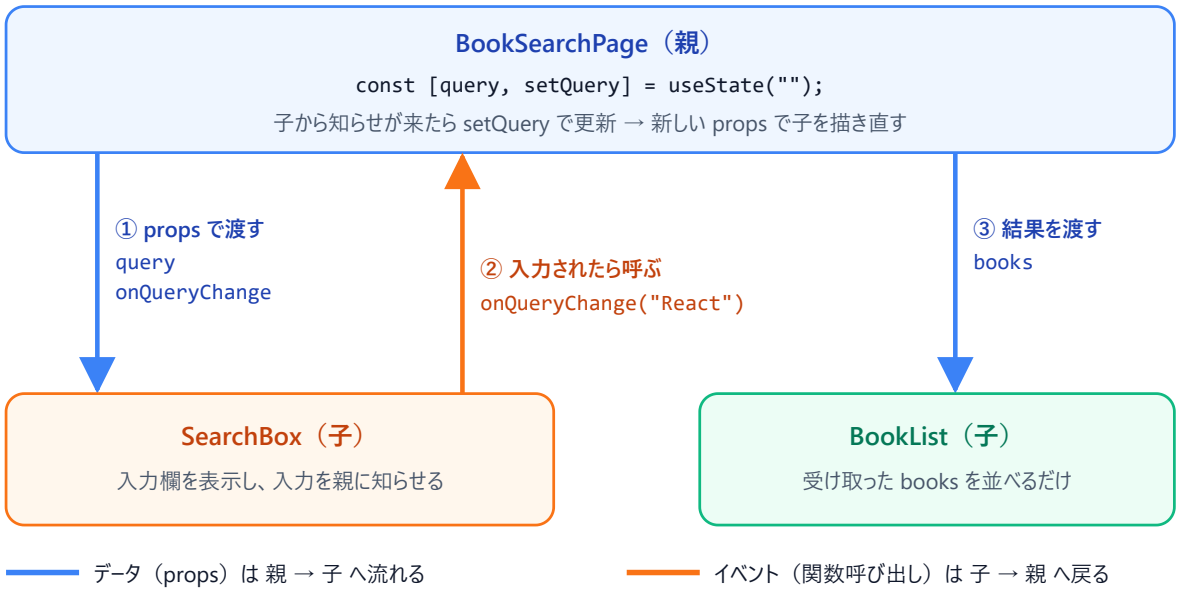


図3-10: データは下へ、イベントは上へ。兄弟の SearchBox と BookList は、共通の親を通してだけ情報をやり取りします

さらに、**兄弟のコンポーネント同士で同じ値を使いたい**とき（検索欄に打った文字で、別の部品の一覧を絞り込む、など）は、その値を**共通の親に持たせます**。これを「**state のリフトアップ**（Lifting State Up：状態を持ち上げる）」と呼びます。兄弟同士は直接やり取りできないので、親が「情報のハブ（中継地点）」になるのです。

検索欄と一覧で検索語を共有する

> **▼ このコードがやること（先に日本語で）**：検索欄（ `SearchBox` ）に文字を打つと、その文字をタイトルに含む本だけが一覧（ `BookList` ）に表示される画面を作ります。検索語 `query` は、2つの子の共通の親 `BookSearchPage` が state として持ちます。親は、検索欄には「今の検索語」と「変わったら呼ぶ関数」を、一覧には「絞り込んだ結果」を、それぞれ props で渡します。

```tsx
import { useState } from "react";

// 本1冊の型
type Book = {
  id: number;
  title: string;
};

// ---- 子1: 検索欄 ----
// 親から「今の検索語」と「検索語が変わったら呼ぶ関数」を受け取る
type SearchBoxProps = {
  query: string;
  onQueryChange: (nextQuery: string) => void;
};

function SearchBox({ query, onQueryChange }: SearchBoxProps) {
  return (
    <input
      value={query}
      // 入力されたら、親から受け取った関数を呼んで「上へ」知らせる
      onChange={(e) => onQueryChange(e.target.value)}
      placeholder="タイトルで検索"
    />
  );
}

// ---- 子2: 検索結果の一覧 ----
// 親から「表示する本の配列」だけを受け取る（自分では state を持たない）
type BookListProps = {
  books: Book[];
};

function BookList({ books }: BookListProps) {
  // 1冊もなければメッセージだけ返す（早期 return）
  if (books.length === 0) {
    return <p>見つかりませんでした</p>;
  }
  return (
    <ul>
      {books.map((book) => (
        <li key={book.id}>{book.title}</li>
      ))}
    </ul>
  );
}

// ---- 親: state を持ち、2つの子に配る ----
const ALL_BOOKS: Book[] = [
  { id: 1, title: "React入門" },
```

```
  { id: 2, title: "TypeScript実践" },
  { id: 3, title: "React Hooks 徹底解説" },
];

export default function BookSearchPage() {
  // 兄弟（SearchBox と BookList）で共有したい値は、共通の親が持つ
  const [query, setQuery] = useState<string>("");

  // state から「表示する本」を計算する（これは state にしない）
  const visibleBooks = ALL_BOOKS.filter((book) => book.title.includes(query));

  return (
    <div>
      <h2>書籍検索</h2>
      {/* 下へ: 今の検索語と、変更を知らせてもらう関数を渡す */}
      <SearchBox query={query} onQueryChange={setQuery} />
      <p>{visibleBooks.length}件ヒット</p>
      {/* 下へ: 計算した結果だけを渡す */}
      <BookList books={visibleBooks} />
    </div>
  );
}
```

▼ **実行結果**（画面の変化。React 19 で実際に入力して確認した結果です）

操作	画面に表示される内容
最初に開いたとき	書籍検索 / 3件ヒット / ・React入門 ・TypeScript実践 ・React Hooks 徹底解説
検索欄に「React」と入力	書籍検索 / 2件ヒット / ・React入門 ・React Hooks 徹底解説
検索欄を「Vue」に書き換え	書籍検索 / 0件ヒット / 見つかりませんでした

▼ コードを1つずつ分解して解説

解説1: 子は「関数を呼ぶ」ことで親に知らせる

```
type SearchBoxProps = {
  query: string;
  onQueryChange: (nextQuery: string) => void;
};
```

- `onQueryChange` は「文字列を1つ受け取る関数」という型の props です。親からは `setQuery` がそのまま渡されます。
- 子の `SearchBox` は、入力されるたびに `onQueryChange(e.target.value)` を呼ぶだけです。**自分では検索語を覚えていません**（`useState` がありません）。

- props の名前を `onXxx`（「Xxx が起きたときに呼ばれる」の意味）にするのが React の習慣です。`onClick` や `onChange` と同じ考え方です。

> **用語: コールバック props** … 親が子に渡す関数の props。子で何かが起きたときに呼ばれ、親に知らせる役目をする。

解説2: 共有したい state は共通の親に置く

```
const [query, setQuery] = useState<string>("");
```

- 検索語は `SearchBox`（入力する側）と `BookList`（絞り込まれる側）の両方に関係します。だから2人の共通の親である `BookSearchPage` に置きます。
- もし `SearchBox` の中に `useState` を書いてしまうと、`BookList` からはその値を見る方法がありません。兄弟同士は直接 props を渡し合えないからです。

> **用語: state のリフトアップ** … 複数の子で共有したい state を、共通の親コンポーネントへ移すこと。

解説3: 計算で求められる値は state にしない

```
const visibleBooks = ALL_BOOKS.filter((book) => book.title.includes(query));
```

- 「表示する本の一覧」は、`query` が決まれば計算で求められます。このような値は **state にせず、レンダーのたびに計算する**のが React の基本です。
- `visibleBooks` まで state にすると、「`query` は変えたのに `visibleBooks` の更新を忘れた」という食い違いが起きやすくなります。state は「それ以上減らせない最小限の値」だけにしましょう。
- `includes` は「その文字列を含むか」を調べるメソッドです。`query` が空文字のときは全部の本が当てはまるので、最初は3件とも表示されます。

> **つまずきポイント（子で props を書き換える）**：子の中で受け取った props に代入しても、親の state は変わりません。props は「親から借りた資料のコピー」のようなもので、書き換えても親の原本には届かないからです。子から変更したいときは、必ず親から受け取った関数を呼びます。

```
// ❌ NG: 受け取った query に代入しても、親の state は変わらない（入力しても文字が増えない）
function SearchBox({ query }: { query: string }) {
  return (
    <input
      value={query}
      onChange={(e) => {
        query = e.target.value;
      }}
    />
  );
}
```

```
// ✅ OK: 親から受け取った関数を呼んで、親に「変えて」と頼む
function SearchBox({ query, onQueryChange }: SearchBoxProps) {
  return <input value={query} onChange={(e) => onQueryChange(e.target.value)} />;
}
```

---

6. useEffect

6.1 副作用とは

React コンポーネントの主な仕事は「UI を描画すること（レンダリング：Rendering。state や props からJSXを作る一連の処理）」です。それ以外の処理を**副作用**（Side Effect：サイドエフェクト。レンダリング以外の、外部世界へ影響を及ぼす処理）と呼びます。「画面を描く」関数の中で副作用を直接実行すると、レンダリングのたびに発生してしまったり、Strict Mode で関数が2回呼ばれるために2回実行されてしまったりするため、`useEffect` で隔離します。

例: - API からデータを取得する - DOM を直接操作する - タイマーを設定する - ログを出力する - ローカルストレージにアクセスする - 外部サービスに接続する

これらの処理は `useEffect` フックの中で行います。

6.2 基本的な使い方

> **▼ このコードがやること（先に日本語で）**：「カウントが変わるたびに、ブラウザのタブに出る文字も一緒に変える」という例で、`useEffect` の基本を学びます。`useEffect` は「**画面が描かれた"あと"に、何か追加の処理（副作用）をする**」ための道具です。書き方は `useEffect(やりたい処理, 依存配列)` の2点セット。**依存配列**（2つ目の `[count]` の部分）は「**この中の値が変わったときだけ、処理をやり直す**」という指定で、ここが `useEffect` の一番のキモです。`[count]` なら「count が変わるたびに実行」、空の `[]` なら「最初の1回だけ実行」になります。

```tsx
// ============================================================================
// useEffect の超基本サンプル（詳細コメント版）
// ----------------------------------------------------------------------------
//  「count が変わるたびに、ブラウザタブのタイトルも追従して変わる」コンポーネント
// ============================================================================

// useState（状態を持つフック）と useEffect（副作用を扱うフック）を取り込む
import { useState, useEffect } from "react";

function PageTitle() {
  // count: 数値の状態。初期値は 0。
  const [count, setCount] = useState<number>(0);

  // ----------------------------------------------------------------------
  // useEffect: 「画面が描かれた *あと* に何かをする」ためのフック
  // ----------------------------------------------------------------------
  // 第1引数: 実行する関数（副作用本体）
  // 第2引数: 依存配列（この配列の中身が変わったときだけ第1引数が実行される）
  //
  // ここでは [count] を指定しているので「count が変わったら関数を実行」となる。
  // 初回マウント時にも1回実行される（このときも count = 0 → 0 という変化扱い）。
  useEffect(() => {
    // document.title はブラウザタブ上部の文字を変える命令（DOM操作 = 副作用）。
    // 副作用を JSX の中で直接書くと不安定なので、useEffect の中に隔離する。
    document.title = `カウント: ${count}`;

    // 開発者ツールの Console タブに出力する。動作確認に便利。
    console.log(`useEffect が実行されました（count = ${count}）`);
  // ← count が変わるたびに上の関数が再実行される
  }, [count]);

  // 画面は「現在のカウント」と「+1ボタン」だけ。
  return (
    <div>
      <p>カウント: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
    </div>
  );
}
```

▼ ブラウザ画面（初期表示）：

```
タブタイトル: 「カウント: 0」          ← document.title が変更された

┌──────────────────────┐
│ カウント: 0          │
│ [ +1 ]               │
└──────────────────────┘
```

▼ Console タブ（開発者ツール）の出力:

```
useEffect が実行されました（count = 0）
```

▼「+1」ボタンを3回押した後:

```
タブタイトル：「カウント：3」
画面：カウント：3
Console:
  useEffect が実行されました（count = 0）  ← 初回マウント時
  useEffect が実行されました（count = 1）  ← 1回目クリック後
  useEffect が実行されました（count = 2）  ← 2回目クリック後
  useEffect が実行されました（count = 3）  ← 3回目クリック後
```

> **依存配列のキモ:** もし `[count]` を `[]`（空配列）にすると、初回しか実行されないため、タブタイトルが「カウント: 0」のまま固まります。逆に第2引数を**完全に省略**すると毎回再描画ごとに実行され、無限ループになりがちです。「何が変わったらこの副作用を再実行したいか？」を意識して書くのが鉄則です。
>
> なお開発モードの **Strict Mode** が有効だと、初回の useEffect は **わざと2回呼ばれます**（マウント → アンマウント → 再マウントをシミュレートして、クリーンアップ忘れを検出するため）。「console.log が2回出る？」と驚かないようにしましょう。本番ビルドでは1回しか呼ばれません。

▼ コードを1つずつ分解して解説

解説1: useStateとuseEffectをimportする

```
import { useState, useEffect } from "react";
```

- `useEffect` を使うには、`useState` と一緒にReactから取り込みます。
- `useEffect` は「**画面が描かれた"あと"に、追加の処理（副作用）をする**」ための道具です。
- 副作用とは「画面を描く以外の処理」のこと。ここでは「ブラウザのタブ名を変える」のがそれにあたります。

> **用語: 副作用（Side Effect）** … 画面描画以外の処理（タブ名変更・API通信・タイマーなど）。useEffectの中で行う。

解説2: useEffectで副作用の処理を書く

```
useEffect(() => {
  document.title = `カウント: ${count}`;
  console.log(`useEffect が実行されました（count = ${count}）`);
}, [count]);
```

- `useEffect(処理, 依存配列)` の2点セットで書きます。第1引数が「やりたい処理」、第2引数が「いつ実行するか」の指定です。
- `document.title = ...` はブラウザのタブ名を変える命令（DOM操作＝副作用）です。これをJSXの中に直接書くと不安定なので、useEffectの中に隔離しています。
- `console.log(...)` で実行されたことを確認できるようにしています。

> **用語: useEffect** … レンダリングのあとに副作用を実行するフック。`useEffect(処理, 依存配列)` の形で使う。

解説3: 依存配列で「いつ実行するか」を決める

```
// ← count が変わるたびに上の関数が再実行される
}, [count]);
```

- 第2引数の `[count]` が**依存配列**で、`useEffect` のいちばんのキモです。
- `[count]` は「**count が変わったときだけ処理をやり直す**」という意味です。初回表示でも1回実行されます。
- もし `[]`（空）にすると「最初の1回だけ」、省略すると「毎回（再描画のたび）」実行されます。

> **用語: 依存配列** … useEffectの第2引数。この中の値が変わったときだけ処理を再実行する。

6.3 依存配列

依存配列の指定方法によって、`useEffect` の実行タイミングが変わります。

> **▼ このコードがやること（先に日本語で）：** `useEffect` の2つ目の引数「**依存配列**」を、書き方ごとに比べます。覚えることは3パターンだけ——①**省略**すると「毎回（再描画のたび）実行」（基本使わない／無限ループの危険）、②**空の `[]`** にすると「**最初の1回だけ**実行」（データ取得の初期化に最適）、③ **`[値]`** にすると「**その値が変わったときだけ**実行」。この「いつ実行されるか」を依存配列でコントロールするのが `useEffect` の使いこなしの中心です。

```tsx
import { useState, useEffect } from "react";

function EffectExamples() {
  const [count, setCount] = useState<number>(0);
  const [name, setName] = useState<string>("");

  // パターン1: 毎回実行（依存配列なし）
  // 第2引数を完全に省略すると、コンポーネントが再描画されるたびに毎回実行される。
  // 注意: 中で state を更新するとすぐ無限ループに陥る。ほぼ使うべきでない。
  useEffect(() => {
    console.log("毎回のレンダリング後に実行");
  // ← 第2引数を省略
  });

  // パターン2: 初回のみ実行（空の依存配列）
  // [] は「依存する値が無い」という意味なので、初回マウント時1回だけ実行される。
  // ※ Strict Mode（開発時のみ）では2回呼ばれるが、本番では1回。
  useEffect(() => {
    console.log("コンポーネントのマウント時に1回だけ実行");
    // API からデータを取得する処理などをここに書く
  // ← 空の配列
  }, []);

  // パターン3: 特定の値が変わったときに実行
  // [count] と書くと「count が前回と異なる場合だけ」関数が再実行される。
  // 初回マウント時にも1回実行される（前回値がない状態を「変化あり」とみなすため）。
  useEffect(() => {
    console.log(`count が変わりました: ${count}`);
  // ← count が変わるたびに実行
  }, [count]);

  // パターン4: 複数の依存値
  // 配列内の値のうち1つでも変わったら実行される。
  useEffect(() => {
    console.log(`count または name が変わりました: ${count}, ${name}`);
  // ← count または name が変わるたびに実行
  }, [count, name]);

  return (
    <div>
      <p>カウント: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <input value={name} onChange={(e) => setName(e.target.value)} />
    </div>
  );
}
```

依存配列の3パターンまとめ:

- **省略する**: 毎レンダリング後に実行（ほぼ使わない）。
- **`[]`（空配列）**: マウント時に1回だけ実行（API初期取得などに最適）。
- **`[a, b]`（値を指定）**: 配列の中身が前回と異なるときだけ実行。

依存配列に含めるべき値を抜かす（**stale closure**：古い変数を参照したまま動くバグ）と、想定通りに動かなくなります。基本的には ESLint の `react-hooks/exhaustive-deps` ルールに従って、effect 内で使う変数は全部入れるのが安全です。

依存配列	実行タイミング	用途
省略	毎回のレンダリング後	ほとんど使わない
`[]`（空配列）	初回マウント時のみ	API 初期データ取得
`[count]`	`count` が変わった時	値の変化に応じた処理
`[count, name]`	いずれかが変わった時	複数の値に応じた処理

▼ コードを1つずつ分解して解説

解説1: 依存配列を省略 → 毎回実行（基本使わない）

```
useEffect(() => {
  console.log("毎回のレンダリング後に実行");
// ← 第2引数を省略
});
```

- 第2引数（依存配列）を**完全に省略**すると、再描画されるたびに毎回実行されます。
- この中で state を更新すると、すぐに無限ループに陥るため、**ほぼ使うべきではありません。**
- 「省略＝毎回」とだけ覚えておけば十分です。

用語: 依存配列の省略 … 第2引数を書かないと毎レンダリング後に実行される。危険なので通常避ける。

解説2: 空配列 [] → 初回マウント時に1回だけ

```
useEffect(() => {
  console.log("コンポーネントのマウント時に1回だけ実行");
// ← 空の配列
}, []);
```

- `[]`（空配列）は「依存する値が無い」という意味なので、**初回マウント時に1回だけ**実行されます。
- APIからの初期データ取得など「最初に1度だけやりたい処理」に最適です。
- ※開発時の Strict Mode では2回呼ばれますが、本番では1回です。

> **用語: マウント** … コンポーネントが初めて画面に表示されること。`[]` の useEffect はこのとき1回実行される。

解説3: [値] → その値が変わったときだけ実行

```
  useEffect(() => {
    console.log(`count が変わりました: ${count}`);
  // ← count が変わるたびに実行
  }, [count]);

  useEffect(() => {
    console.log(`count または name が変わりました`);
  // ← count または name が変わるたびに実行
  }, [count, name]);
```

- `[count]` は「count が前回と異なるときだけ」実行されます。値の変化に応じた処理に使います。
- `[count, name]` のように複数入れると、**どれか1つでも変われば**実行されます。
- effect の中で使う state や props は、基本的に全部この配列に入れるのが安全です（入れ忘れるとバグの原因になります）。

> **用語: stale closure（古い値の参照）** … 依存配列に入れ忘れた値が古いまま使われるバグ。依存値は漏らさず入れる。

> **つまずきポイント（依存配列の書き忘れで値が古いまま）**：「1秒ごとに +1 する」つもりで次の NG のように書くと、画面は「1」になったまま止まります。effect は最初の1回しか作られないので、中の `seconds` は**作られたときのスナップショット（0）のまま**です。毎秒「0 + 1 = 1 にして」と予約し続けるだけなので、1 から増えません（React 19 で実際に動かして確認した結果です）。ESLint は次の警告で教えてくれます。

```
React Hook useEffect has a missing dependency: 'seconds'. Either include it or remove the dependency array. You can also do a functional update 'setSeconds(s => ...)' if you only need 'seconds' in the 'setSeconds' call
```

```
// ❌ NG: effect の中で seconds を使っているのに、依存配列が空
useEffect(() => {
  const id = setInterval(() => {
    // この seconds は、effect を作ったとき（0）の値のまま
    setSeconds(seconds + 1);
  }, 1000);
  return () => clearInterval(id);
}, []);
```

```
// ✅ OK: 関数型更新にすれば seconds を使わないので、依存配列は空のままでよい
useEffect(() => {
  const id = setInterval(() => {
    // prev には常に最新の値が入る
    setSeconds((prev) => prev + 1);
  }, 1000);
  return () => clearInterval(id);
}, []);
```

> 警告文の後半「functional update（関数型更新）を使えば…」が、まさにこの OK の書き方です。もう1つの直し方は依存配列を `[seconds]` にすることですが、その場合は1秒ごとにタイマーを止めて作り直すことになるので、このケースでは関数型更新のほうがすっきりします。

実践例: API からデータを取得

```
import { useState, useEffect } from "react";

// 受け取るユーザーデータの型
type User = {
  id: number;
  name: string;
  email: string;
};

function UserList() {
  // 取得したユーザー配列。初期値は空配列。
  const [users, setUsers] = useState<User[]>([]);
  // 読み込み中フラグ
  const [loading, setLoading] = useState<boolean>(true);
  // エラー情報。エラー無しなら null。string | null のユニオン型を明示。
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    // API からユーザーデータを取得
    // async/await は「非同期処理を同期っぽく書く」JS の構文。
    // useEffect の第1引数の関数自体は async にできないため、内側に async 関数を定義する。
    const fetchUsers = async () => {
      try {
        setLoading(true);
        // fetch はブラウザ組み込みのHTTPクライアント関数。Promise を返す。
        // await でレスポンスが返ってくるまで待つ。
        const response = await fetch("https://jsonplaceholder.typicode.com/users");

        // response.ok は HTTPステータスが200-299のとき true。
        // それ以外（404, 500 等）は手動でエラーを投げる必要がある（fetch は HTTP エラーで
reject しない）。
        if (!response.ok) {
          throw new Error(`HTTP error! status: ${response.status}`);
        }

        // .json() はレスポンスボディを JSON としてパースする非同期メソッド。
        // 型注釈 User[] で「返ってくるのは User の配列」と宣言。
        const data: User[] = await response.json();
        setUsers(data);
      } catch (err) {
        // err が Error インスタンスならその message を、そうでなければ汎用文言を使う。
        // err instanceof Error は TypeScript の型ガード。
        setError(err instanceof Error ? err.message : "不明なエラー");
      } finally {
        // finally ブロックは成功・失敗どちらでも必ず実行される。
        setLoading(false);
      }
```

```
    };

    fetchUsers();
  // 空配列 → 初回マウント時に1回だけ実行
  }, []);

  // 早期 return パターンで状態別の画面を切り替え
  if (loading) {
    return <p>読み込み中...</p>;
  }

  if (error) {
    return <p style={{ color: "red" }}>エラー: {error}</p>;
  }

  // 正常時の表示
  return (
    <div>
      <h2>ユーザー一覧</h2>
      <ul>
        {users.map((user) => (
          // key には DB の id を使う
          <li key={user.id}>
            {user.name} ({user.email})
          </li>
        ))}
      </ul>
    </div>
  );
}
```

この結果、画面にはまず「**読み込み中...**」と表示されます。

API からデータ取得が完了すると、「**ユーザー一覧**」という見出しと、ユーザーのリストが表示されます:

- Leanne Graham (Sincere@april.biz)
- Ervin Howell (Shanna@melissa.tv)
- ... (以下続く)

ネットワークエラーが発生した場合は、赤字で「**エラー: Failed to fetch**」等と表示されます。

▼ コードを1つずつ分解して解説

解説1: データ・読み込み・エラーの3つのstateを持つ

```
const [users, setUsers] = useState<User[]>([]);
const [loading, setLoading] = useState<boolean>(true);
const [error, setError] = useState<string | null>(null);
```

- API通信では「取得したデータ」「読み込み中かどうか」「エラーが起きたか」の**3つの状態**を持つのが定番です。
- `users` は初期値が空配列、`loading` は最初 `true`（読み込み中）、`error` は `string | null`（エラー無しなら null）型です。
- この3つで「読み込み中／成功／失敗」の画面を出し分けます。

> **用語: ローディング状態** … 通信が終わるまでの「読み込み中」を表すstate。ユーザーに待ち時間を伝える。

解説2: useEffectの中で非同期関数を定義して呼ぶ

```
useEffect(() => {
  const fetchUsers = async () => {
    try {
      setLoading(true);
      const response = await fetch("https://jsonplaceholder.typicode.com/users");
      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }
      const data: User[] = await response.json();
      setUsers(data);
```

- `async` / `await` は「**通信のような時間のかかる処理を、順番に待ちながら書く**」ための構文です。
- `useEffect` の第1引数の関数自体は `async` にできないため、**内側に `fetchUsers` という async 関数を定義して呼ぶ**形にしています。
- `await fetch(...)` でデータを取りに行き、`await response.json()` で中身をJavaScriptのデータに変換して `setUsers` に保存します。

> **用語: async / await** … 非同期処理（通信など）を同期的に見える形で書く構文。`await` は結果が返るまで待つ。

> **つまずきポイント（useEffect に async 関数を直接渡す）**：「内側に関数を作るのは面倒だから」と `useEffect(async () => {...})` と書くのは NG です。`async` 関数は必ず Promise（あとで結果が届く約束の箱）を返しますが、useEffect が「戻り値」として受け取れるのは**クリーンアップ関数だけ**だからです。TypeScript は次のエラーで止めてくれます。

```
Argument of type '() => Promise<void>' is not assignable to parameter of type
'EffectCallback'.
  Type 'Promise<void>' is not assignable to type 'void | Destructor'.
```

```tsx
// ❌ NG: useEffect に async 関数を直接渡している
useEffect(async () => {
  const response = await fetch("/api/books");
  setBooks(await response.json());
}, []);
```

```tsx
// ✅ OK: effect の中で async 関数を作り、すぐに呼ぶ
useEffect(() => {
  const fetchBooks = async () => {
    const response = await fetch("/api/books");
    setBooks(await response.json());
  };
  fetchBooks();
}, []);
```

解説3: try/catch/finallyで成功・失敗・後処理を分ける

```tsx
      } catch (err) {
        setError(err instanceof Error ? err.message : "不明なエラー");
      } finally {
        setLoading(false);
      }
    };
    fetchUsers();
  // 空配列 → 初回マウント時に1回だけ実行
  }, []);
```

- `try { }` の中で通信を試み、失敗したら `catch (err) { }` に飛んでエラーメッセージを state に保存します。
- `finally { }` は**成功・失敗どちらでも必ず**実行されるので、ここで `setLoading(false)`（読み込み終了）にします。
- 依存配列は `[]` なので、この通信は**初回に1回だけ**実行されます。

> **用語: try / catch / finally** … 失敗するかもしれない処理を安全に扱う構文。catchはエラー時、finallyは必ず実行。

解説4: 状態別に早期returnで画面を出し分ける

```
if (loading) {
  return <p>読み込み中...</p>;
}
if (error) {
  return <p style={{ color: "red" }}>エラー: {error}</p>;
}
return (
  <div>
    <h2>ユーザー一覧</h2>
    <ul>{users.map((user) => (...))}</ul>
  </div>
);
```

- まず `loading` が true なら「読み込み中...」を返して**ここで終了**（早期return）します。
- 次に `error` があればエラー表示を返します。
- どちらでもない（＝成功）ときだけ、最後の `return` まで進んでユーザー一覧を表示します。

> **用語: 早期return** … 条件を満たしたら途中でreturnして関数を抜けること。状態別の画面切り替えに便利。

6.4 クリーンアップ

`useEffect` の中で `return` した関数は、**クリーンアップ関数**として実行されます。コンポーネントがアンマウント（画面から消える）されるとき、または次の effect が実行される前に呼ばれます。

```tsx
import { useState, useEffect } from "react";

function Timer() {
  // 経過秒数
  const [seconds, setSeconds] = useState<number>(0);
  // 動作中フラグ（true=動作中, false=停止）
  const [isRunning, setIsRunning] = useState<boolean>(false);

  useEffect(() => {
    // 早期 return: 動作中でなければ何もしない（後続のクリーンアップも登録されない）
    // タイマーが停止中なら何もしない
    if (!isRunning) return;

    // 1秒ごとにカウントアップ
    // setInterval(関数, ミリ秒) はブラウザ組み込み関数。
    // 指定ミリ秒ごとに関数を実行し続け、識別ID（intervalId）を返す。
    const intervalId = setInterval(() => {
      // 関数型更新を使うと、依存配列に seconds を入れなくて済む。
      setSeconds((prev) => prev + 1);
    }, 1000);

    // クリーンアップ: タイマーを解除する
    // useEffect の中で return した関数は「次の effect 実行直前」または「アンマウント時」に呼ばれる。
    // タイマーを止めないと、コンポーネントが消えても動き続けてメモリリークになる。
    return () => {
      // タイマー停止
      clearInterval(intervalId);
      console.log("タイマーをクリーンアップしました");
    };
  // isRunning が変わるたびに再設定
  }, [isRunning]);

  // リセット処理: 停止＋秒数0
  const handleReset = () => {
    setIsRunning(false);
    setSeconds(0);
  };

  return (
    <div>
      <h2>タイマー</h2>
      <p style={{ fontSize: "48px", fontFamily: "monospace" }}>
        {/* 分: 秒数を60で割って小数点以下を切り捨て、2桁0埋め */}
        {Math.floor(seconds / 60)
          // 数値 → 文字列に変換
          .toString()
          // 2桁にして足りなければ "0" を前に追加（1 → "01"）
          .padStart(2, "0")}
```

```
        {/* 区切り文字 ":" 続いて 秒（60で割った余り、2桁0埋め） */}
        :{(seconds % 60).toString().padStart(2, "0")}
      </p>
      {/* 動作中なら開始ボタンを無効化 */}
      <button onClick={() => setIsRunning(true)} disabled={isRunning}>
        開始
      </button>
      {/* 停止中なら停止ボタンを無効化 */}
      <button onClick={() => setIsRunning(false)} disabled={!isRunning}>
        停止
      </button>
      <button onClick={handleReset}>リセット</button>
    </div>
  );
}
```

この結果、画面には大きなフォントで「00:00」と表示され、3つのボタンがあります。

【開始】を押すと、1秒ごとにカウントアップ:「00:01」→「00:02」→ ...

【停止】を押すと、カウントが止まります。

【リセット】を押すと、「00:00」に戻り、停止します。

▼ コードを1つずつ分解して解説

解説1: 経過秒数と動作中フラグのstateを持つ

```
  const [seconds, setSeconds] = useState<number>(0);
  const [isRunning, setIsRunning] = useState<boolean>(false);
```

- `seconds` は経過秒数（数値）、`isRunning` は「タイマーが動いているか」を表す真偽値です。
- 「開始」で `isRunning` を `true`、「停止」で `false` にして、タイマーのオン/オフを切り替えます。
- この `isRunning` が次の useEffect の依存配列のカギになります。

> **用語: フラグ** ... `true` / `false` で状態を表す値。ここでは「動作中かどうか」を表す。

解説2: setIntervalで1秒ごとにカウントアップする

```
  useEffect(() => {
    if (!isRunning) return;
    const intervalId = setInterval(() => {
      setSeconds((prev) => prev + 1);
    }, 1000);
```

- `if (!isRunning) return;` で、停止中なら何もせずに抜けます（早期return）。
- `setInterval(関数, 1000)` は「**1000ミリ秒（1秒）ごとに関数をくり返し実行**」するブラウザの組み込み関数で、識別用のID（`intervalId`）を返します。
- `setSeconds((prev) => prev + 1)` は**関数型更新**で、「直前の値に+1」します。これにより依存配列に `seconds` を入れずに済みます。

> **用語: setInterval** … 一定時間ごとに処理をくり返す関数。止めるには clearInterval にIDを渡す。

解説3: クリーンアップ関数でタイマーを止める

```
    return () => {
      clearInterval(intervalId);
      console.log("タイマーをクリーンアップしました");
    };
  }, [isRunning]);
```

- `useEffect` の中で `return` **した関数**が**クリーンアップ関数**です。「次のeffect実行の直前」や「コンポーネントが消えるとき」に呼ばれます。
- `clearInterval(intervalId)` でタイマーを止めます。これを忘れると、画面が消えてもタイマーが動き続け、**メモリリーク**になります。
- 依存配列が `[isRunning]` なので、開始/停止のたびに「古いタイマーを止めて新しく設定し直す」動きになります。

> **用語: クリーンアップ関数** … useEffectが返す後片付け用の関数。タイマー解除やリスナー解除に使う。

ウィンドウサイズの監視（クリーンアップの実用例）

```tsx
import { useState, useEffect } from "react";

// ウィンドウサイズの型
type WindowSize = {
  width: number;
  height: number;
};

function WindowSizeDisplay() {
  // window.innerWidth / innerHeight はブラウザの「ビューポート」サイズ。
  // useState の初期値として使うことで、初回描画から正しい値を表示できる。
  const [windowSize, setWindowSize] = useState<WindowSize>({
    width: window.innerWidth,
    height: window.innerHeight,
  });

  useEffect(() => {
    // リサイズイベントのハンドラ
    const handleResize = () => {
      setWindowSize({
        width: window.innerWidth,
        height: window.innerHeight,
      });
    };

    // イベントリスナーを登録
    // window.addEventListener("resize", 関数) でウィンドウサイズ変化時に関数が呼ばれる。
    window.addEventListener("resize", handleResize);

    // クリーンアップ: イベントリスナーを解除
    // 解除し忘れると、コンポーネントが何度もマウント/アンマウントするたびに
    // リスナーが増殖してメモリリーク&パフォーマンス低下を招く。
    return () => {
      window.removeEventListener("resize", handleResize);
    };
  // 初回マウント時にのみ設定
  }, []);

  return (
    <div>
      <h2>ウィンドウサイズ</h2>
      <p>幅: {windowSize.width}px</p>
      <p>高さ: {windowSize.height}px</p>
    </div>
  );
}
```

この結果、画面には「**幅: 1920px**」「**高さ: 1080px**」のように現在のウィンドウサイズが表示されます。ブラウザのウィンドウをリサイズすると、数値がリアルタイムに変化します。

▼ コードを1つずつ分解して解説

解説1: 現在のウィンドウサイズで初期化する

```
const [windowSize, setWindowSize] = useState<WindowSize>({
  width: window.innerWidth,
  height: window.innerHeight,
});
```

- `window.innerWidth` / `window.innerHeight` は、ブラウザの表示領域の現在の幅と高さです。
- これを `useState` の初期値に使うことで、**初回表示から正しいサイズ**を表示できます。
- `windowSize` は `{ width, height }` という形のオブジェクトstateです。

> **用語: window** … ブラウザのウィンドウ全体を表す組み込みオブジェクト。サイズなどの情報を持つ。

解説2: リサイズイベントを監視する関数を登録する

```
useEffect(() => {
  const handleResize = () => {
    setWindowSize({
      width: window.innerWidth,
      height: window.innerHeight,
    });
  };
  window.addEventListener("resize", handleResize);
```

- `handleResize` は「今のサイズを取得して state を更新する」関数です。
- `window.addEventListener("resize", handleResize)` で「**ウィンドウサイズが変わるたびに `handleResize` を呼んで**」と登録します。
- これでリサイズのたびに state が更新され、画面の表示も追従します。

> **用語: addEventListener** … 「○○が起きたらこの関数を呼ぶ」とイベントを登録するメソッド。

解説3: クリーンアップでリスナーを解除する

```
    return () => {
      window.removeEventListener("resize", handleResize);
    };
  // 初回マウント時にのみ設定
  }, []);
```

- `return` した関数（クリーンアップ）で、`removeEventListener` を呼んで**登録したリスナーを解除**します。
- 解除し忘れると、マウント/アンマウントのたびにリスナーが増殖し、メモリリークや動作の重さの原因になります。
- 依存配列は `[]` なので、登録は初回マウント時に1回だけ、解除はアンマウント時に行われます。

> **用語: removeEventListener** … addEventListenerで登録したイベントの監視を解除するメソッド。後片付けに必須。

6.5 コンポーネントライフサイクルと useEffect

1 マウント（初回表示）

コンポーネント関数が実行される

↓

JSX が返される

↓

DOM に反映

↓

useEffect が実行される（依存配列: []）

↓

2 更新（state/props 変更時）

繰り返し

state または props が変更

↓

コンポーネント関数が再実行

↓

新しい JSX が返される

↓

差分を検出して DOM を更新

↓

前回の useEffect のクリーンアップ関数を実行

↓

新しい useEffect が実行される（依存配列の値が変わった場合）

↓



上の図を、時間の流れに沿って横に並べ直すと次のようになります。「① レンダー → ② effect」が基本の組で、2回目以降は「③ 前回の片付け」が ② の直前に割り込む、と覚えると整理しやすくなります。

図3-11: useEffect の実行タイミング（依存配列 [count]、Strict Mode なしの場合）。下のサンプルの実際のログと同じ順番です

実行順序をログで確かめる

▼ このコードがやること（先に日本語で）：子コンポーネント `CountLogger` の中に、①レンダー（関数が呼ばれた）、②effect の実行、③クリーンアップ、のそれぞれで `console.log` を仕込みます。そして「最初の表示 → 【+1】ボタン →【消す】ボタン」と操作したときに、ログがどの順番で出るかを確かめます。クリーンアップの中の `count` が「いつの値」なのかにも注目してください。

```tsx
import { useEffect, useState } from "react";

// count を受け取って表示するだけの子コンポーネント
function CountLogger({ count }: { count: number }) {
  // ① コンポーネント関数が呼ばれた（レンダー）
  console.log(`① レンダー（count = ${count}）`);

  useEffect(() => {
    // ② 画面に反映されたあとで実行される
    console.log(`② effect 実行（count = ${count}）`);

    // ③ 次の effect の前・画面から消えるときに実行される
    return () => {
      console.log(`③ クリーンアップ（count = ${count} の effect を片付け）`);
    };
  }, [count]);

  return <p>count: {count}</p>;
}

export default function EffectTimingDemo() {
  const [count, setCount] = useState<number>(0);
  const [isVisible, setIsVisible] = useState<boolean>(true);

  return (
    <div>
      {/* isVisible が false になると CountLogger は画面から消える */}
      {isVisible && <CountLogger count={count} />}
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setIsVisible(false)}>消す</button>
    </div>
  );
}
```

▼ **実行結果**（ブラウザの Console。React 19 で実際に操作して記録したものです。`---` の行は操作の区切りとして書き足しています）

```
--- 最初の表示 ---
① レンダー（count = 0）
② effect 実行（count = 0）
--- 【+1】を押す ---
① レンダー（count = 1）
③ クリーンアップ（count = 0 の effect を片付け）
② effect 実行（count = 1）
--- 【消す】を押す ---
③ クリーンアップ（count = 1 の effect を片付け）
```

開発モードで **Strict Mode** が有効な場合（Next.js の開発サーバーは最初から有効です）は、最初の表示と更新のときにレンダーが2回ずつ呼ばれ、最初の表示では「effect → 片付け → effect」と、わざと1往復多く実行されます。実際のログは次のとおりです。

```
--- 最初の表示 ---
① レンダー（count = 0）
① レンダー（count = 0）
② effect 実行（count = 0）
③ クリーンアップ（count = 0 の effect を片付け）
② effect 実行（count = 0）
--- 【+1】を押す ---
① レンダー（count = 1）
① レンダー（count = 1）
③ クリーンアップ（count = 0 の effect を片付け）
② effect 実行（count = 1）
--- 【消す】を押す ---
③ クリーンアップ（count = 1 の effect を片付け）
```

▼ コードを1つずつ分解して解説

解説1: レンダーのログは「関数が呼ばれた」印

```
function CountLogger({ count }: { count: number }) {
  console.log(`① レンダー（count = ${count}）`);
```

- コンポーネント関数の本体に直接書いた `console.log` は、関数が呼ばれるたび（レンダーのたび）に実行されます。
- このタイミングでは、まだ画面は更新されていません。「新しい見た目の設計図（JSX）を作っている最中」です。

解説2: effect は画面に反映された「あと」

```
  useEffect(() => {
    console.log(`② effect 実行（count = ${count}）`);
```

- ログを見ると、必ず ① のあとに ② が出ています。effect は「画面に反映し終わったあと」に実行されるからです。
- 依存配列が `[count]` なので、【+1】で `count` が変わったときだけ再実行されます。【消す】のときは再実行されません。

解説3: クリーンアップは「前回の値」で動く

```
    return () => {
      console.log(`③ クリーンアップ（count = ${count} の effect を片付け）`);
    };
```

- 【+1】を押したときのクリーンアップは `count = 0` と表示されています。クリーンアップは「**前回の effect が作った片付け係**」なので、前回のスナップショット（0）を覚えているのです。
- 【消す】を押すと `CountLogger` が画面から消える（アンマウント）ので、最後のクリーンアップだけが実行されます。タイマーやイベント監視を止めるのは、まさにこのタイミングです（6.4）。

> **用語: アンマウント（unmount）** … コンポーネントが画面から取り除かれること。最後のクリーンアップが実行される。

6.6 useRef（値の保持・要素のフォーカス）

`useState` 以外にも、Reactには覚えておくと便利なフックがいくつかあります。ここからは、よく使う5つのフック（`useRef` / `useMemo` / `useCallback` / `useContext` / `useReducer`）を、それぞれ独立した小さなサンプルで順番に見ていきます。

最初は **`useRef`**（ユーズ・レフ：ref は reference〔参照〕の略）です。`useRef` には大きく2つの使い道があります。**(1) 画面に表示しない値をこっそり覚えておく** ことと、**(2) 画面の特定の要素（input など）を直接さわる** ことです。

> **いつ使う？**「値を覚えておきたいけれど、その値が変わっても画面を描き直す必要はない」とき（例: タイマーのID、前回の値など）。または「この入力欄にカーソルを合わせたい（フォーカスしたい）」のように、特定のHTML要素を直接操作したいとき。
>
> **普通の変数や `useState` と何が違う？** - **普通の変数（`let count = 0`）** … コンポーネントは再描画のたびに関数全体が実行され直すので、普通の変数は毎回0に戻ってしまいます。値を覚えておけません。 - **`useState`** … 値を覚えておけますが、`setXxx` で値を変えると**毎回画面が描き直されます**。 - **`useRef`** … 値を覚えておけて、しかも**値を変えても画面は描き直されません**。だから「画面に出さない裏方の値」を持つのにぴったりです。`ref.current` に値が入っていて、`ref.current = 新しい値` で書き換えます。

input にフォーカスを当てるカウントメモ

> **▼ このコードがやること（先に日本語で）**：ボタンを押すと、入力欄に自動でカーソルが移動（フォーカス）します。また「画面に出さないメモ用の数」をボタンを押すたびに+1して、`useRef` だと画面が描き直されないことを確かめます。`useRef` は「特定の入力欄を直接さわる」「画面に出さない値を覚える」の2役をこなす、と覚えてください。

```tsx
// ============================================================================
// useRef のサンプル（詳細コメント版）
// ----------------------------------------------------------------------------
// (1) 入力欄を直接さわってフォーカスを当てる
// (2) 画面に出さない値（メモ）をこっそり覚えておく
// ============================================================================

// React の useRef フックを取り込む。
import { useRef } from "react";

function FocusDemo() {
  // ------------------------------------------------------------------------
  // (1) 入力欄（input要素）を覚えておくための「箱」を作る
  // ------------------------------------------------------------------------
  // useRef<HTMLInputElement>(null) は
  //   「中身がinput要素になる予定の箱を、最初は空（null）で作って」という意味。
  // 後で <input ref={inputRef} /> と書くと、その入力欄が inputRef.current に入る。
  const inputRef = useRef<HTMLInputElement>(null);

  // ------------------------------------------------------------------------
  // (2) 画面に出さない「メモ用の数」を覚えておく箱
  // ------------------------------------------------------------------------
  // この値を増やしても画面は描き直されない（ここが useState との違い）。
  const clickCountRef = useRef<number>(0);

  // ボタンが押されたときの処理
  const handleFocus = () => {
    // inputRef.current は「今その入力欄を指しているか？」。
    // null でなければ .focus() でカーソルを当てる。
    // ?. は「null じゃなければ呼ぶ」という安全な書き方。
    inputRef.current?.focus();

    // メモ用の数を +1。current を直接書き換えるだけでOK。
    // setXxx ではないので画面は描き直されない。
    clickCountRef.current = clickCountRef.current + 1;

    // 確認用に、覚えている数をコンソールに出す。
    console.log(`フォーカスボタンを押した回数: ${clickCountRef.current}`);
  };

  return (
    <div>
      {/* ref={inputRef} と書くと、この入力欄が inputRef.current に入る */}
      <input ref={inputRef} type="text" placeholder="ここに入力" />
      <button onClick={handleFocus}>入力欄にフォーカス</button>
    </div>
  );
}
```

▼ ブラウザでの動き:

操作	起きること
「入力欄にフォーカス」を押す	入力欄に自動でカーソルが移動する
もう一度押す	カーソルが当たり、コンソールに「2」と出る（画面の見た目は変わらない）

▼ コードを1つずつ分解して解説

解説1: useRefをimportする

```
import { useRef } from "react";
```

- `useRef` を使うために、Reactから取り込みます。
- `useRef` は「**画面を描き直さずに値を覚えておく**」「**特定のHTML要素を直接さわる**」ための道具です。

> **用語: useRef** … `ref.current` という箱に値を入れて覚えておくフック。値を変えても再描画は起きない。

解説2: 要素を入れる「箱」を作る

```
  const inputRef = useRef<HTMLInputElement>(null);
```

- `useRef<HTMLInputElement>(null)` は「**input要素を入れる予定の箱を、最初は空（null）で作って**」という意味です。
- `<HTMLInputElement>` が「中身の型（=input要素）」、`(null)` が「最初の中身（まだ空っぽ）」です。
- 作った箱は `inputRef.current` で中身を取り出せます。

> **用語: ref（リファレンス）** …「参照」の意味。特定の要素や値を指し示しておくための箱。中身は `.current` に入る。

解説3: 画面に出さない値も覚えられる

```
  const clickCountRef = useRef<number>(0);
```

- 数値も `useRef` で覚えられます。`useState` と違い、**この値を変えても画面は描き直されません。**
- だから「画面には出さないけど覚えておきたい裏方の値」（押した回数のメモなど）にぴったりです。

解説4: 要素と箱を結びつけて、直接さわる

```
    inputRef.current?.focus();
    clickCountRef.current = clickCountRef.current + 1;
```

- `<input ref={inputRef} />` と書くことで、その入力欄が `inputRef.current` に入ります。
- `inputRef.current?.focus()` で「その入力欄にカーソルを当てる（フォーカスする）」操作をします。`?.` は「中身が空（null）でなければ呼ぶ」という安全な書き方です。
- `clickCountRef.current = ...` のように、ref の値は `.current` に直接代入して書き換えます（`setXxx` は不要です）。

> **用語: フォーカス** … 入力欄にカーソルが当たって、キーボード入力を受け付ける状態になること。

6.7 useMemo（重い計算結果のメモ化）

`useMemo`（ユーズ・メモ：memo は memoize〔メモ化＝計算結果を覚えておく〕の略）は、**重い計算の結果を覚えておいて、ムダな計算をくり返さない**ためのフックです。

> **いつ使う？**「時間のかかる計算（大きな配列の合計・並べ替え・絞り込みなど）があって、関係ない部分の再描画のたびに計算し直したくない」とき。
>
> **普通の計算（変数）と何が違う？** コンポーネントは再描画のたびに関数全体が**最初から実行され直す**ので、普通に書いた計算は**毎回実行されます**。`useMemo` で包むと、「指定した値が変わったときだけ計算し直し、変わっていなければ前回の結果を使い回す」ようになります。つまり**計算結果のキャッシュ（一時保存）**です。

合計金額をメモ化する

> **▼ このコードがやること（先に日本語で）**：たくさんの商品の合計金額を計算して表示します。別のボタン（テーマ切り替え）を押しても、商品リストが変わっていなければ合計の計算はやり直しません。`useMemo` を使うと「商品リストが変わったときだけ合計を計算し直す」ようにできる、という点を確かめます。

```tsx
// ============================================================================
// useMemo のサンプル（詳細コメント版）
// ----------------------------------------------------------------------------
// 重い合計計算を「リストが変わったときだけ」やり直す。
// ============================================================================

import { useState, useMemo } from "react";

function PriceTotal() {
  // 商品の値段リスト（このサンプルでは固定）
  const [prices] = useState<number[]>([1200, 800, 3000, 450, 980]);

  // 計算とは関係ない別の状態（テーマの明暗）。
  // これが変わるだけでもコンポーネントは再描画される。
  const [isDark, setIsDark] = useState<boolean>(false);

  // ----------------------------------------------------------------------
  // useMemo で「合計の計算」を包む
  // ----------------------------------------------------------------------
  // useMemo(計算する関数, 依存配列) の形。
  //   第1引数: 計算してその結果を return する関数
  //   第2引数: [prices] = 「prices が変わったときだけ計算し直す」
  // prices が変わらない限り、isDark を切り替えても再計算されない。
  const total = useMemo(() => {
    console.log("合計を計算しました（重い計算のつもり）");
    // reduce で配列の数値をすべて足し合わせる。
    return prices.reduce((sum, price) => sum + price, 0);
  }, [prices]);

  return (
    <div style={{ background: isDark ? "#333" : "#fff", color: isDark ? "#fff" : "#000"
}}>
      <p>合計金額: {total} 円</p>
      {/* このボタンを押しても prices は変わらないので、合計は再計算されない */}
      <button onClick={() => setIsDark(!isDark)}>テーマを切り替え</button>
    </div>
  );
}
```

▼ コンソールで確認できること:

操作	「合計を計算しました」が出るか
初回表示	出る（1回目の計算）
「テーマを切り替え」を押す	**出ない**（pricesが変わっていないので再計算されない）

▼ コードを1つずつ分解して解説

解説1: useMemoをimportする

```
import { useState, useMemo } from "react";
```

- `useMemo` を `useState` と一緒にReactから取り込みます。
- `useMemo` は「**計算結果を覚えておいて、ムダな計算をくり返さない**」ための道具です。

> **用語: メモ化（memoization）** … 計算結果を覚えておき、同じ条件なら再計算せず前回の結果を使い回すこと。

解説2: useMemoで重い計算を包む

```
  const total = useMemo(() => {
    return prices.reduce((sum, price) => sum + price, 0);
  }, [prices]);
```

- `useMemo(計算する関数, 依存配列)` の形で使います。第1引数が「計算して結果を `return` する関数」、第2引数が「いつ計算し直すか」です。
- 第2引数の `[prices]` が**依存配列**です。`prices` が変わったときだけ計算をやり直し、変わっていなければ**前回の結果をそのまま使い回します**。
- `prices.reduce(...)` は配列の数値をすべて足し合わせる計算です（ここでは合計＝重い計算の代わり）。

> **用語: 依存配列** … フックの第2引数。この中の値が変わったときだけ処理（ここでは計算）をやり直す。

解説3: 関係ないstateを変えても再計算されない

```
      <button onClick={() => setIsDark(!isDark)}>テーマを切り替え</button>
```

- このボタンは `isDark`（テーマの明暗）を切り替えるだけで、`prices` は変えません。
- そのためボタンを押して再描画されても、`useMemo` のおかげで**合計の計算はやり直されません**（コンソールに「合計を計算しました」が出ません）。
- これが `useMemo` の効果です。「依存していない変化では計算しない」ことでムダを減らせます。

> **用語: 再描画（再レンダリング）** … stateが変わると、コンポーネント関数がもう一度実行されて画面が更新されること。useMemoはこのときのムダ計算を防ぐ。

6.8 useCallback（関数のメモ化）

`useCallback`（ユーズ・コールバック）は、**関数そのものを覚えておいて、毎回作り直さない**ためのフックです。`useMemo` が「計算結果」を覚えるのに対し、`useCallback` は「関数」を覚えます。

> **いつ使う？** ほとんどの場合、**`React.memo` で最適化した子コンポーネントに関数を渡すとき**に使います。子に渡す関数が毎回新しく作られると、子は「中身が変わった」と勘違いしてムダに再描画されてしまいます。それを防ぐために関数を固定します。
>
> **普通の関数定義と何が違う？** コンポーネントは再描画のたびに関数全体が実行されるので、中で `const handleX = () => {...}` と書いた関数は**毎回「別の新しい関数」として作り直されます**。`useCallback` で包むと、「依存配列の値が変わらない限り、前回と同じ関数を使い回す」ようになります。

React.memo した子に関数を渡す

> **▼ このコードがやること（先に日本語で）**：「いいね」ボタンを持つ子部品を `React.memo` で最適化し、その子に押されたときの処理（関数）を `useCallback` で固定して渡します。親の別の数字（カウント）が変わっても、関数が同じなら子はムダに描き直されない、という点を確かめます。

```tsx
// ============================================================================
// useCallback のサンプル（詳細コメント版）
// ----------------------------------------------------------------------------
// React.memo した子に、useCallback で固定した関数を渡す。
// ============================================================================

import { useState, useCallback, memo } from "react";

// 子コンポーネント: React.memo で包むと、
// 「渡されたprops（ここでは onLike）が前回と同じなら描き直さない」ようになる。
const LikeButton = memo(function LikeButton(props: { onLike: () => void }) {
  console.log("LikeButton が描き直されました");
  return <button onClick={props.onLike}>いいね</button>;
});

function Parent() {
  // いいねの数
  const [likes, setLikes] = useState<number>(0);
  // いいねとは無関係なカウント（再描画を起こすためのもの）
  const [count, setCount] = useState<number>(0);

  // ------------------------------------------------------------------------
  // useCallback で「いいね処理の関数」を固定する
  // ------------------------------------------------------------------------
  // useCallback(関数, 依存配列) の形。
  //   第2引数 [] = 「ずっと同じ関数を使い回す（作り直さない）」
  // これで count が変わって親が再描画されても、handleLike は同じ関数のまま。
  // → 子(LikeButton)は「onLikeが変わっていない」と判断し、描き直さない。
  const handleLike = useCallback(() => {
    // 関数型の更新（prev => ...）を使うと、依存配列を空にしても安全に+1できる。
    setLikes((prev) => prev + 1);
  }, []);

  return (
    <div>
      <p>いいね: {likes}</p>
      <p>カウント: {count}</p>
      {/* 固定した関数を子に渡す */}
      <LikeButton onLike={handleLike} />
      {/* このボタンは count を変えるだけ。子は描き直されないはず */}
      <button onClick={() => setCount(count + 1)}>カウント+1</button>
    </div>
  );
}
```

▼ コンソールで確認できること:

操作	「LikeButton が描き直されました」が出るか
「カウント+1」を押す	**出ない**（handleLike が同じ関数なので子は描き直されない）
もし `useCallback` を外していたら	出る（毎回新しい関数になり、子がムダに描き直される）

▼ コードを1つずつ分解して解説

解説1: useCallbackとmemoをimportする

```
import { useState, useCallback, memo } from "react";
```

- `useCallback`（関数を固定する道具）と `memo`（子のムダな再描画を防ぐ道具）を取り込みます。
- `useCallback` は「**関数そのものを覚えておいて、毎回作り直さない**」ための道具です。

> **用語: React.memo** … コンポーネントを包むと、「渡されたpropsが前回と同じなら描き直さない」ようにする仕組み。

解説2: 子コンポーネントをmemoで包む

```
const LikeButton = memo(function LikeButton(props: { onLike: () => void }) {
  return <button onClick={props.onLike}>いいね</button>;
});
```

- `memo(...)` で子を包むと、親が再描画されても**渡されたprops（ここでは `onLike`）が前回と同じなら、子は描き直されません。**
- ただし「同じか」の判定は厳密なので、毎回新しい関数を渡すと「変わった」と見なされてしまいます。だから関数を固定する `useCallback` とセットで使います。

解説3: useCallbackで関数を固定する

```
  const handleLike = useCallback(() => {
    setLikes((prev) => prev + 1);
  }, []);
```

- `useCallback(関数, 依存配列)` の形で、関数を覚えておきます。
- 第2引数の `[]` は「**ずっと同じ関数を使い回す（作り直さない）**」という意味です。
- `setLikes((prev) => prev + 1)` のように「前の値（`prev`）から計算する」書き方にすると、依存配列が空でも常に正しく+1できます。

> **用語: useCallback** … 関数をメモ化（記憶）するフック。依存配列が変わらない限り、同じ関数を使い回す。

解説4: 固定した関数を子に渡す

```
      <LikeButton onLike={handleLike} />
      <button onClick={() => setCount(count + 1)}>カウント+1</button>
```

- 固定した `handleLike` を子の `onLike` に渡します。
- 「カウント+1」を押して親が再描画されても、`handleLike` は同じ関数のままです。だから子（`LikeButton`）は「props は変わっていない」と判断し、**ムダに描き直されません。**
- これが `useCallback` + `React.memo` のセットの効果です。

> **用語: props（プロップス）** … 親から子へ渡すデータや関数。子はこれを受け取って表示や動作に使う。

6.9 useContext（Propsのバケツリレー回避）

`useContext`（ユーズ・コンテキスト：context は「文脈・共有情報」の意味）は、**親から子・孫へとデータを直接届ける**ためのフックです。間にいるコンポーネントを経由せずに、離れた子孫が値を受け取れます。

> **いつ使う？** ログイン中のユーザー情報・テーマ（ダーク/ライト）・言語設定など、「アプリの広い範囲で使う共通データ」を配るとき。
>
> **普通のProps（バケツリレー）と何が違う？** Propsだけで深い階層に値を渡すと、親 → 子 → 孫 → ひ孫… と、**間のコンポーネントが自分は使わないのに値を受け取って下に渡す**必要があります。これを「Propsのバケツリレー（prop drilling）」と呼びます。`useContext` を使うと、間を飛ばして必要な場所で直接受け取れます。

テーマを離れた子に届ける

> **▼ このコードがやること（先に日本語で）**：アプリ全体の「テーマ（ライト/ダーク）」を、間のコンポーネントを経由せずに、奥にあるボタンに直接届けます。`createContext`（共有箱を作る）→ `Provider`（値を入れて配る）→ `useContext`（受け取る）という3ステップの流れを確かめます。

```tsx
// ============================================================================
// useContext のサンプル（詳細コメント版）
// ----------------------------------------------------------------------------
// createContext → Provider → useContext の3ステップで
// 離れた子孫に値を届ける。
// ============================================================================

import { createContext, useContext } from "react";

// ──────────────────────────────────────────────────────────────
// (1) 共有する箱（Context）を作る
// ──────────────────────────────────────────────────────────────
// createContext("light") の "light" は「もし値が配られていなかったときの初期値」。
const ThemeContext = createContext<string>("light");

// 一番奥のコンポーネント（孫）。Propsを一切受け取っていない点に注目。
function ThemedButton() {
  // (3) useContext で、配られている値を直接受け取る。
  const theme = useContext(ThemeContext);
  return (
    <button style={{ background: theme === "dark" ? "#333" : "#eee" }}>
      今のテーマ: {theme}
    </button>
  );
}

// 間にいるコンポーネント（子）。themeを受け取らず、ただ孫を表示するだけ。
function Toolbar() {
  return <ThemedButton />;
}

// 一番上のコンポーネント（親）。
function App() {
  return (
    // (2) Provider で値を「配る」。value に渡した値が子孫に届く。
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}
```

▼ **ポイント**: `Toolbar` は `theme` を一切さわっていないのに、奥の `ThemedButton` がちゃんと "dark" を受け取れます。Propsで渡していたら、`Toolbar` も `theme` を受け取って下に渡す必要がありました。

▼ コードを1つずつ分解して解説

解説1: createContextで共有箱を作る

```
import { createContext, useContext } from "react";

const ThemeContext = createContext<string>("light");
```

- `createContext` で「みんなで共有する箱」を作ります。`<string>` は中身の型、`("light")` は「値が配られていないときの初期値」です。
- この `ThemeContext` を通して、離れたコンポーネント同士が同じ値をやり取りします。

> **用語: Context（コンテキスト）** … 親から子孫へ、間を飛ばして値を共有するための仕組み。`createContext` で箱を作る。

解説2: Providerで値を配る

```
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
```

- `ThemeContext.Provider` で囲んだ範囲の**中にあるすべての子孫**に、`value` に渡した値（ここでは `"dark"`）が届きます。
- `Toolbar` も、その奥の `ThemedButton` も、この囲みの中なので値を受け取れます。

> **用語: Provider（プロバイダー）** … Contextの値を「配る人」。`value` に入れた値が、囲んだ範囲の子孫すべてに届く。

解説3: useContextで受け取る

```
function ThemedButton() {
  const theme = useContext(ThemeContext);
  return <button>今のテーマ: {theme}</button>;
}
```

- `useContext(ThemeContext)` と書くだけで、配られている値（`"dark"`）を**直接受け取れます**。
- `ThemedButton` はPropsを一切受け取っていません。間の `Toolbar` も `theme` を渡していません。それでも値が届くのが `useContext` の利点です。

> **用語: useContext** … Providerが配っている値を、間を飛ばして受け取るフック。Propsのバケツリレーを減らせる。

6.10 useReducer（複数のstateをまとめる）

`useReducer`（ユーズ・リデューサー：reduce〔まとめる〕の派生語）は、**複数の関連する状態の更新を1か所のルールにまとめる**ためのフックです。`useState` の「もう少し整理された版」と考えると分かりやすいです。

> **いつ使う？** 状態の種類が増えて更新パターンが複雑になってきたとき（例: フォームの複数項目、増減＋リセットがあるカウンターなど）。「どんな操作で状態がどう変わるか」を1か所にまとめて見通しよくできます。
>
> **`useState` と何が違う？** `useState` は値ごとに `setXxx` を呼んで更新します。`useReducer` は「**こういう操作（action）が来たら、状態をこう変える**」というルール（reducer関数）を1か所に集め、`dispatch(操作)` を呼ぶだけで更新します。更新ロジックが散らからず、まとまるのが利点です。

increment / decrement / reset を持つカウンター

> **▼ このコードがやること（先に日本語で）**：+1・-1・0に戻す、の3つの操作ができるカウンターを `useReducer` で作ります。「どの操作で数がどう変わるか」を1つの関数（reducer）にまとめ、ボタンからは `dispatch` でその操作を送るだけ、という流れを確かめます。

```tsx
// ============================================================================
// useReducer のサンプル（詳細コメント版）
// ----------------------------------------------------------------------------
// 「操作(action)が来たら状態をどう変えるか」を reducer 関数にまとめる。
// ============================================================================

import { useReducer } from "react";

// ---------------------------------------------------------------------------
// (1) 状態の形を決める
// ---------------------------------------------------------------------------
type CounterState = { count: number };

// (2) どんな操作があるかを決める（"type" で操作の種類を表す）
type CounterAction =
  | { type: "increment" }
  | { type: "decrement" }
  | { type: "reset" };

// ---------------------------------------------------------------------------
// (3) reducer 関数: 「今の状態」と「来た操作」を受け取り、「新しい状態」を返す
// ---------------------------------------------------------------------------
// ここに「どの操作で状態がどう変わるか」のルールを全部まとめる。
function counterReducer(state: CounterState, action: CounterAction): CounterState {
  switch (action.type) {
    case "increment":
      // +1した新しい状態を返す
      return { count: state.count + 1 };
    case "decrement":
      // -1した新しい状態を返す
      return { count: state.count - 1 };
    case "reset":
      // 0に戻した状態を返す
      return { count: 0 };
    default:
      // 知らない操作なら何もしない
      return state;
  }
}

function Counter() {
  // -------------------------------------------------------------------------
  // useReducer(reducer関数, 初期状態) を呼ぶ
  // -------------------------------------------------------------------------
  // 戻り値は [今の状態, dispatch関数] の2つ。
  //   state    … 今の状態（state.count で数を取り出す）
  //   dispatch … 「この操作をして」と reducer に伝える関数
  const [state, dispatch] = useReducer(counterReducer, { count: 0 });
```

```
  return (
    <div>
      <p>カウント: {state.count}</p>
      {/* dispatch({ type: "..." }) で操作を送る */}
      <button onClick={() => dispatch({ type: "decrement" })}>-1</button>
      <button onClick={() => dispatch({ type: "reset" })}>リセット</button>
      <button onClick={() => dispatch({ type: "increment" })}>+1</button>
    </div>
  );
}
```

▼ 動作の流れ:

操作	送られる action	count の変化
「+1」を押す	`{ type: "increment" }`	0 → 1
「-1」を押す	`{ type: "decrement" }`	1 → 0
「リセット」を押す	`{ type: "reset" }`	何でも → 0

▼ コードを1つずつ分解して解説

解説1: useReducerをimportする

```
import { useReducer } from "react";
```

- `useReducer` をReactから取り込みます。
- `useReducer` は「**状態の更新ルールを1か所にまとめる**」ための道具です。`useState` の整理版だと考えてください。

> **用語: useReducer** ... 「操作（action）が来たら状態をこう変える」というルールをまとめて状態を管理するフック。

解説2: 操作の種類を決める

```
type CounterAction =
  | { type: "increment" }
  | { type: "decrement" }
  | { type: "reset" };
```

- どんな操作（action）があるかを先に決めておきます。`type` が操作の名前です。
- ここでは「increment（+1）」「decrement（-1）」「reset（0に戻す）」の3つを用意しています。

> **用語: action（アクション）** ... 「何をしたいか」を表すオブジェクト。`type` でどの操作かを示す。

解説3: reducer関数に更新ルールをまとめる

```
function counterReducer(state: CounterState, action: CounterAction): CounterState {
  switch (action.type) {
    case "increment":
      return { count: state.count + 1 };
    case "decrement":
      return { count: state.count - 1 };
    case "reset":
      return { count: 0 };
    default:
      return state;
  }
}
```

- `reducer` は「**今の状態（state）と、来た操作（action）を受け取り、新しい状態を返す**」関数です。
- `switch` で「どの操作か」を見分け、操作ごとに新しい状態を `return` します。
- 更新ルールがこの1か所に集まるので、`useState` のように更新処理があちこちに散らばりません。

> **用語: reducer（リデューサー）** ... 「今の状態」と「操作」から「新しい状態」を作る関数。状態更新のルールをまとめる役。

解説4: useReducerを呼んでdispatchで操作を送る

```
  const [state, dispatch] = useReducer(counterReducer, { count: 0 });
  ...
      <button onClick={() => dispatch({ type: "increment" })}>+1</button>
```

- `useReducer(reducer関数, 初期状態)` を呼ぶと、`[今の状態, dispatch関数]` が返ります。
- `state.count` で今の数を取り出し、画面に表示します。
- ボタンでは `dispatch({ type: "increment" })` のように「操作を送る」だけです。すると reducer がルールに従って新しい状態を作り、画面が更新されます。

> **用語: dispatch（ディスパッチ）** ... 「この操作をして」と reducer に伝える関数。`dispatch(action)` の形で呼ぶ。

`useReducer` の流れを図にすると、次のようになります。レストランにたとえると、ボタン（お客さん）は「何が欲しいか」を注文票（action）に書いて渡すだけで、料理の作り方（状態の計算）は厨房（reducer）が全部知っている、という分担です。

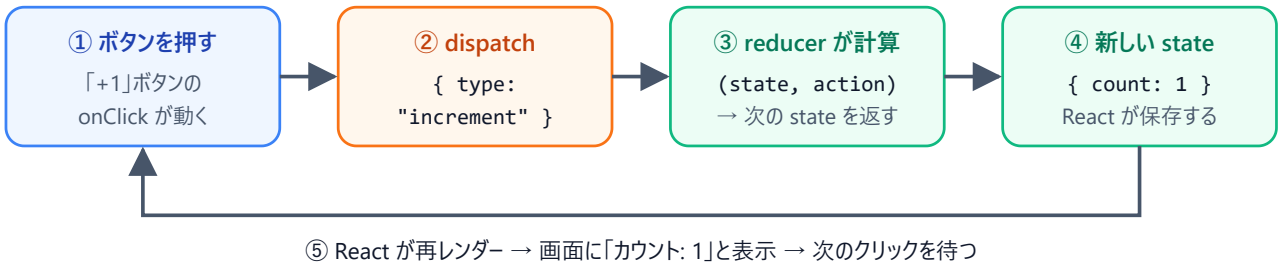


図3-12: useReducer の流れ。画面は「何が起きたか（action）」を送るだけで、計算は reducer にまとまっています

reducer を React なしで動かしてみる

> **▼ このコードがやること（先に日本語で）**：上のカウンターの `counterReducer` は、React の機能を何も使っていない「ただの関数」です。そこで、ボタンを「+1, +1, -1, リセット」の順に押したつもりで action を並べ、Node.js で1つずつ reducer に通して、状態がどう変わるかを表示します。最後に、配列の `reduce` メソッドで同じ計算を一気に行い、「reducer」という名前の由来も確かめます。

```ts
// ファイル名: reducer.ts
// 状態の形
type CounterState = { count: number };

// 操作（action）の種類
type CounterAction =
  | { type: "increment" }
  | { type: "decrement" }
  | { type: "reset" };

// 6.10 と同じ reducer 関数（React を使わない、ただの関数）
function counterReducer(state: CounterState, action: CounterAction): CounterState {
  switch (action.type) {
    case "increment":
      return { count: state.count + 1 };
    case "decrement":
      return { count: state.count - 1 };
    case "reset":
      return { count: 0 };
    default:
      return state;
  }
}

// ボタンを「+1, +1, -1, リセット」の順に押したつもりの action の列
const actions: CounterAction[] = [
  { type: "increment" },
  { type: "increment" },
  { type: "decrement" },
  { type: "reset" },
];

// dispatch の代わりに、action を1つずつ reducer に通す
let state: CounterState = { count: 0 };
for (const action of actions) {
  const next = counterReducer(state, action);
  console.log(`${action.type}: ${state.count} → ${next.count}`);
  state = next;
}

// 配列の reduce を使うと「最終状態」を一度に計算できる（名前の由来）
const finalState = actions.reduce(counterReducer, { count: 0 });
console.log("reduce で計算した最終状態:", finalState);
```

▼ 実行結果（`node reducer.ts`）

```
increment: 0 → 1
increment: 1 → 2
decrement: 2 → 1
reset: 1 → 0
reduce で計算した最終状態: { count: 0 }
```

▼ コードを1つずつ分解して解説

解説1: dispatch の代わりに reducer を直接呼ぶ

```
const next = counterReducer(state, action);
```

- React の中で `dispatch(action)` が呼ばれると、React は内部でこれと同じように `counterReducer(今の state, action)` を呼び、返ってきた値を新しい state として保存します（図の ③④）。
- reducer は「今の state」と「action」だけで次の state が決まる関数なので、React がなくてもこうして動作を確かめられます。

解説2: 毎回「新しいオブジェクト」を返している

```
case "increment":
  return { count: state.count + 1 };
```

- reducer は `state.count++` のように元の state を書き換えず、`{ count: ... }` という**新しいオブジェクト**を返しています。4.4 で学んだ「直接書き換えず、新しく作って差し替える」ルールは、reducer でも同じです。

解説3: 名前の由来は配列の reduce

```
const finalState = actions.reduce(counterReducer, { count: 0 });
```

- 配列の `reduce`（リデュース：まとめる）は、「初期値」と「要素を1つずつ取り込む関数」を受け取り、配列全体を1つの値にまとめるメソッドです。
- 「今の状態」と「次の action」から「次の状態」を作る関数は、`reduce` にそのまま渡せる形をしています。だから **reducer**（まとめる係）と呼ばれます。

> **用語: 純粋関数（pure function）** … 同じ入力には必ず同じ出力を返し、外の値を書き換えない関数。reducer は純粋関数として書く。

---

7. カスタムフック

7.1 なぜカスタムフックを作るのか

複数のコンポーネントで**同じロジック**を使い回したい場合、**カスタムフック**（Custom Hook：ユーザー定義のフック。`use`で始まる関数として作る再利用可能なロジック）を作ります。

カスタムフックなしの場合:

```
// ComponentA.tsx
function ComponentA() {
  // ウィンドウサイズの state
  const [windowSize, setWindowSize] = useState({ width: 0, height: 0 });

  useEffect(() => {
    // リサイズ時のハンドラ
    const handleResize = () => {
      setWindowSize({ width: window.innerWidth, height: window.innerHeight });
    };
    // 初回値を反映
    handleResize();
    // イベント登録
    window.addEventListener("resize", handleResize);
    // クリーンアップ
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  return <p>幅: {windowSize.width}</p>;
}

// ComponentB.tsx – 全く同じロジックをコピペ...
function ComponentB() {
  // 同じ state と同じ effect をコピペで持っている
  const [windowSize, setWindowSize] = useState({ width: 0, height: 0 });

  useEffect(() => {
    const handleResize = () => {
      setWindowSize({ width: window.innerWidth, height: window.innerHeight });
    };
    handleResize();
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  return <p>高さ: {windowSize.height}</p>;
}
```

このように同じコードを何度も書くのは **DRY 原則（Don't Repeat Yourself）** に反します。

7.2 基本的な作り方

カスタムフックのルール: - 関数名は `use` で始める（必須。React がフックとして認識する） - 中で useState、useEffect などのフックを使える - 値や関数を返す

useWindowSize フック

> **▼ このコードがやること（先に日本語で）**:「今のブラウザ画面の横幅・高さを教えてくれる」**自作の道具（カスタムフック）** を作ります。カスタムフックとは「**`useState` や `useEffect` を使う処理を、`use〇〇` という名前の関数にまとめて、複数のコンポーネントで使い回せるようにしたもの**」です。ここでは「画面サイズを state で覚え、`useEffect` で"画面サイズが変わったら更新する"仕掛けをセットする」処理を `useWindowSize` という1つの関数に閉じ込めています。こうしておくと、どの画面からでも `const size = useWindowSize()` の1行で画面サイズを使えます。

```tsx
import { useState, useEffect } from "react";

type WindowSize = {
  width: number;
  height: number;
};

// カスタムフック: ウィンドウサイズを返す
// 関数名は必ず use で始める（React がフックとして認識する条件）。
// 戻り値の型を WindowSize と明示しておくと利用側の補完が効く。
function useWindowSize(): WindowSize {
  // 内部で他のフック（useState, useEffect）を呼んで状態管理。
  const [windowSize, setWindowSize] = useState<WindowSize>({
    width: window.innerWidth,
    height: window.innerHeight,
  });

  useEffect(() => {
    const handleResize = () => {
      setWindowSize({
        width: window.innerWidth,
        height: window.innerHeight,
      });
    };

    window.addEventListener("resize", handleResize);
    // クリーンアップでリスナー解除
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  // 最終的に値を返す。返した値が呼び出し元コンポーネントで使える。
  return windowSize;
}

// 使い方: どのコンポーネントでも簡単に使える！
function Header() {
  // 分割代入で width だけ取り出す
  const { width } = useWindowSize();
  // 三項演算子で表示テキストを切替
  return <header>{width > 768 ? "デスクトップメニュー" : "モバイルメニュー"}</header>;
}

function Footer() {
  // 同じカスタムフックを別コンポーネントから呼んでも、それぞれ独立に state を持つ。
  // 「フックの共有」はロジックの共有であり、state そのものは共有されない点に注意。
  const { width, height } = useWindowSize();
  return (
    <footer>
      画面サイズ: {width} x {height}
```

```
      </footer>
    );
}
```

> `Header` コンポーネントでは、画面幅が768pxより大きい場合「**デスクトップメニュー**」、小さい場合「**モバイルメニュー**」と表示されます。ウィンドウをリサイズすると自動的に切り替わります。

▼ コードを1つずつ分解して解説

解説1: use で始まる関数として定義する

```
function useWindowSize(): WindowSize {
  const [windowSize, setWindowSize] = useState<WindowSize>({
    width: window.innerWidth,
    height: window.innerHeight,
  });
```

- カスタムフックは**関数名を必ず `use` で始めます**（Reactがフックとして認識する条件）。
- `(): WindowSize` は「この関数は WindowSize 型の値を返す」という宣言です。
- 中で `useState` などの普通のフックを使えます。ここでは画面サイズを state で覚えています。

> **用語: カスタムフック** … `useState` や `useEffect` を使う処理を `use〇〇` という関数にまとめ、使い回せるようにしたもの。

解説2: 処理の中身は通常のコンポーネントと同じ

```
  useEffect(() => {
    const handleResize = () => {
      setWindowSize({ width: window.innerWidth, height: window.innerHeight });
    };
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);
  return windowSize;
}
```

- 中身は、6.4 のウィンドウサイズ監視とまったく同じ処理（リスナー登録＋クリーンアップ）です。
- 違いは「これをコンポーネントの中ではなく**関数に閉じ込めている**」点だけです。
- 最後に `return windowSize;` で値を返します。**この返した値が、呼び出し元のコンポーネントで使えます。**

> **用語: ロジックの切り出し** … 複数の場所で使う処理を関数にまとめること。カスタムフックはその一種。

解説3: どのコンポーネントからも1行で使える

```
function Header() {
  const { width } = useWindowSize();
  return <header>{width > 768 ? "デスクトップメニュー" : "モバイルメニュー"}</header>;
}
```

- `const { width } = useWindowSize();` の**1行**で、画面幅を取得できます。
- `Header` でも `Footer` でも同じ書き方で使えます。同じロジックを何度もコピペせずに済みます。
- 注意：別コンポーネントから呼ぶと**それぞれ独立した state を持ちます**。「ロジックの共有」であって「state そのものの共有」ではありません。

> **用語: DRY原則** … 同じコードをくり返さない（Don't Repeat Yourself）という考え方。カスタムフックで実現できる。

---

useLocalStorage フック

```
import { useState, useEffect } from "react";

// ローカルストレージと同期する state を提供するカスタムフック
// <T> は「型を引数で受け取る」ジェネリクス。呼び出し側が string でも number でも指定可能。
// 戻り値の型は「[現在値, 更新関数]」のタプル型（順序固定の配列型）。
function useLocalStorage<T>(key: string, initialValue: T): [T, (value: T) => void] {
  // 初期値: ローカルストレージに値があればそれを使う
  // useState の第1引数に「関数」を渡すと、その関数の戻り値が初期値になる（遅延初期化）。
  // 重い処理を初回マウント時だけ実行したいときに使うパターン。
  const [storedValue, setStoredValue] = useState<T>(() => {
    try {
      // localStorage は文字列でしか保存できないので、保存時は JSON.stringify、
      // 読み出し時は JSON.parse する。
      const item = window.localStorage.getItem(key);
      // item が null（未保存）なら初期値、あればJSONパース。
      // as T は型アサーション（「これは T 型ですよ」と TS に教える）。
      return item ? (JSON.parse(item) as T) : initialValue;
    } catch {
      // パース失敗やストレージ無効時は初期値にフォールバック
      return initialValue;
    }
  });

  // state が変わったらローカルストレージにも保存
  useEffect(() => {
    try {
      window.localStorage.setItem(key, JSON.stringify(storedValue));
    } catch (error) {
      // プライベートブラウジング等で書き込み失敗する可能性がある
      console.error("ローカルストレージへの保存に失敗:", error);
    }
  // key と storedValue のどちらか変わったら再実行
  }, [key, storedValue]);

  // 配列で「現在値」と「更新関数」を返す → useState とほぼ同じインターフェース
  return [storedValue, setStoredValue];
}

// 使い方
function Settings() {
  // ジェネリクスで型を指定。<string> は省略しても初期値から推論される。
  const [theme, setTheme] = useLocalStorage<string>("theme", "light");
  const [fontSize, setFontSize] = useLocalStorage<number>("fontSize", 16);

  return (
    <div>
      <h2>設定</h2>
```

```
      <div>
        <label>テーマ: </label>
        <select value={theme} onChange={(e) => setTheme(e.target.value)}>
          <option value="light">ライト</option>
          <option value="dark">ダーク</option>
        </select>
      </div>

      <div>
        <label>フォントサイズ: {fontSize}px</label>
        <input
          // スライダー入力
          type="range"
          // 最小値
          min={12}
          // 最大値
          max={24}
          value={fontSize}
          // 文字列→数値変換
          onChange={(e) => setFontSize(Number(e.target.value))}
        />
      </div>

      {/* テンプレートリテラルで "16px" のようなCSS値を作る */}
      <p style={{ fontSize: `${fontSize}px` }}>
        このテキストのフォントサイズが変わります。
      </p>
    </div>
  );
}
```

この結果、画面にはテーマ切り替えのセレクトボックスとフォントサイズのスライダーが表示されます。

フォントサイズを20pxに変更すると、プレビューテキストのサイズが大きくなります。

ページを再読み込みしても設定が保持されます（ローカルストレージに保存されているため）。

▼ コードを1つずつ分解して解説

解説1: ジェネリクスで「どんな型でも扱える」フックにする

```
function useLocalStorage<T>(key: string, initialValue: T): [T, (value: T) => void] {
```

- `<T>` は**ジェネリクス**で、「型を呼び出し側から受け取る」仕組みです。文字列でも数値でも使えるようになります。
- 引数の `key` は保存場所の名前、`initialValue` は初期値（型 `T`）です。
- 戻り値の `[T, (value: T) => void]` は「`[現在値, 更新関数]`」という、`useState` とよく似た形です。

> **用語: ジェネリクス（ `<T>` ）** ... 型を後から決められるようにする仕組み。1つのフックを色々な型で使い回せる。

解説2: 初回だけストレージから値を読み込む（遅延初期化）

```
  const [storedValue, setStoredValue] = useState<T>(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? (JSON.parse(item) as T) : initialValue;
    } catch {
      return initialValue;
    }
  });
```

- `useState(() => {...})` のように**関数を渡す**と、その戻り値が初期値になります。これを**遅延初期化**と呼び、重い処理を初回だけ実行できます。
- `localStorage.getItem(key)` で保存済みの値を読み、あれば `JSON.parse` でデータに戻し、なければ初期値を使います。
- `try/catch` で、読み込みやパースに失敗しても初期値で安全に動くようにしています。

> **用語: localStorage** ... ブラウザに値を保存しておく仕組み。ページを閉じても残る。文字列でのみ保存できる。

解説3: 値が変わるたびにストレージへ保存する

```
  useEffect(() => {
    try {
      window.localStorage.setItem(key, JSON.stringify(storedValue));
    } catch (error) {
      console.error("ローカルストレージへの保存に失敗:", error);
    }
  }, [key, storedValue]);
  return [storedValue, setStoredValue];
}
```

- `useEffect` の依存配列が `[key, storedValue]` なので、値が変わるたびにストレージへ保存します。
- localStorage は文字列しか保存できないので、`JSON.stringify` でオブジェクトを文字列に変換してから保存します。
- 最後に `[storedValue, setStoredValue]` を返すので、利用側は `useState` と同じ感覚で使えます。

用語: JSON.stringify / JSON.parse … stringifyはデータを文字列に、parseは文字列をデータに戻す。localStorage保存に必須。

useToggle フック

```
import { useState, useCallback } from "react";

// true/false を切り替えるシンプルなカスタムフック
// useCallback は「関数をメモ化（記憶）して、依存配列が変わるまで同じ参照を返す」フック。
// 子コンポーネントに props として関数を渡すときに、不要な再レンダリングを防ぐ目的で使う。
function useToggle(initialValue: boolean = false): [boolean, () => void] {
  // 真偽値の state
  const [value, setValue] = useState<boolean>(initialValue);

  // useCallback(関数, 依存配列) で関数をメモ化。
  // 依存配列 [] なので、この toggle 関数の参照はコンポーネントの寿命を通じて変わらない。
  // setValue(prev => !prev) は関数型更新で、前の値を反転する。
  const toggle = useCallback(() => {
    setValue((prev) => !prev);
  }, []);

  // [現在値, 切替関数] のタプルを返す
  return [value, toggle];
}

// 使い方
function App() {
  // 3つの独立した toggle 状態
  const [isMenuOpen, toggleMenu] = useToggle(false);
  const [isDarkMode, toggleDarkMode] = useToggle(false);
  const [isModalOpen, toggleModal] = useToggle(false);

  return (
    // 背景・文字色を isDarkMode で切替
    <div style={{ backgroundColor: isDarkMode ? "#333" : "#fff", color: isDarkMode ?
"#fff" : "#000" }}>
      <button onClick={toggleDarkMode}>
        {isDarkMode ? "ライトモード" : "ダークモード"}に切り替え
      </button>

      <button onClick={toggleMenu}>
        メニュー{isMenuOpen ? "を閉じる" : "を開く"}
      </button>

      {/* && 演算子: isMenuOpen が true のときだけ <nav>...</nav> を表示 */}
      {isMenuOpen && (
        <nav>
          <ul>
            <li>ホーム</li>
            <li>書籍一覧</li>
            <li>設定</li>
          </ul>
```

```
        </nav>
      )}

      <button onClick={toggleModal}>モーダルを開く</button>

      {isModalOpen && (
        <div style={{ border: "2px solid #ccc", padding: "16px", margin: "16px" }}>
          <h3>モーダルの内容</h3>
          <p>これはモーダルウィンドウです。</p>
          {/* モーダル内の閉じるボタンも同じ toggleModal を共有 */}
          <button onClick={toggleModal}>閉じる</button>
        </div>
      )}
    </div>
  );
}
```

この結果、画面には3つのボタンが表示されます。

【ダークモードに切り替え】を押すと、背景が黒・文字が白に変わり、ボタンのテキストが「**ライトモードに切り替え**」に変わります。

【メニューを開く】を押すと、ナビゲーションリストが表示され、ボタンが「**メニューを閉じる**」に変わります。

▼ コードを1つずつ分解して解説

解説1: 真偽値を切り替えるフックを定義する

```
function useToggle(initialValue: boolean = false): [boolean, () => void] {
  const [value, setValue] = useState<boolean>(initialValue);
```

- `useToggle` は「`true` / `false` を切り替える」だけのシンプルなカスタムフックです。
- 引数 `initialValue: boolean = false` は「初期値。省略したら false」というデフォルト値付きの引数です。
- 戻り値の型 `[boolean, () => void]` は「`[現在値, 切替関数]`」を表します。

用語: トグル（toggle） … on/offを交互に切り替えること。ここでは true ⇄ false の反転。

解説2: useCallbackで切替関数をメモ化する

```
  const toggle = useCallback(() => {
    setValue((prev) => !prev);
  }, []);
  return [value, toggle];
}
```

- `setValue((prev) => !prev)` は**関数型更新**で、`!prev`（前の値の反転）にします。`true` なら `false`、`false` なら `true` になります。
- `useCallback(関数, [])` は「**関数をメモ化（記憶）して、毎回作り直さない**」フックです。依存配列が `[]` なので、この `toggle` の参照はずっと同じになります。
- 子コンポーネントに関数を渡すとき、不要な再レンダリングを防ぐ目的で使います。

> **用語: useCallback** … 関数をメモ化し、依存配列が変わるまで同じ関数を返すフック。参照の安定化に使う。

解説3: 複数の独立したトグル状態を使う

```
  const [isMenuOpen, toggleMenu] = useToggle(false);
  const [isDarkMode, toggleDarkMode] = useToggle(false);
  const [isModalOpen, toggleModal] = useToggle(false);
```

- 同じ `useToggle` を3回呼んで、メニュー・ダークモード・モーダルの**3つの独立した状態**を作っています。
- それぞれ `[現在値, 切替関数]` を受け取り、`onClick={toggleMenu}` のようにボタンに結びつけます。
- `{isMenuOpen && <nav>...</nav>}` のように `&&` で「true のときだけ表示」と組み合わせるのが定番です。

> **用語: 状態の独立性** … 同じカスタムフックを複数回呼んでも、それぞれ別々のstateを持つ。混ざらない。

7.3 useBooks フックの予告

カスタムフックを使うと、書籍管理アプリのデータ操作を次のような `useBooks` フックにまとめる設計もできます。ここでは「カスタムフックで整理すると、画面側のコードがどれだけすっきりするか」の完成イメージとして眺めてください。

> **注意:** 本書のアプリ（第7章以降）では、一覧のデータ取得を Next.js の **Server Component**（サーバー側で動く部品。第4章で学びます）が担当するため、この `useBooks` はそのままの形では登場しません。ただし「ロジックを1か所にまとめて、画面は表示に集中させる」という考え方は、第7章の `BookForm` などの設計でもそのまま活きています。

```tsx
// 予告: 後の章で実装するカスタムフック
// このフックが返す値の型
type UseBooksReturn = {
  // 全書籍データ
  books: Book[];
  // 読み込み中フラグ
  loading: boolean;
  // エラーメッセージ または null
  error: string | null;
  // Omit<Book, "id"> は「Book 型から id プロパティを除外した型」（新規追加時はIDは未確定）
  // Promise<void> は「非同期処理だが戻り値なし」を表す
  addBook: (book: Omit<Book, "id">) => Promise<void>;
  // Partial<Book> は「Book の全プロパティをオプショナル化」した型（一部だけ更新する用）
  updateBook: (id: number, updates: Partial<Book>) => Promise<void>;
  deleteBook: (id: number) => Promise<void>;
  // 検索クエリ設定
  searchBooks: (query: string) => void;
  // 検索結果
  filteredBooks: Book[];
};

function useBooks(): UseBooksReturn {
  // API との通信、state 管理、検索ロジックなどを
  // このフックに集約するイメージ（中身は省略。本書のアプリでは第7章以降の
  // Server Component と Supabase で同じ役割を実現します）
}

// 使い方（完成イメージ）
function BookListPage() {
  // 必要な値だけを分割代入で取り出す
  const { books, loading, error, deleteBook, searchBooks, filteredBooks } = useBooks();

  // 状態別の早期 return
  if (loading) return <p>読み込み中...</p>;
  if (error) return <p>エラー: {error}</p>;

  return (
    <div>
      {/* 子コンポーネントに「検索処理」を関数として渡す */}
      <SearchBar onSearch={searchBooks} />
      {/* リストデータと削除コールバックを渡す */}
      <BookList books={filteredBooks} onDelete={deleteBook} />
    </div>
  );
}
```

このカスタムフックを使うことで、書籍データの取得・追加・更新・削除・検索といったすべてのロジックが1箇所にまとまり、コンポーネントは UI の描画に集中できるようになります。

---

8. よくあるミスと対処法

8.1 state の直接変更

最も多い間違い: state のオブジェクトや配列を直接変更してしまうこと。

▼ **このコードがやること（先に日本語で）**：初心者が**最もやりがちな失敗**——「stateを直接書き換えてしまう」——の **NG例とOK例を並べて**示します。`user.name = "佐藤"` のように直接いじると、**Reactが変化に気づけず、画面が更新されません。**正しくは、スプレッド構文 `...` で**新しいオブジェクト／配列を作って** `setUser(...)` / `setItems(...)` に渡します。「直接いじらず、作り替えて差し替える」——4.4・4.5でも出たこの鉄則を、失敗例とセットで体に染み込ませる節です。

```tsx
import { useState } from "react";

type User = {
  name: string;
  age: number;
};

function UserEditor() {
  // オブジェクト state と配列 state
  const [user, setUser] = useState<User>({ name: "田中", age: 25 });
  const [items, setItems] = useState<string[]>(["A", "B", "C"]);

  // ========== NG な例 ==========

  const badUpdateName = () => {
    // NG: オブジェクトのプロパティを直接変更
    // user オブジェクトは React 内部にも参照されている。
    // .name を直接書き換えてしまうと、React が「同じ参照だから変化なし」と判断して再描画しない。
    user.name = "鈴木";
    // 同じ参照のオブジェクトなので React は変化を検知できない！
    setUser(user);
  };

  const badAddItem = () => {
    // NG: 配列を直接変更
    // push は配列を変更する破壊的メソッド。
    items.push("D");
    // 同じ参照の配列なので React は変化を検知できない！
    setItems(items);
  };

  const badSortItems = () => {
    // NG: sort は元の配列を変更する破壊的メソッド
    items.sort();
    setItems(items);
  };

  // ========== OK な例 ==========

  const goodUpdateName = () => {
    // OK: 新しいオブジェクトを作成
    // {...user, name: "鈴木"} は「user のコピー + name 上書き」の新しいオブジェクト。
    setUser({ ...user, name: "鈴木" });
  };

  const goodAddItem = () => {
    // OK: 新しい配列を作成
    // [...items, "D"] は「items の全要素 + "D"」の新しい配列。
    setItems([...items, "D"]);
```

```
  };

  const goodSortItems = () => {
    // OK: コピーしてからソート
    // [...items] でまずコピーを作り、その上で .sort()。元の items は変わらない。
    setItems([...items].sort());
  };

  return (
    <div>
      <p>名前: {user.name}</p>
      {/* join(", ") は配列要素を区切り文字で連結した文字列を作る */}
      <p>アイテム: {items.join(", ")}</p>
      <button onClick={goodUpdateName}>名前を変更</button>
      <button onClick={goodAddItem}>アイテム追加</button>
      <button onClick={goodSortItems}>ソート</button>
    </div>
  );
}
```

> **なぜ直接変更がダメなのか**: React は state の更新を**参照の比較（ `===` ：Object.is 相当）** で検出します。同じオブジェクト/配列への参照のままだと、中身が変わっていても「変化なし」と判断され、再レンダリングが発生しません。これを **イミュータブル更新**（Immutable Update：不変更新。常に新しいオブジェクトを作って更新する考え方）と呼びます。

▼ コードを1つずつ分解して解説

解説1: NG例 — オブジェクト/配列を直接書き換える

```
  const badUpdateName = () => {
    user.name = "鈴木";
    // 同じ参照のオブジェクトなので React は変化を検知できない！
    setUser(user);
  };
  const badAddItem = () => {
    items.push("D");
    // 同じ参照の配列なので React は変化を検知できない！
    setItems(items);
  };
```

- `user.name = "鈴木"` のように**直接プロパティを書き換える**のはNGです。
- `items.push("D")` の `push` も、元の配列を**直接変更する破壊的メソッド**なのでNGです。
- どちらも「同じ入れ物」のままなので、Reactは「参照が同じ＝変化なし」と判断し、**画面が更新されません。**

> **用語: 破壊的メソッド** … `push` / `sort` / `splice` など、元の配列そのものを変更してしまうメソッド。stateには使わない。

解説2: OK例 — スプレッド構文で新しいものを作る

```
const goodUpdateName = () => {
  setUser({ ...user, name: "鈴木" });
};
const goodAddItem = () => {
  setItems([...items, "D"]);
};
const goodSortItems = () => {
  setItems([...items].sort());
};
```

- `{ ...user, name: "鈴木" }` は「user をコピーして name だけ上書きした**新しいオブジェクト**」です。
- `[...items, "D"]` は「items の全要素＋"D" の**新しい配列**」です。
- 並び替えも `[...items].sort()` のように、まず `[...items]` でコピーを作ってから sort します。元の配列は変えません。
- いずれも「**新しい入れ物**」を渡すので、Reactが変化を検知して画面を更新します。

> **用語: イミュータブル更新（不変更新）** … 元を変えず、常に新しいオブジェクト/配列を作って差し替える更新方法。Reactの鉄則。

8.2 useEffect の無限ループ

> **▼ このコードがやること（先に日本語で）：** `useEffect` で起こりがちな「**無限ループ**」——処理が止まらずアプリが固まる失敗——の**典型パターンと、その直し方**を示します。原因はだいたい同じで、「**`useEffect` の中で state を更新しているのに、依存配列の指定を間違えている**」こと。すると『state更新 → 再描画 → またuseEffect実行 → state更新 →…』が永遠に続きます。この節で「やってはいけない書き方」を知っておくと、実際に画面が固まったとき、すぐ原因に気づけます。

```tsx
import { useState, useEffect } from "react";

function InfiniteLoopExample() {
  const [count, setCount] = useState<number>(0);
  const [data, setData] = useState<string[]>([]);

  // ========== NG: 無限ループ ==========

  // パターン1: 依存配列を省略して state を更新
  // 依存配列なしの useEffect は毎回のレンダリング後に走る。
  // 中で setCount を呼ぶと state が変わり → 再レンダリング → またこの useEffect が走る → ...
  useEffect(() => {
    // state 更新 → 再レンダリング → useEffect 再実行 → state 更新 → ...
    setCount(count + 1);
  // 依存配列がない！
  });

  // パターン2: useEffect 内で毎回新しいオブジェクト/配列を state に設定
  // ["A","B","C"] は毎回「新しい配列オブジェクト」として作られるため、
  // 参照比較では常に「変化あり」となり、再描画が止まらない。
  useEffect(() => {
    // 毎回新しい配列オブジェクトが作られる → 再レンダリング → ...
    setData(["A", "B", "C"]);
  // 依存配列がない！
  });

  // パターン3: 依存配列に毎回変わる値を入れる
  // { key: "value" } は実行のたびに新しいオブジェクト参照になるため、
  // 毎レンダリングで「依存値が変わった」とみなされてしまう。
  useEffect(() => {
    console.log("実行");
  // オブジェクトリテラルは毎回新しい参照 → 毎回実行
  }, [{ key: "value" }]);

  // ========== OK: 正しい使い方 ==========

  // 修正1: 適切な依存配列を指定
  // [] で初回マウント時のみ実行。関数型更新で外部の count に依存しない。
  useEffect(() => {
    // 初回のみ実行
    setCount((prev) => prev + 1);
  // 空配列 → 初回マウント時のみ
  }, []);

  // 修正2: 条件付きで実行
  // 「data が空のときだけ」更新するので無限ループにならない。
  useEffect(() => {
    if (data.length === 0) {
      // data が空のときだけ設定
```

```
      setData(["A", "B", "C"]);
    }
  }, [data.length]);

  // 修正3: useMemo でオブジェクトの参照を安定させる
  // useMemo は「依存配列が変わるまで同じ値（参照）を返す」フック。
  //  （useMemo の使い方は 6.7 で解説しました）

  return <p>カウント: {count}</p>;
}
```

無限ループを防ぐチェックリスト:

1. `useEffect` の依存配列は省略していないか?
2. `useEffect` 内で依存配列に含まれる state を無条件に更新していないか?
3. 依存配列にオブジェクトリテラルや配列リテラルを直接書いていないか?
4. `useEffect` 内の関数が毎回再生成されていないか?

▼ コードを1つずつ分解して解説

解説1: NG — 依存配列を省略してstateを更新する

```
  useEffect(() => {
    // state 更新 → 再レンダリング → useEffect 再実行 → ...
    setCount(count + 1);
  // 依存配列がない！
  });
```

- 依存配列を**省略**したuseEffectは、再描画のたびに毎回実行されます。
- その中で `setCount` （state更新）を呼ぶと、「**state更新** → **再描画** → **またuseEffect** → **state更新** → ...」が永遠に続きます。
- これが無限ループの典型パターンです。アプリが固まります。

> **用語: 無限ループ** ... 処理が永遠にくり返されて止まらない状態。useEffectとstate更新の組み合わせで起きやすい。

解説2: NG — 依存配列に毎回新しい参照を入れる

```
useEffect(() => {
  // 毎回新しい配列オブジェクト → 再レンダリング → ...
  setData(["A", "B", "C"]);
// 依存配列がない！
});

useEffect(() => {
  console.log("実行");
// オブジェクトリテラルは毎回新しい参照 → 毎回実行
}, [{ key: "value" }]);
```

- `["A","B","C"]` や `{ key: "value" }` は、実行のたびに**新しい入れ物（参照）**として作られます。
- Reactは参照で比較するため、中身が同じでも「毎回変わった」とみなし、効果が止まりません。
- 依存配列にオブジェクト/配列リテラルを直接書くのは避けます。

> **用語: 参照の比較** … Reactは値の中身ではなく「同じ入れ物か」で変化を判定する。毎回新しく作ると常に「変化あり」になる。

解説3: OK — 適切な依存配列と条件・関数型更新

```
useEffect(() => {
  // 初回のみ実行
  setCount((prev) => prev + 1);
// 空配列 → 初回マウント時のみ
}, []);

useEffect(() => {
  if (data.length === 0) {
    // data が空のときだけ設定
    setData(["A", "B", "C"]);
  }
}, [data.length]);
```

- `[]`（空配列）にして「初回だけ」実行すれば、ループしません。`(prev) => prev + 1` の関数型更新で外部の `count` にも依存しません。
- `if (data.length === 0)` のように**条件を付ける**と、必要なときだけ更新されてループを防げます。
- 依存配列には `data.length`（変化を見たい値）を入れます。

> **用語: 関数型更新** … `setX((prev) => ...)` の形。直前の値を使うので、依存配列に値を入れずに済む。

8.3 key の重要性

`key` は React がリスト内の各要素を識別するために使います。正しい `key` を指定しないと、予期しないバグが発生します。

> **▼ このコードがやること（先に日本語で）**：2.5 の `map` で出てきた **`key` （各要素の目印）** を、なぜ正しく付けないといけないのかを、**バグが起きる例**で見せます。Reactはリストを更新するとき、`key` を手がかりに「どの項目が同じで、どれが増減したか」を見分けます。ここで**配列の並び順（index）を key に使う**と、項目を削除・並べ替えしたときにReactが取り違え、**入力中の文字が別の行に移る**などの不可解なバグが起きます。だから `key` には「**順番に左右されない固有のID（本やTODOの `id` ）**」を使う——これがこの節の結論です。

```tsx
import { useState } from "react";

type Item = {
  id: number;
  text: string;
};

function KeyExample() {
  const [items, setItems] = useState<Item[]>([
    { id: 1, text: "アイテム1" },
    { id: 2, text: "アイテム2" },
    { id: 3, text: "アイテム3" },
  ]);

  // 先頭に追加する
  const addToTop = () => {
    // Date.now() は現在時刻のミリ秒値。簡易的な一意ID生成に使う。
    const newItem: Item = {
      id: Date.now(),
      text: `アイテム${items.length + 1}`,
    };
    // [新規, ...既存] で先頭に挿入した新しい配列を作る
    setItems([newItem, ...items]);
  };

  return (
    <div>
      <button onClick={addToTop}>先頭に追加</button>

      <h3>NG: index を key に使用</h3>
      {/* 先頭に追加すると全要素の index がずれてしまい、React は
          「key=0 の中身が変わった」と解釈する。input の入力値(=非制御の DOM 状態)が
          別の要素に紐づいてしまう。 */}
      {items.map((item, index) => (
        <div key={index}>
          <span>{item.text}</span>
          {/* input の値が正しい要素に紐づかない！ */}
          <input type="text" placeholder="メモを入力" />
        </div>
      ))}

      <h3>OK: 一意な id を key に使用</h3>
      {/* item.id は要素固有なので、配列の中で順序が変わっても
          「この id の要素はこの DOM ノード」という対応が保たれる。 */}
      {items.map((item) => (
        <div key={item.id}>
          <span>{item.text}</span>
          <input type="text" placeholder="メモを入力" />
        </div>
```

```
      ))}
    </div>
  );
}
```

この結果を確認するには、以下の操作をしてみてください:

1. 「NG」セクションの各入力欄に「メモA」「メモB」「メモC」と入力する
2. 【先頭に追加】ボタンをクリックする
3. **NG の方**: 入力したメモの位置がずれる! 「メモA」が新しいアイテムの横に表示される
4. **OK の方**: 入力したメモは元のアイテムに正しく紐づいたまま

これは、`index` を key にすると、先頭に要素を挿入したとき全要素の index が変わり、React が要素の対応を正しく把握できなくなるためです。

▼ コードを1つずつ分解して解説

解説1: 先頭に項目を追加する処理

```
const addToTop = () => {
  const newItem: Item = {
    id: Date.now(),
    text: `アイテム${items.length + 1}`,
  };
  setItems([newItem, ...items]);
};
```

- `Date.now()` は現在時刻のミリ秒値で、簡易的な一意IDとして使っています（呼ぶたびに違う値）。
- `[newItem, ...items]` は「**新項目を先頭に、続けて既存の全要素**」を並べた新しい配列です。
- この「先頭への追加」が、key の付け方による違いが分かりやすく出る操作です。

用語: Date.now() … 現在時刻のミリ秒値を返す。簡易的な一意ID生成に使われることがある。

解説2: NG — index を key にすると取り違える

```
      {items.map((item, index) => (
        <div key={index}>
          <span>{item.text}</span>
          <input type="text" placeholder="メモを入力" />
        </div>
      ))}
```

- `key={index}`（配列の番号）を使うと、先頭に追加したとき**全要素の index がずれます**。
- Reactは key を頼りに要素を見分けるため、「key=0 の中身が変わった」と誤解し、入力欄の中身が別の行に紐づいてしまいます。
- 結果、**入力中の文字が別の行に移る**という不可解なバグが起きます。

> **用語: key** … リストの各要素を見分けるためのReactの目印。順番に左右されない固有の値を使うべき。

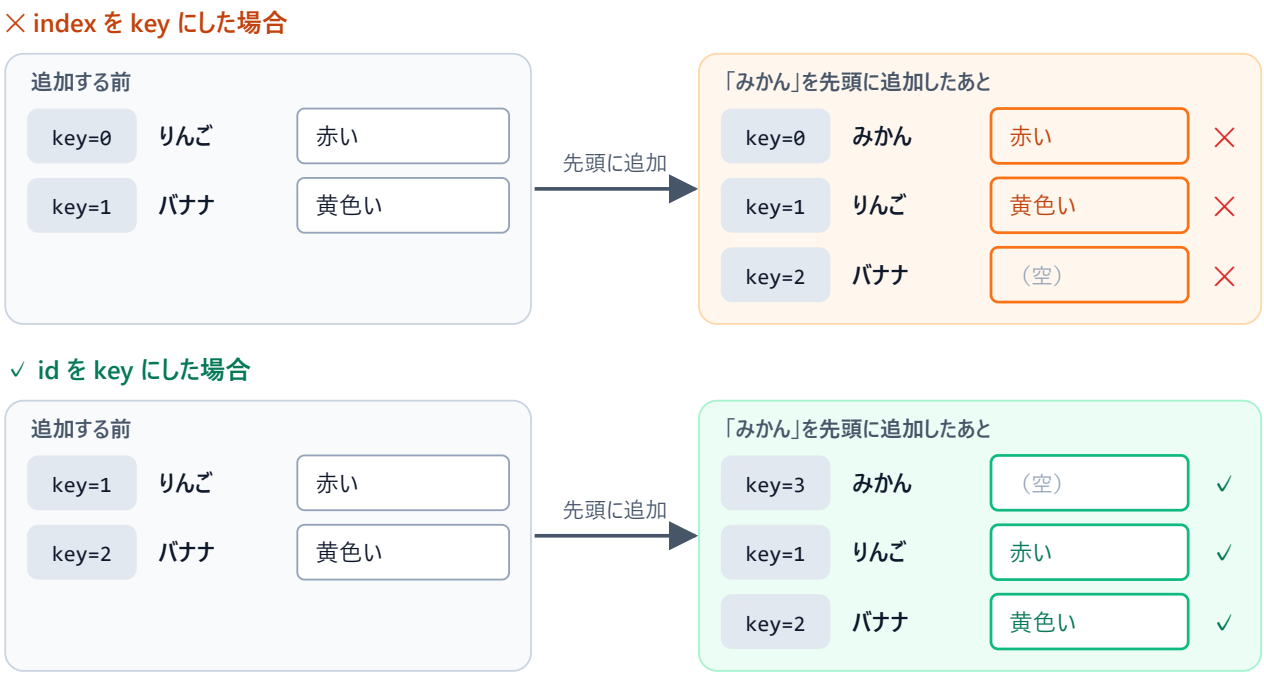
解説3: OK — 固有の id を key にする

```
{items.map((item) => (
  <div key={item.id}>
    <span>{item.text}</span>
    <input type="text" placeholder="メモを入力" />
  </div>
))}
```

- `key={item.id}` のように、**順番に左右されない固有のID**を使います。
- これなら並び順が変わっても「この id の要素はこのDOM」という対応が保たれ、入力欄の中身も正しい行に残ります。
- 結論：key には本やTODOの `id` のような**一意で変わらない値**を使うこと。

> **用語: 一意なID** … データごとに重複せず、並び替えても変わらない値（DBの主キーなど）。keyの理想的な値。

何が起きているのかを、Before / After の図で確認しましょう。key は、教室の座席表で使う「名札」のようなものです。**座席番号**（index） で生徒を覚えていると、先頭に転校生が座った瞬間に全員の番号が1つずつずれて、先生（React）は「1番の席の人＝さっきまで1番だった人」と思い込み、その人のノート（入力欄の中身）を別の生徒に渡してしまいます。**名札**（id） で覚えていれば、席が動いても取り違えません。



React は同じ key の行（DOM）を使い回す。index は「位置」なので、先頭に追加するとメモが別の果物に付いてしまう

図3-13: key の付け方によるリストの使い回しの違い（Before / After）。入力欄のメモは DOM が覚えている値です

key による「使い回し」を Node.js でまねしてみる

> **▼ このコードがやること（先に日本語で）**：React がリストを描き直すときの「同じ key の行（DOM）は使い回す」という動きを、小さな模型で再現します。「りんご（メモ: 赤い）」「バナナ（メモ: 黄色い）」が並んだ状態で先頭に「みかん」を追加し、①index を key にした場合と ②id を key にした場合で、各行のメモ欄がどうなるかを表示します。

```ts
// ファイル名: key-sim.ts
// 画面に並んでいる「行」1つ分（React が作った DOM の代わり）
type Row = {
  // この行に付いている key
  key: string;
  // 行の中の入力欄にユーザーが打った文字（DOM が覚えている状態）
  memo: string;
};

// 表示したいデータ1件分
type Fruit = {
  id: number;
  name: string;
};

// key の決め方（index を使うか、id を使うか）を関数で受け取る
type GetKey = (fruit: Fruit, index: number) => string;

// 新しいデータを描くとき、同じ key の行があればそれを使い回す（React のまね）
function rerender(prevRows: Row[], fruits: Fruit[], getKey: GetKey): Row[] {
  return fruits.map((fruit, index) => {
    const key = getKey(fruit, index);
    // 同じ key の行を探す。見つかればその行（メモ入り）をそのまま使う
    const reused = prevRows.find((row) => row.key === key);
    const memo = reused ? reused.memo : "";
    console.log(`  ${fruit.name}: key=${key} → メモ欄「${memo || "(空)"}」`);
    return { key, memo };
  });
}

const before: Fruit[] = [
  { id: 1, name: "りんご" },
  { id: 2, name: "バナナ" },
];
// 先頭に「みかん」を追加したあとのデータ
const after: Fruit[] = [{ id: 3, name: "みかん" }, ...before];

// ---- index を key にした場合 ----
const byIndex: GetKey = (_fruit, index) => String(index);
// 追加前の画面: りんごの行に「赤い」、バナナの行に「黄色い」とメモしてある
const indexRows: Row[] = [
  { key: "0", memo: "赤い" },
  { key: "1", memo: "黄色い" },
];
console.log("【index を key にした場合】");
rerender(indexRows, after, byIndex);

// ---- id を key にした場合 ----
const byId: GetKey = (fruit) => String(fruit.id);
```

```
const idRows: Row[] = [
  { key: "1", memo: "赤い" },
  { key: "2", memo: "黄色い" },
];
console.log("【id を key にした場合】");
rerender(idRows, after, byId);
```

▼ 実行結果（`node key-sim.ts`）

```
【index を key にした場合】
 みかん: key=0 → メモ欄「赤い」
 りんご: key=1 → メモ欄「黄色い」
 バナナ: key=2 → メモ欄「(空)」
【id を key にした場合】
 みかん: key=3 → メモ欄「(空)」
 りんご: key=1 → メモ欄「赤い」
 バナナ: key=2 → メモ欄「黄色い」
```

本物の React（React 19）で同じ操作をした場合も、入力欄の中身はこの模型とまったく同じ結果になりました。

▼ コードを1つずつ分解して解説

解説1: 同じ key の行を探して使い回す

```
const reused = prevRows.find((row) => row.key === key);
const memo = reused ? reused.memo : "";
```

- `find` は「条件に合う最初の要素」を返す配列のメソッドです（見つからなければ `undefined`）。
- React も、描き直す前と後で**同じ key の要素を「同じもの」とみなして DOM を使い回します。**入力欄に打った文字のように DOM 自身が覚えている値は、その DOM と一緒に引き継がれます。
- 同じ key が見つからなければ、新しい DOM（空のメモ欄）が作られます。

解説2: index だと「位置」で対応づけてしまう

```
const byIndex: GetKey = (_fruit, index) => String(index);
```

- index を key にすると、先頭に「みかん」が入った瞬間、「みかん」が key=0 になります。key=0 は追加前に「りんご（赤い）」の行だったので、**みかんの行に「赤い」が引き継がれて**しまいます。
- 引数名の先頭の `_`（`_fruit`）は「この引数は使わない」という印として、よく使われる書き方です。

解説3: id なら「データそのもの」で対応づけられる

```
const byId: GetKey = (fruit) => String(fruit.id);
```

- id はデータごとに決まっていて、並び順が変わっても変わりません。だから「りんご」は追加の前も後も key=1 で、メモ「赤い」が正しく残ります。
- 新しく来た「みかん」は key=3 で、前の画面には無かったので新しい空の行になります。これが期待どおりの動きです。

---

key のベストプラクティス:

```
// ベスト: データベースの ID を使う
// 配列内で一意かつ、再レンダリングしても変わらない値が理想。
{items.map((item) => <Item key={item.id} />)}

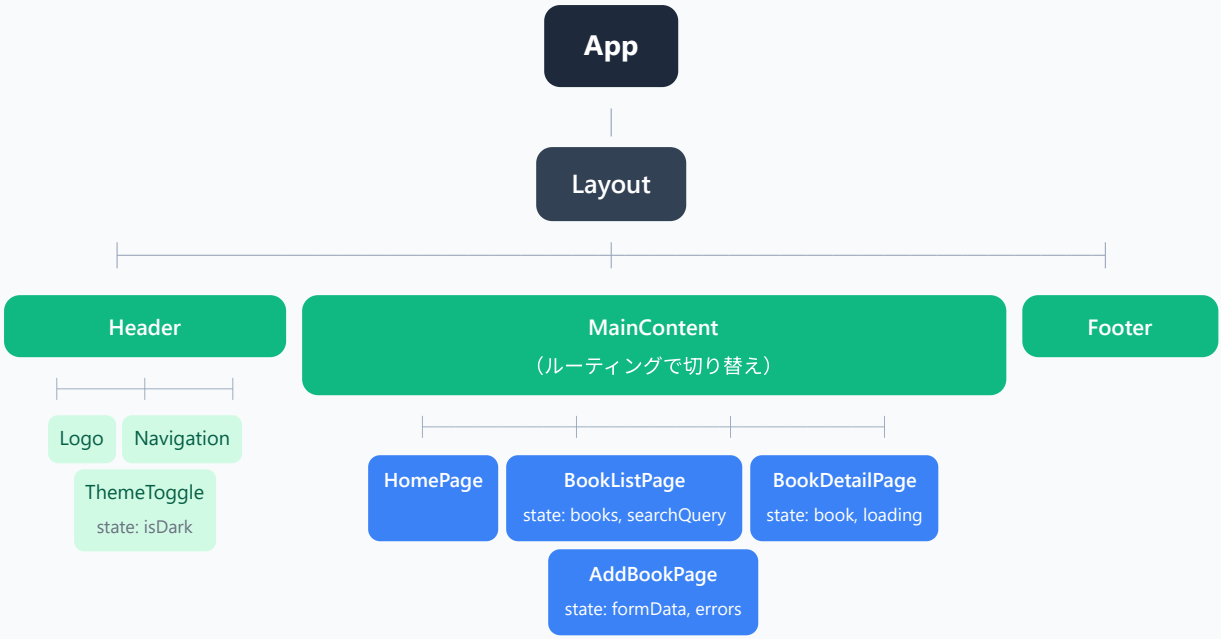
// OK: ユニークな文字列を使う
// slug 等の人間が読めるユニークIDでもOK。
{items.map((item) => <Item key={item.slug} />)}

// 最終手段: index を使う（並び替え・追加・削除がない静的リストのみ）
// 並びが固定で、要素が増減しないリストでは index でも問題ない。
{staticItems.map((item, index) => <Item key={index} />)}

// NG: ランダム値を使う（毎回変わるので意味がない）
// Math.random() は描画のたびに違う値を返す。key の意味（=同一性の判別）が成立しない。
// 全要素が毎回「別物」と判定され、入力欄の状態が失われる等の不具合の元。
{items.map((item) => <Item key={Math.random()} />)}
```

書籍管理アプリのコンポーネント設計

この章で学んだ知識を活かして、書籍管理アプリの全体像を確認しましょう。



ページ別の子コンポーネント

BookListPage の子コンポーネント

- SearchBar — props: onSearch / state: query
- FilterPanel — props: onFilter / state: selectedCategory
- BookGrid — props: books

↳ BookGrid の子:

- BookCard — props: book, onDelete, onToggleFavorite
- BookCard
- BookCard...

↳ BookCard の子:

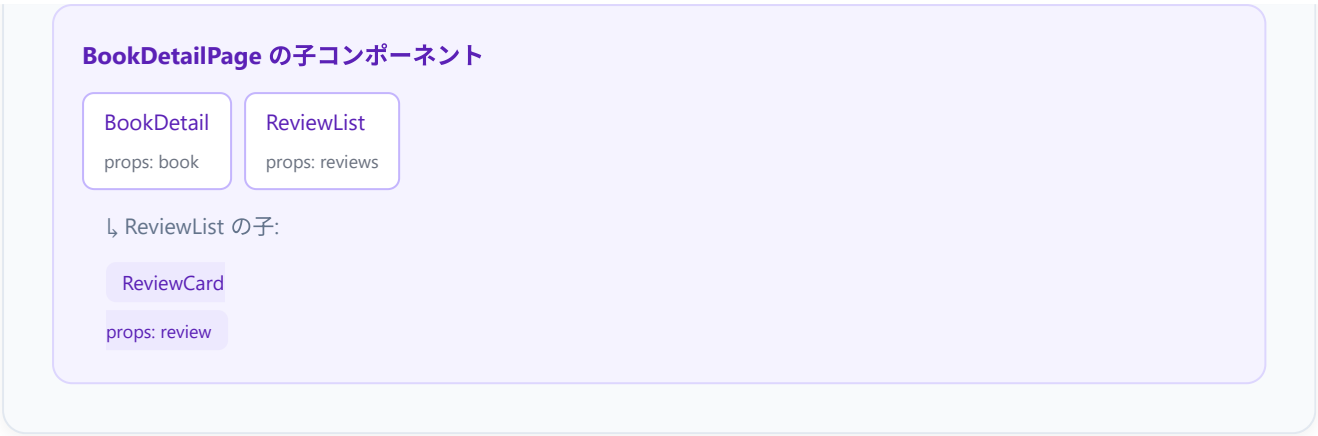
- BookImage — props: src, alt
- BookInfo — props: title, author, price
- BookActions — props: onEdit, onDelete
- RatingStars — props: rating

AddBookPage の子コンポーネント

- BookForm — props: onSubmit / state: formData, errors

↳ BookForm の子:

- FormInput — props: label, value, onChange, error
- FormSelect — props: options, value, onChange
- Button — props: label, onClick, variant



各コンポーネントにどの `state` が必要で、どの `props` を受け取るかが一目でわかります。後の章でこの設計に基づいて実装を進めていきます。

練習問題（解答・解説つき）

この章の内容を確かめる問題です。まず自分で考えて（できれば紙やエディタに書いて）から、後ろの「解答と解説」を見てください。答えを見る前に「なぜそうなるか」を一言で説明できれば、理解できている証拠です。

問題1: JSX のルール違反を直す（2節）

次のコンポーネントには、JSX のルール違反が3つあります。すべて直してください。

```
function Profile() {
  return (
    <h2 class="name">田中太郎</h2>
    <img src="/avatar.png" alt="アバター">
  );
}
```

問題2: props を受け取るコンポーネントを作る（3節）

`title`（文字列）と `price`（数値）を props で受け取り、`React入門（2,800円）` のように表示する `BookTitle` コンポーネントを、props の型定義つきで書いてください（価格は3桁ごとにカンマを入れます）。

問題3: 1回のクリックで count はいくつになる？（4節）

`count` の初期値が 0 のとき、次のボタンを**1回だけ**押すと、画面の `count` はいくつになるでしょう。

```
const handleClick = () => {
  setCount(count + 1);
  setCount(count + 1);
  setCount((prev) => prev + 1);
};
```

問題4: 削除ボタンのバグを2つ見つける（4〜5節）

次の TODO 削除のコードには、バグが2つあります。それぞれ何が起きるかを説明し、直してください。

```
const handleDelete = (id: number) => {
  const index = todos.findIndex((todo) => todo.id === id);
  todos.splice(index, 1);
  setTodos(todos);
};

// 一覧の中の削除ボタン
<button onClick={handleDelete(todo.id)}>削除</button>
```

問題5: ログの順番を答える（6節）

次の `CityLabel` を `city="東京"` で表示し、次に `city="大阪"` に変え、最後に画面から消しました。Console に出るログを順番に書いてください（Strict Mode なしの場合）。

```
function CityLabel({ city }: { city: string }) {
  console.log("A: レンダー", city);
  useEffect(() => {
    console.log("B: effect", city);
    return () => console.log("C: 片付け", city);
  }, [city]);
  return <p>{city}</p>;
}
```

問題6: key に一番ふさわしいのは？（2.5・8.3節）

本の一覧を `books.map((book, index) => <BookCard key={???} book={book} />)` で表示します。`???` に入れるものとして一番ふさわしいものを選び、ほかの2つがダメな理由も答えてください。

- (a) `index`
- (b) `Math.random()`
- (c) `book.id`

問題7: 文字が入力できない入力欄（5.2節）

次の入力欄は、キーボードで文字を打っても何も入力されません。理由を説明し、入力できるように直してください。

```
function NameInput() {
  const [name, setName] = useState("");
  return <input value={name} />;
}
```

解答と解説

問題1の解答

```
function Profile() {
  return (
    // (1) 複数の要素は Fragment（または div）で1つに包む
    <>
      {/* (2) class ではなく className */}
      <h2 className="name">田中太郎</h2>
      {/* (3) 中身のないタグは /> で閉じる */}
      <img src="/avatar.png" alt="アバター" />
    </>
  );
}
```

- (1) `return` で返せるのは1つの要素だけです（2.2「複数の要素を返す場合」）。元のコードでは `JSX expressions must have one parent element.` というエラーになります。
- (2) `class` は JavaScript の予約語なので、JSX では `className` と書きます。図3-3 で見たとおり、HTML になるときに `class` に戻ります。
- (3) JSX では `<img>` や `<input>` のような自己閉じタグも、必ず `/>` で閉じます。

問題2の解答

```tsx
// props の形を型で決める
type BookTitleProps = {
  title: string;
  price: number;
};

function BookTitle({ title, price }: BookTitleProps) {
  // toLocaleString() で 2800 を "2,800" にする
  return (
    <p>
      {title} ({price.toLocaleString()}円)
    </p>
  );
}

// 使い方: 文字列は "..."、数値は { } で渡す
<BookTitle title="React入門" price={2800} />
```

- props の型は `type ○○Props = { ... }` で決め、引数で `{ title, price }` と分割代入して受け取るのが定番です（3.2）。
- 数値の props は `price={2800}` のように `{ }` で渡します。`price="2800"` と書くと文字列になり、`Type 'string' is not assignable to type 'number'.` という型エラーになります。

問題3の解答

答え: 2

- この回のレンダーでは `count` は 0 のスナップショットなので、最初の2行はどちらも「1 にして」という予約です。3行目は「前の値 + 1」という関数の予約です。
- 予約の列は `[1, 1, (prev) => prev + 1]` になり、React は書いた順に「1 にする → 1 にする → 1 + 1 = 2」と処理します（4.3・図3-5）。
- 4.3 の `state-queue.ts` で、予約の列をこの3つにして `applyUpdates` を呼ぶと、同じく 2 が得られます。本物の React で試しても 2 になります。

問題4の解答

- **バグ1:** `onClick={handleDelete(todo.id)}` … `()` を付けているので、ボタンを**描画した瞬間に** `handleDelete` が実行されてしまいます。クリックしていないのに削除処理が走り、逆にクリックしても何も起きません（5.1・図3-7）。TypeScript では `Type 'void' is not assignable to type 'MouseEventHandler<HTMLButtonElement> | undefined'.` というエラーになります。

- **バグ2: `splice` で直接書き換えて同じ配列を渡している** … `splice` は元の配列を直接変更する破壊的メソッドです。`setTodos(todos)` に「同じ箱」を渡しているので、React は「変化なし」と判断し、画面が更新されません（4.4・図3-6、8.1）。

```
const handleDelete = (id: number) => {
  // filter で「削除する TODO 以外」を集めた新しい配列を作る
  setTodos(todos.filter((todo) => todo.id !== id));
};

// アロー関数で包んで「クリックされたら呼ぶ関数」を渡す
<button onClick={() => handleDelete(todo.id)}>削除</button>
```

問題5の解答

```
A: レンダー 東京
B: effect 東京
A: レンダー 大阪
C: 片付け 東京
B: effect 大阪
C: 片付け 大阪
```

- 最初の表示（マウント）では「レンダー → effect」の順です。effect は画面に反映されたあとに動きます。
- `city` が変わると「レンダー → **前回（東京）の片付け** → 新しい effect」の順になります。片付けの中の `city` は、その effect を作ったときの値（東京）のままです。
- 画面から消える（アンマウント）ときは、最後の片付け（大阪）だけが動きます。6.5 の図3-11・ログのサンプルと同じ流れです（このログは React 19 で実際に動かして確認したものです）。

問題6の解答

答え: (c) `book.id`

- (a) `index` … 並び替え・先頭への追加・削除をすると、key と本の対応がずれて、入力中の値などが別の本に付いてしまいます（8.3・図3-13）。本の一覧は追加・削除されるので不向きです。
- (b) `Math.random()` … 描画のたびに違う値になるので、毎回すべての行が「新しいもの」と判断され、作り直されます。入力中の文字が消えたり、ムダな処理が増えたりします。
- (c) `book.id` … データベースが付けた一意な番号で、並び順に関係なく変わりません。key の理想的な値です。

問題7の解答

- `value={name}` で入力欄の表示を state に固定しているのに、`onChange` が無いので state が変わりません。キーを押しても、次の描画で元の値（空文字）に戻されてしまいます（5.2・図3-8 の ③ が起きない）。Console には `You provided a \` value` prop to a form field without an `onChange` handler.` で始まる警告が出ます。

```tsx
function NameInput() {
  const [name, setName] = useState("");
  // value と onChange をセットで書く（制御コンポーネント）
  return <input value={name} onChange={(e) => setName(e.target.value)} />;
}
```

まとめ

この章で学んだ React の基礎をまとめます。

概念	要点
コンポーネント	UIの再利用可能な部品。関数として定義し、JSX を返す
JSX/TSX	JavaScript の中に HTML 風のコードを書ける構文
Props	親から子へデータを渡す仕組み。TypeScript で型を定義
State (useState)	コンポーネントが持つ変化するデータ。更新すると再レンダリング
イベントハンドリング	onClick, onChange, onSubmit でユーザー操作を処理
useEffect	副作用（API取得、DOM操作等）を実行するフック
カスタムフック	ロジックを再利用可能な関数に切り出す仕組み

次の章（第4章） では、この章で学んだ React の上に乗っている **Next.js** の基礎（App Router によるファイルベースのルーティング、Server Component と Client Component の使い分けなど）に進みます。